

Mensch-Maschine-Interaktion

Vorlesungsnotizen

Stand: 31. August 2026

Inhaltsverzeichnis

1	Kapitel A: Einführung	5
1.1	Was Mensch-Maschine-Interaktion ist	5
1.2	Benutzungsschnittstelle und Funktionalität	5
1.3	Beteiligte Disziplinen	6
1.4	Drei Begriffsebenen der Interaktion	6
1.5	Computerseitige Aspekte	6
1.6	Erweiterter Kontext und Ergonomie	7
1.7	Gliederung der Vorlesung	7
2	Kapitel B: Interaktive Systeme	8
2.1	Systemarchitektur: Architekturmodelle	8
2.1.1	Das Seeheim-Modell (Schichtenmodell)	8
2.1.2	Model-View-Controller (MVC)	9
2.1.3	Serviceorientierte Architektur (SOA) – Gerätemodell	10
2.2	Das Mensch-Maschine-Interaktionsmodell	11
2.2.1	Die Benutzerseite – Kognitiver Prozess des Menschen	11
2.2.2	Die Computerseite – Informationsverarbeitung des Systems	12
2.2.3	Kernbegriffe der Interaktion	12
2.3	Kommunikationskanäle	12
2.3.1	Die vier Grundkomponenten	13
2.3.2	Kombination mehrerer Kanäle	14
2.4	Interaktionsstile (Dialoggestaltung)	14
2.4.1	Sprachbasierte Interaktion	14
2.4.2	Direkte Manipulation (DMI)	15
2.4.3	Gestenbasierte Interaktion	16
2.5	Interaktionsumgebungen	17
3	Kapitel C: Wahrnehmung	17

3.1	Visuelle Wahrnehmung (Sehsinn)	17
3.1.1	Sinneszellen: Stäbchen und Zapfen	17
3.1.2	Anpassungsmechanismen: Akkommodation und Adaption	17
3.1.3	Präattentive und Attentive Wahrnehmung	18
3.2	Auditive Wahrnehmung (Hörsinn)	18
3.2.1	Auditory Icons vs. Earcons	19
3.3	Haptische Wahrnehmung (Tastsinn & Propriozeption)	19
4	Kapitel C.2: Farbmodelle	19
4.1	Physikalische Grundlagen der Farbwahrnehmung	19
4.2	RGB-Modell (Additiv)	20
4.3	CIE-Modell (Geräteunabhängiges Referenzmodell)	21
4.4	HSV- und HLS-Modelle (Nutzerorientiert)	22
4.5	CMYK-Modell (Subtraktiv)	24
5	Kapitel C.4: Diskretisierung	24
5.1	Mathematische Grundlagen: Komplexe Zahlen	25
5.2	Fouriertransformation (FT)	25
5.2.1	Eindimensionale FT	25
5.2.2	Rechtecksignal als Beispiel	27
5.2.3	Zweidimensionale FT (für Bilder)	28
5.2.4	Klausurrelevante Eigenschaften der FT	29
5.3	Dirac-Funktion und Dirac-Feld	29
5.4	Abtasttheorem (Whittaker-Shannon)	31
5.5	Quantisierung	33
5.6	Dithering	33
5.7	Signalverarbeitungspipeline	35
5.8	Lernkonzepte	36
6	D – Bilderzeugung	36
6.1	Daten-/Prozessmodell der Bilderzeugung	36
6.2	Bilderzeugungs-Pipeline	37
7	E – Computergraphik und OpenGL	58
7.1	Von der festen zur programmierbaren Pipeline	58
7.2	OpenGL als Schnittstelle	59

7.3	Die Koordinatentransformations-Pipeline im Shader	61
7.4	Szenengraphen	61
7.5	Strahlverfolgungsbasierte Bilderzeugung	62
8	F – Objektrepräsentation	63
8.1	Geometrie und Topologie	63
8.2	Geometrische Modellierungstechniken	63
8.3	Darstellung von Kurven und Flächen	64
8.4	Interpolation, Approximation, Extrapolation	65
8.5	Bézier-Kurven	65
8.6	B-Splines	67
8.7	Datenstrukturen für Netze	69
8.8	Simplex, Triangulierung und polyedrische Komplexe	70
8.9	Polygontriangulierung	71
8.10	Voronoi-Diagramm	71
8.11	Delaunay-Triangulierung	72
8.12	Beneath-Beyond-Algorithmus	72
8.13	Netzverbesserung	73
9	G – Interaktionsgestaltung	73
9.1	Interaktionsstile	73
9.2	Gestaltprinzipien der visuellen Wahrnehmung	74
9.3	Sensomotorische Designanforderungen	74
9.4	WIMP-Systeme	75
9.5	Zeigergeräte	76
9.6	Dialoge und Formulare	76
9.7	Fat Finger Problem	77
10	H – Virtuelle und Augmentierte Realität	77
10.1	Taxonomie: Extended Reality (XR)	77
10.2	Immersion, Präsenz und Illusion	78
10.3	VR-Lokomotion	79
10.4	Tracking und Detektion	79
10.5	Adaptives Streaming für XR	80
10.6	Interaktionsaufgaben in virtuellen 3D-Umgebungen	81

10.7 Haptische 3D-Umgebungen	81
11 I – Eingabesensorik	82
11.1 Sensoren und Eingabegeräte	83
11.2 Die Verarbeitungskette der digitalen Bildverarbeitung	84
11.3 Bildrestauration	84
11.4 Bildverbesserung	85
11.5 Segmentierung	86
11.6 Mustererkennung	87
12 J – UX/UI-Design und Entwicklungsprozesse	88
12.1 UX-Design: Grundbegriffe	88
12.2 Benutzergerechte Gestaltung	88
12.3 ISO 9241 – Ergonomie der Mensch-System-Interaktion	89
12.4 User Interface Engineering (UIE)	89
12.5 Design Thinking	90

1 Kapitel A: Einführung

1.1 Was Mensch-Maschine-Interaktion ist

Definition

Mensch-Computer-Interaktion befasst sich mit der **Konzeption, Bewertung und Implementierung** interaktiver Systeme für den menschlichen Gebrauch sowie mit der Untersuchung der wesentlichen Phänomene, die sie umgeben (Hewett et al., ACM SIGCHI).

Das Fach steht damit im Schnittpunkt von drei Größen, die alle drei veränderlich sind und die nur gemeinsam betrachtet werden können.

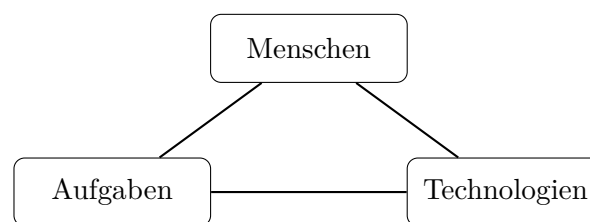


Abbildung 1: Das Spannungsdreieck der MMI

Die Rechnerleistung folgt dem Mooreschen Gesetz, die menschliche Wahrnehmung und Kognition dagegen der Evolution. Speicher, Geschwindigkeit, Portabilität und Bezahlbarkeit wachsen exponentiell; die Verarbeitungsgrenzen des Menschen bleiben konstant. Genau diese Schere ist der Gegenstand des Fachs: nicht was ein System berechnen kann, sondern was ein Mensch davon aufnehmen und steuern kann.

Hinweis

Warum das keine Randfrage ist, zeigen die Standardbeispiele der Vorlesung: der A320-Absturz in Bangalore (1990) und der Abschuss eines iranischen Passagierflugzeugs (Juli 1988). In beiden Fällen funktionierte die Technik wie spezifiziert, und die Schnittstelle vermittelte dem Bediener ein falsches Bild des Systemzustands.

1.2 Benutzungsschnittstelle und Funktionalität

Ein interaktives System steht zwischen einem **Benutzer** mit bestimmten Nutzungsbedingungen und technischer Qualifizierung und einer **Aufgabe** mit funktionalen Anforderungen und Anwendungsbedingungen. Die Software zerfällt in zwei Teile, die getrennt entworfen werden müssen: die Funktionalität, die die Aufgabe löst, und die Benutzungsschnittstelle, die sie zugänglich macht.

Der Umfang ist dabei kein Nebenschauplatz. Myers und Rosson stellten auf der CHI 1992 fest, dass der Anteil des Programmcodes für die Benutzungsschnittstelle 50 % des Gesamtcodes überschreitet.

1.3 Beteiligte Disziplinen

MMI ist keine Teildisziplin der Informatik, sondern ein Querschnittsfach.

Disziplin	Beitrag
Psychologie	Wahrnehmungs-, Kognitions-, Arbeitspsychologie, Physiologie
Informatik	Graphische Systeme, Software-Engineering, KI
Ingenieurwissenschaften	Elektrotechnik, Maschinenbau, Bauwesen und Architektur
Medizin	Belastung, Beanspruchung, Arbeitsschutz
BWL	Organisation, Wirtschaftlichkeit
Semiotik	Zeichen, Bedeutung, Metaphern

1.4 Drei Begriffsebenen der Interaktion

Die Vorlesung unterscheidet konsequent drei Ebenen, die im Sprachgebrauch häufig vermischt werden.

Definition

Interaktionsaufgabe: eine Aktion innerhalb einer Anwendung, unabhängig von ihrer Umsetzung. Beispiele: Objektauswahl, Transformation von Objekten, Eingabe von Texten.

Interaktionstechnik: eine konkrete Realisierung einer Interaktionsaufgabe. Beispiele für die Objektauswahl: Zeigen mit einem Zeigegerät, Eingabe des Objektnamens, Auswahl eines Eintrags in einer Liste.

Interaktionsstil (auch Dialogstil, Dialogform): das durchgängige Muster, nach dem Techniken gewählt werden. Beispiele: direkt-manipulativ, sprachbasiert, formularbasiert.

Die Trennung ist praktisch, nicht akademisch: dieselbe Aufgabe lässt sich durch verschiedene Techniken lösen, und die Wahl zwischen ihnen ist genau die Entwurfsentscheidung, die in den Kapiteln G und J bewertet wird.

1.5 Computerseitige Aspekte

Die Interaktion läuft über physiologische Kanäle, und jeder Kanal bringt sein eigenes Teilgebiet der Informatik mit.

Kanal	Ausgabe (Synthese)	Eingabe (Analyse)
visuell	Szenendarstellung, Bildsynthese (Kapitel D, E, F)	Signalanalyse von Bildern und Bewegung (Kapitel I)
auditiv	Tonsynthese, Sprachsynthese	Verarbeitung gesprochener Sprache
haptisch/motorisch	Kraftrückwirkung, mechanikbasierte Simulation	Robotik, Kollisionsbehandlung (Kapitel H)

Quer zu allen Kanälen liegen drei Aspekte: die **Hardware** der Ein- und Ausgabebege-
räte, die **Signalanalyse und -generierung**, und die **Software**, deren Aufgabe die

Abstraktion von konkreten Geräten zu geräteunabhängigen Bibliotheken und Programmierschnittstellen ist. Kapitel E behandelt mit OpenGL genau diese Abstraktionsschicht für den visuellen Kanal.

1.6 Erweiterter Kontext und Ergonomie

Ein interaktives System steht nicht allein zwischen Benutzer und Aufgabe, sondern in einem Umfeld aus Kollegen, Firma und Kunden. Aus diesem Umfeld kommen zwei weitere Anforderungsrichtungen: **Kommunikation und Kooperation** auf der Benutzerseite und **Organisation und Koordination** auf der Aufgabenseite. Die Passung zwischen technischem System und sozialem Umfeld heißt **sozio-technische Integration**.

Definition

Ergonomie ist die Wissenschaft von der Anpassung der Arbeitsbedingungen an den Menschen. In der MMI zerfällt sie in **Hardware-Ergonomie** (Geräte, Haltung, Beleuchtung) und **Software-Ergonomie** (Dialoggestaltung, Rückmeldung, Fehlertoleranz).

1.7 Gliederung der Vorlesung

Die zehn Kapitel teilen sich in benutzerseitige und computerseitige Aspekte.

Kapitel	Inhalt
A Einführung	Einordnung, Begriffe, Kontext
B Interaktive Systeme	Architekturmodelle, Kanäle, Interaktionsstile
C Sensorische Datenverarbeitung	Wahrnehmung, Farbmodelle, Diskretisierung
D Interaktive Bilderzeugung	Pipeline, Beleuchtung, Projektion, Rasterung
E Computergraphik und OpenGL	Graphikhardware, Szenengraph, Shader
F Geometrisches Modellieren	Kurven, Flächen, Netze, Triangulierung
G Interaktionssysteme	WIMP, sensomotorische Anforderungen, Gesten
H Virtuelle/Augmentierte Realität	XR-Taxonomie, Tracking, Kollision
I Eingabesensorik	Sensoren, Bildvorverarbeitung, Filter, Merkmale
J UX-Design	Prozesse, Evaluation, Gestaltungsrichtlinien

Kapitel A bis C und G, J behandeln überwiegend den Menschen, Kapitel D bis F, H und I überwiegend die Maschine. Die Zielsetzung ist in beiden Hälften dieselbe: Algorithmen, Systeme und die zugrundeliegende Theorie, die Programmierung und der Entwurf von Schnittstellen sowie Verfahren zu ihrer Analyse.

2 Kapitel B: Interaktive Systeme

2.1 Systemarchitektur: Architekturmodelle

Architekturmodelle dienen in der Mensch-Maschine-Interaktion dazu, die Struktur eines interaktiven Systems so zu organisieren, dass die Benutzungsschnittstelle (UI) und die Anwendungslogik sinnvoll getrennt werden.

Man unterteilt diese Modelle grundsätzlich in drei Kategorien:

1. **Schichtenmodelle** (Beispiel: Seeheim-Modell)
2. **Objektorientierte Modelle** (Beispiel: Model-View-Controller – MVC)
3. **Gerätemodelle** (Beispiel: Serviceorientierte Architektur)

2.1.1 Das Seeheim-Modell (Schichtenmodell)

Das Seeheim-Modell ist ein klassisches Beispiel für eine hierarchische Trennung in drei Schichten.

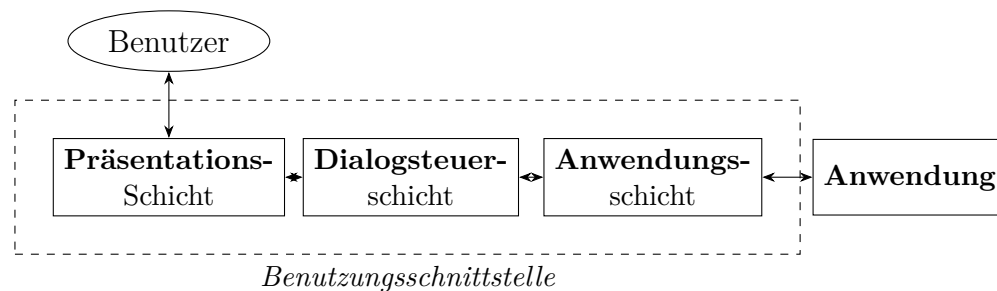


Abbildung 2: Das Seeheim-Modell

Die Schichten:

- **Präsentations-Schicht:** Direkt dem Benutzer zugewandt; beinhaltet die visuelle Darstellung sowie die physische Interaktion (Widgets wie Buttons, Menüs, Eingabefelder).
- **Dialogsteuer-Schicht:** Das „Gehirn“ bzw. „Gedächtnis“ des Dialogs; legt fest, wie das System auf Eingaben reagiert und in welchem Zustand sich die Interaktion befindet.
- **Anwendungs-Schicht:** Schnittstelle zur eigentlichen Fachlogik; definiert, welche Daten und Funktionen der Anwendung für die UI verfügbar sind.

Vorteile:

- **Portierbarkeit & Wiederverwendbarkeit:** UI lässt sich austauschen, ohne den Anwendungscode zu ändern.
- **Aufgabenteilung:** UI-Designer und Entwickler der Anwendungslogik können unabhängig voneinander arbeiten.

- **Rapid Prototyping:** Die Oberfläche lässt sich schnell verändern, da sie vom Kern getrennt ist.

Nachteile:

- **Leistungseinbußen („Sonderwege“):** Schnelle grafische Ausgaben (z. B. Echtzeit-Visualisierungen) müssen die Schichten oft umgehen, da der Weg durch den Dialog-controller zu langsam ist.
- **Fließende Grenzen:** In der Praxis ist eine strikte Grenze zwischen Präsentation und Steuerung oft schwer zu ziehen.
- **Komplexität des Kontrollflusses:** Keine klaren Vorgaben, wie Nachrichten zwischen den Schichten fließen (Prozeduraufruf vs. Nachrichtenaustausch).

2.1.2 Model-View-Controller (MVC)

Das heute am weitesten verbreitete Entwurfsmuster für moderne Benutzeroberflächen (Android, Java Swing, Qt).

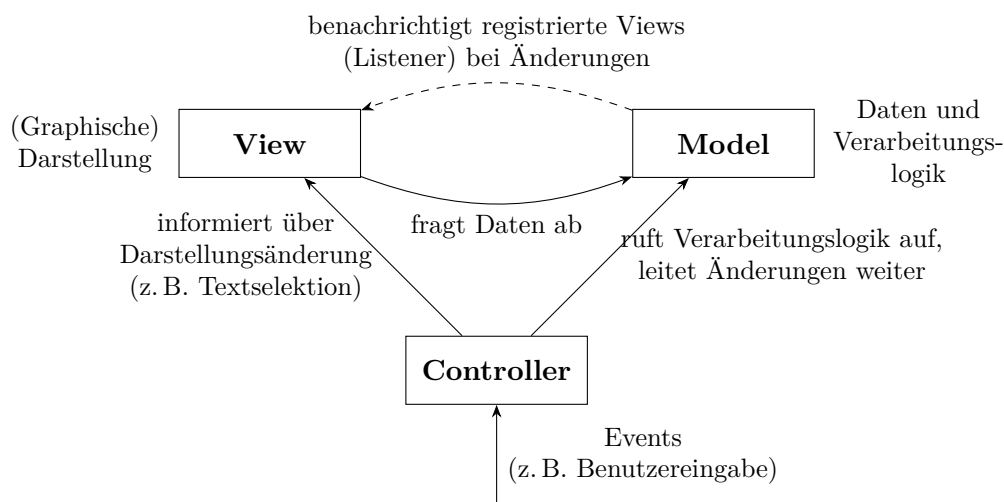


Abbildung 3: Model-View-Controller-Entwurfsmuster (MVC)

Die drei Komponenten:

- **Model:** Verwaltet Daten und Verarbeitungslogik; informiert registrierte Views über Änderungen.
- **View:** Grafische Darstellung; fragt Daten aktiv beim Model ab und reagiert auf Änderungsmitteilungen.
- **Controller:** Verarbeitet Events (Mausklicks, Tastatureingaben); ruft Logik im Model auf und leitet Änderungen an die View weiter.

Vorteile:

- **Mehrfache Ansichten:** Ein Model kann gleichzeitig in verschiedenen Views dargestellt werden (z. B. als Tabelle und als Diagramm).

- **Synchronisation:** Durch den Listener-Mechanismus bleiben alle Views automatisch aktuell.

Beispiel

In einem Musik-Player ist das *Model* die Playlist, der aktuelle Track und der Wiedergabestatus. Die *View* zeigt Titel, Fortschrittsbalken und Album-Cover an. Der *Controller* verarbeitet Eingaben wie Play / Pause oder Weiter.

Beispiele: Java AWT/Swing, Windows-UIs, Android-Apps, Qt-Framework.

2.1.3 Serviceorientierte Architektur (SOA) – Gerätemodell

SOA wird als Dreiecksbeziehung dargestellt und dient in der MMI als Gerätemodell: die Anwendung nutzt Hardware-Dienste (z. B. Grafik) über einen standardisierten Service-Kanal.

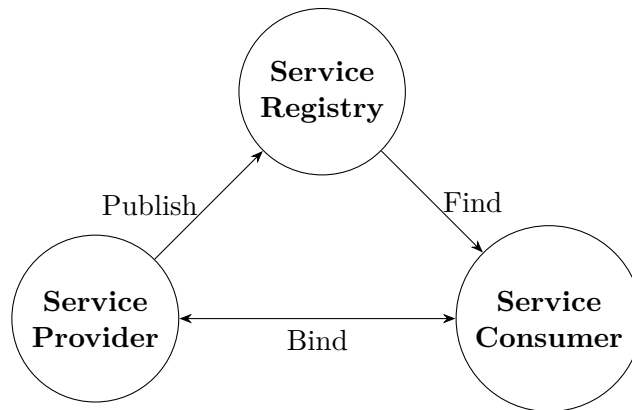


Abbildung 4: Serviceorientierte Architektur (SOA)

Die Akteure:

- **Service Provider:** Erstellt einen Dienst und stellt ihn bereit (*Publish*).
- **Service Registry:** Verzeichnis, in dem der Consumer passende Dienste sucht (*Find*).
- **Service Consumer:** Findet den Dienst und bindet ihn ein (*Bind*).

Beispiel

In der Computergraphik fungiert das Anwendungsprogramm als **Client** und nutzt die Dienste eines Graphiksystems (z. B. OpenGL) als **Server**.

Vorteile:

- **Lose Kopplung:** Provider und Consumer müssen sich nicht direkt kennen – die Vermittlung läuft über die Registry.
- **Austauschbarkeit:** Ein Dienst (z. B. ein Eingabegerät) kann gegen einen anderen ausgetauscht werden, ohne die Anwendung zu ändern.

- **Wiederverwendbarkeit:** Einmal registrierte Dienste können von mehreren Consumern gleichzeitig genutzt werden.

Nachteile:

- **Overhead durch Registry:** Jede Kommunikation läuft über eine zusätzliche Vermittlungsschicht, was Latenz erzeugen kann.
- **Komplexität:** Das Verwalten von Diensten (Publish, Find, Bind) erhöht den Implementierungsaufwand gegenüber direkten Aufrufen.
- **Single Point of Failure:** Fällt die Registry aus, können Consumer keine neuen Dienste mehr finden.

2.2 Das Mensch-Maschine-Interaktionsmodell

Dieses Modell beschreibt die Interaktion als einen **geschlossenen Regelkreis (Loop)** zwischen einem menschlichen Benutzer und einem Computersystem. Jede Aktion ruft eine Reaktion hervor; die Kommunikation findet über eine Veränderung der „Umwelt“ statt (z. B. was auf dem Bildschirm angezeigt wird).

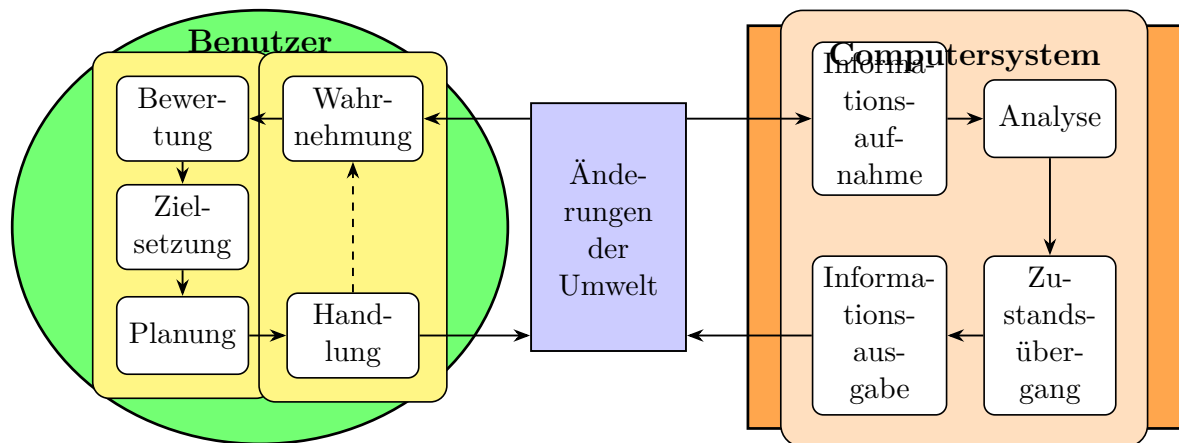


Abbildung 5: Mensch-Maschine-Interaktionsmodell

2.2.1 Die Benutzerseite – Kognitiver Prozess des Menschen

Der Mensch durchläuft intern vier Hauptphasen:

1. **Wahrnehmung (Perception):** Der Benutzer nimmt Signale der Maschine über seine Sinnesorgane (Augen, Ohren, Tastsinn) auf.
2. **Bewertung (Evaluation):** Das Wahrgenommene wird mit dem ursprünglichen Ziel verglichen. „Ist das passiert, was ich wollte?“
3. **Zielsetzung (Goal):** Basierend auf der Bewertung entscheidet der Benutzer, was als Nächstes zu tun ist.
4. **Planung & Handlung (Execution):** Der Benutzer plant die physische Bewegung und führt sie über sein motorisches System aus.

2.2.2 Die Computerseite – Informationsverarbeitung des Systems

Parallel dazu verarbeitet das System die Eingaben in vier Schritten:

1. **Informationsaufnahme:** Das System registriert Signale über **Sensoren** oder Eingabegeräte (Maus, Tastatur, Kamera, Mikrofon).
2. **Analyse:** Das System interpretiert die Daten – **Bedeutungsanalyse** oder Mustererkennung (z. B. „Der Klick erfolgte auf den Speichern-Button“).
3. **Zustandsübergang:** Die Programmlogik wird ausgeführt; der interne Zustand der Software ändert sich (z. B. Datei wird gespeichert, Objekt in einer 3D-Szene wird verschoben).
4. **Informationsausgabe:** Das System erzeugt Feedback über **Medien** (Grafik auf dem Display, Ton aus Lautsprechern).

2.2.3 Kernbegriffe der Interaktion

- **Interaktionsaufgabe (Interaction Task):** Eine *abstrakte* Aktion innerhalb einer Anwendung.
Beispiele: Objektauswahl, Text eingeben, Objekt im Raum transformieren (drehen / verschieben).
- **Interaktionstechnik (Interaction Technique):** Die *konkrete technische Realisierung* einer Interaktionsaufgabe.
Beispiele für „Objekt auswählen“:
 1. Mit der Maus auf ein Icon klicken.
 2. Den Namen des Objekts per Tastatur tippen.
 3. In einer VR-Umgebung mit einem „virtuellen Laserstrahl“ zeigen.
- **Interaktionsstil (Dialogstil):** Legt fest, in welcher *Form* der Dialog zwischen Mensch und Maschine gestaltet ist (siehe B.3).
- **Interaktionsumgebung:** Die technische Umgebung, in der Aufgaben, Techniken und Stile umgesetzt werden (siehe B.4).

2.3 Kommunikationskanäle

Dieses Thema befasst sich damit, wie Informationen physisch zwischen Mensch und Maschine übertragen werden. Man unterscheidet strikt zwischen der **Sender-** und der **Empfängerseite**.

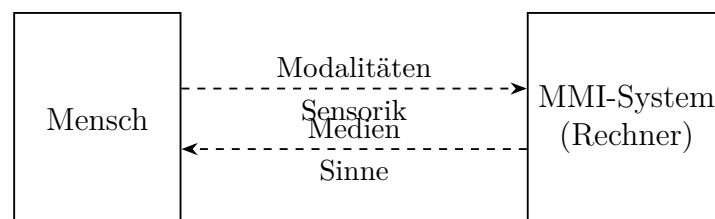


Abbildung 6: Kommunikationskanäle zwischen Mensch und MMI-System

2.3.1 Die vier Grundkomponenten

- **Modalität** (Mensch → Maschine): Typ des Ausgabesignals, den der **Mensch** erzeugt.
Beispiele: Sprache, Töne, Gestik, Mimik, Körperbewegungen, Biosignale (Gehirnströme).
- **Sensorik** (Aufnahme durch die Maschine): Technische Kanäle, mit denen das **System** Signale des Menschen aufnimmt.
Beispiele: Mikrofone, Kameras, Touchscreens, elektrochemische Sensoren, elektromagnetische Antennen.
- **Medien** (Maschine → Mensch): Typ des Ausgabesignals, den die **Maschine** erzeugt.
Beispiele: Text, Grafik (Standbild/Animation), Sprachsynthese, künstlicher oder realistischer Ton.
- **Sinne** (Aufnahme durch den Menschen): Menschliche Sinnesorgane, die Signale der Maschine registrieren.
Beispiele: Sehen (visuell), Hören (auditiv), Tasten (haptisch/taktil), Riechen (olfaktorisch), Schmecken (gustatorisch).

Hinweis

Schema für die Klausur:

Mensch (Modalität) → Sensorik → Rechner → Medium → Sinne → Mensch

Beispiele für Systeme mit allen vier Kanälen:

- **Sprachassistent (z. B. Alexa):**
 - *Modalität:* Mensch erzeugt **Sprache**
 - *Sensorik:* System nimmt Signal über **Mikrofon** auf
 - *Medien:* System gibt **Sprachsynthese** und **Text auf Display** aus
 - *Sinne:* Mensch registriert über **Hören (auditiv)** und **Sehen (visuell)**
- **Navigationssystem im Auto:**
 - *Modalität:* Mensch erzeugt **Sprache** (Sprachbefehl) und **Gestik** (Tippen auf Touchscreen)
 - *Sensorik:* System nimmt Signale über **Mikrofon** und **Touchscreen** auf
 - *Medien:* System gibt **Grafik** (Karte) und **Sprachsynthese** aus
 - *Sinne:* Mensch registriert über **Sehen (visuell)** und **Hören (auditiv)**

2.3.2 Kombination mehrerer Kanäle

Begriff	Erklärung	Beispiel
Multimedial	Mehr als ein Medium (Ausgabeseite des Rechners)	Lernsystem mit Text, Bild und Video
Multimodal	Mehr als eine Modalität (Eingabeseite des Menschen)	Sprechen + Zeigen: „Lösche dieses Objekt“ + Fingerzeig
Multicodal	Verschiedene Kodierungen für dasselbe Medium	„STOPP“ als Wort + rotes Achteck-Symbol

Multimediale Interaktion:

- **Vorteil:** Informationen über mehrere Kanäle (Hören & Sehen) werden besser aufgenommen und behalten.
- **Nachteil:** Gefahr der **Informationsüberlastung** – schwierig, die Aufmerksamkeit auf das Wesentliche zu lenken.

Multimodale Interaktion:

- **Vorteil: Höhere Robustheit** – bei Umgebungslärm kann das System die Absicht durch Gestik erkennen, auch wenn die Sprache nicht verstanden wird.
- **Nachteil: Technische Komplexität** – das System muss verschiedene Eingabeströme (z. B. Lippenbewegung + Ton) in Echtzeit synchronisieren und verschmelzen.

2.4 Interaktionsstile (Dialoggestaltung)

Ein **Interaktionsstil** (auch Dialogstil oder Dialogform) legt fest, wie der Austausch zwischen Mensch und Maschine strukturell gestaltet ist. Es werden drei Hauptstile unterschieden: **sprachbasiert**, **direkt-manipulativ** und **gestenbasiert**.

2.4.1 Sprachbasierte Interaktion

Nutzt geschriebene oder gesprochene Sprache zur Kommunikation. Zwei Varianten:

A. Kommandosprachen (CLI) – synthetische Sprachen mit strenger Syntax.

Merkmale: Zeichen, Abkürzungen und Wörter nach einer festen Grammatik.

Beispiele: Linux-Terminal (`ls -al`), Windows CMD, SQL-Abfragen.

Zielgruppe: Professionelle Nutzer und Administratoren.

- **Vorteile:**
 - **Effizienz:** Erfahrene Nutzer lösen komplexe Aufgaben sehr schnell.
 - **Skriptfähigkeit:** Erlaubt Makros und Automatisierungen.
 - **Direkter Zugriff:** Tiefe Systemfunktionen ohne grafische Umwege.
- **Nachteile:**
 - **Hoher Lernaufwand:** Befehle müssen auswendig gelernt werden (*Recall* statt *Recognition*).

- **Fehleranfälligkeit:** Ein Tippfehler führt oft zu Fehlern; Hilfestellungen sind schwer zu geben.

B. Natürliche Sprache – Interaktion wie im Alltag.

Beispiele: Sprachassistenten (Alexa, Siri), Chatbots (ChatGPT), Auskunftssysteme der Bahn.

- **Vorteile:**
 - **Geringer Lernaufwand:** Keine speziellen Befehle nötig.
 - **Barrierefreiheit:** Ideal, wenn die Hände beschäftigt sind.
- **Nachteile:**
 - **Ineffizienz:** Natürliche Sprache ist redundant.
 - **Mehrdeutigkeit:** Wörter können verschiedene Bedeutungen haben.
Beispiel: „Wir fliegen nach Hamburg“ vs. „Der Raum ist voller Fliegen“.

Techniken der sprachbasierten Interaktion:

- **Eingabe:** Tastatur, Handschrift, Spracheingabe, Gebärdenspracheingabe.
- **Ausgabe:** Text auf dem Display, Sprachsynthese, Gebärdensprachausgabe.

2.4.2 Direkte Manipulation (DMI)

Objekte auf dem Bildschirm werden direkt mit einem Eingabegerät (Maus, Finger) angefasst und verändert.

Zentrale Charakteristika:

1. **Sichtbarkeit** aller relevanten Objekte und Aktionen.
2. **Schnelle, umkehrbare Aktionen:** Änderungen sind schrittweise ausführbar und können sofort rückgängig gemacht werden.
3. **Visuelle Manipulation:** Ersetzen von Textbefehlen durch Ziehen, Klicken oder Verschieben.

Das WYSIWYG-Prinzip: „What you see is what you get“ – das Dokument wird am Bildschirm exakt so dargestellt, wie es später aussieht.

- *WYSIWYG:* Microsoft Word.
- *Nicht-WYSIWYG:* $\text{\LaTeX}$ (Code wird kompiliert, Ergebnis erst danach sichtbar).

Metaphern – Konzepte aus der physischen Welt für intuitive Bedienung:

- **Desktop-Metapher:** Bildschirm als Schreibtisch mit Ordnern und Papierkorb.
- **Buch-Metapher:** Umblättern von Seiten in einer App.

Vorteile:

- Intuitive Bedienung; Fehler sofort sichtbar und korrigierbar.
- Weitgehend sprachunabhängig.

Nachteile:

- Platzraubend auf dem Display.
- Icons können verwechselt werden.
- Geringere Präzision (vgl. „Fat Finger Problem“).

2.4.3 Gestenbasierte Interaktion

Interaktion durch Bewegungen des Körpers oder einzelner Körperteile (Finger, Hände, Augen).

Geste vs. Pose:

- **Geste (Bewegt-Geste):** Dynamische Bewegung mit kommunikativer Absicht (z. B. Winken).
- **Pose (Statische Geste):** Symbolhafte Haltung ohne Bewegung (z. B. „Daumen hoch“).

Die 5 Gestenstile:

1. **Deiktische Gesten:** Zeige- oder Verweisgesten zur Auswahl von Objekten (besonders in VR oder am interaktiven Tisch).
2. **Manipulative Gesten:** Direkte Steuerung von Objekten.
Beispiel: Pinch-to-zoom (Zwei-Finger-Spreizgeste) auf dem iPad.
3. **Symbolische Gesten:** Kodieren einen festen Befehl.
Beispiel: „V“-Zeichen mit den Fingern.
4. **Sprachbegleitende Gesten:** Co-verbale Bewegungen während des Redens. Technisch schwer zu erfassen (Echtzeit-Synchronisation mit Sprache erforderlich).
5. **Zeichen-/Gebärdensprache:** Komplexe lexikalische Sprachen mit eigener Grammatik aus Sequenzen von Gesten.

Stil	Hauptmerkmal	Beispiel
Sprachbasiert (CLI)	Effizienz durch Textbefehle	Linux Terminal (ls)
Sprachbasiert (Natürl.)	Intuition durch Alltagssprache	Siri / Alexa
Direkte Manipulation	Visuelles Handeln (Metaphern)	Drag & Drop am Desktop
Gestenbasiert	Körperbewegung als Eingabe	Zoom-Geste am Smartphone

2.5 Interaktionsumgebungen

Interaktionsumgebungen sind die technischen Umgebungen, in denen Aufgaben, Techniken und Stile umgesetzt werden.

- **CLI (Command Line Interface):** Textbasiert, streng reglementiert und kodiert.
- **GUI (Graphical User Interface):** Nutzt Metaphern (z. B. den „Schreibtisch“) und grafische Widgets.
- **WIMP-Systeme:** Unterform der GUI – bestehend aus **W**indows, **I**cons, **M**enus und **P**ointers.
- **NUI (Natural User Interface):** Ermöglicht „natürliche“ Interaktionen, z. B. durch Berührung (iPad) oder Sprache.
- **TUI (Tangible User Interface):** Greifbare Schnittstellen, bei denen physische Objekte als Eingabemedien dienen.
- **BCI (Brain Computer Interface):** Direkte Steuerung durch Gehirnströme (Biosignale).

3 Kapitel C: Wahrnehmung

Die Wahrnehmung ist der Prozess, bei dem Reize aus der Umwelt über die Sinnesorgane aufgenommen und im Gehirn verarbeitet werden. Für die MMI ist dieses Kapitel essenziell, um zu verstehen, wie Informationen präsentiert werden müssen, damit der Mensch sie optimal aufnehmen kann.

3.1 Visuelle Wahrnehmung (Sehsinn)

Der dominante Wahrnehmungskanal in der MMI.

3.1.1 Sinneszellen: Stäbchen und Zapfen

- **Stäbchen:** Zuständig für das **Hell-Dunkel-Sehen**. Extrem lichtempfindlich, ermöglichen Sehen bei Dämmerung – können aber keine Farben unterscheiden.
- **Zapfen:** Zuständig für das **Farbsehen**. Drei Typen (für verschiedene Wellenlängen); benötigen mehr Licht als Stäbchen.

3.1.2 Anpassungsmechanismen: Akkommodation und Adaption

Akkommodation (Fokussierung):

- **Definition:** Anpassung der **Brechkraft** der Linse.
- **Zweck:** Objekte in beliebigen Abständen scharf auf der Netzhaut abbilden.
- **Auslöser:** Änderung der Entfernung zum betrachteten Objekt.

Was ist Brechkraft der Linse?

Die Linse im Auge ist elastisch und kann ihre Form verändern. Je stärker sie gewölbt ist, desto stärker bricht (bündelt) sie die Lichtstrahlen – das ist eine hohe Brechkraft. Ziel ist immer, die Strahlen **genau auf der Retina** zu fokussieren:

- **Fernes Objekt** → Strahlen fast parallel → Linse flach (geringe Brechkraft).
- **Nahes Objekt** → Strahlen divergieren stark → Linse stark gewölbt (hohe Brechkraft).

Akkommodation = der Vorgang, bei dem die Augenmuskeln die Linsenform automatisch anpassen.

Hinweis

VR/AR-Relevanz: Vergenz-Akkommodations-Konflikt – wenn virtuelle Tiefe nicht mit der physischen Akkommodation übereinstimmt.

Adaption (Helligkeitsanpassung):

- **Definition:** Anpassung der **Pupillenweite** an die Lichtintensität (analog zur Blende einer Kamera).
- Viel Licht → Pupille klein (Kontraktion).
- Wenig Licht → Pupille groß (Dilatation).

Merkmal	Akkommodation	Adaption
Was wird angepasst?	Brechkraft (Linse)	Pupillenweite
Ziel	Schärfe bei variabler Distanz	Lichteinfall regulieren
Auslöser	Entfernung zum Objekt	Lichtintensität

3.1.3 Präattentive und Attentive Wahrnehmung

- **Präattentive Wahrnehmung: Vorbewusste, unterschwellige Wahrnehmung.** Läuft parallel ab und erkennt einfache Merkmale (Farbe, Form, Bewegung) sofort – ohne bewusste Aufmerksamkeit.
Beispiel: Ein roter Punkt in einer Menge blauer Punkte fällt sofort auf.
- **Attentive Wahrnehmung: Bewusste Erkennung und Einordnung komplexer Strukturen.** Erfordert Aufmerksamkeit und sequenziellen Fokus.

3.2 Auditive Wahrnehmung (Hörsinn)

Wird zur Entlastung des visuellen Kanals oder für Alarmsignale genutzt.

- Unterscheidung zwischen **Hörschwelle** (leisester Ton) und **Schmerzgrenze** (lautester Ton).
- **Echoisches Gedächtnis:** Kurzzwischenspeicher für auditive Reize – Kapazität ca. 5 Einheiten, Dauer 3–4 Sekunden.

3.2.1 Auditory Icons vs. Earcons

- **Auditory Icons:** Nutzen **natürliche Geräusche** als direkte Analogie zu Objekten oder Aktionen.
Beispiel: Papierknüllen für „Löschen“, Glockenläuten für neue Nachricht.
- **Earcons:** Nutzen **abstrakte Tonfolgen oder Melodien**. Bedeutung ist nicht intuitiv – **muss gelernt werden**. Können hierarchisch strukturiert sein.
Beispiel: Verschiedene Pieptöne für Systemfehlerklassen („OS Error“ vs. „Execution Error“).

Merkmal	Auditory Icons	Earcons
Natur	Natürliche Geräusche	Abstrakte Melodien / Rhythmen
Bedeutung	Intuitiv (analog zur Realität)	Abstrakt (muss gelernt werden)
Lernaufwand	Gering	Hoch

3.3 Haptische Wahrnehmung (Tastsinn & Propriozeption)

Gewinnt in VR und bei mobilen Geräten (Vibration) zunehmend an Bedeutung.

- **Propriozeption** („Eigenwahrnehmung“): Wahrnehmung der eigenen **Körperlage im Raum** und der Stellung der Gelenke zueinander – man weiß z. B. auch mit geschlossenen Augen, wo sich die eigene Hand befindet.
- **Kinästhesie** („Bewegungssinn“): Empfindung für **Winkeländerungen in den Gelenken** und deren Richtung – man spürt, ob und in welche Richtung sich ein Gelenk gerade bewegt, unabhängig vom Sehsinn.
- **Homunculus:** Die Sinneszellen sind im Gehirn ungleich verteilt. Hände, Zunge und Lippen belegen einen viel größeren Bereich im sensorischen Cortex als z. B. der Rücken – je mehr Rezeptoren eine Körperstelle hat, desto größer ist ihre „Fläche“ im Gehirn.

4 Kapitel C.2: Farbmodelle

Farbmodelle sind mathematische Beschreibungen, um Farben durch Zahlenwerte zu repräsentieren. In der MMI entscheidend für Erzeugung, Übertragung und Anzeige von Bildern auf verschiedenen Ausgabemedien.

4.1 Physikalische Grundlagen der Farbwahrnehmung

Der menschliche Farbeindruck ist das Ergebnis einer Interaktion zwischen Licht, Materie und dem biologischen Sehapparat.

- **Sichtbares Spektrum:** ca. **400 nm (Violett)** bis ca. **700 nm (Rot)**.
- **Drei Faktoren des Farbeindrucks:**

1. **Spektralverteilung der Lichtquelle** $E(\lambda)$: Energieverteilung des einfallenden Lichts.
2. **Reflektanz des Objekts** $\rho(\lambda)$: Anteil des reflektierten Lichts. Reflektiertes Licht:

$$L(\lambda) = E(\lambda) \cdot \rho(\lambda)$$

3. **Beobachter**: Farbwahrnehmung p durch die spektrale Absorptionskurve $\sigma(\lambda)$ der Netzhautzellen:

$$p = \int \sigma(\lambda) L(\lambda) d\lambda$$

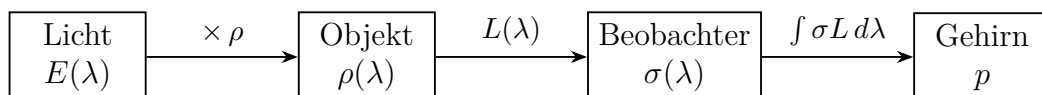


Abbildung 7: Sensorische Farbwahrnehmung: Licht $\rightarrow$ Objekt $\rightarrow$ Auge $\rightarrow$ Gehirn

Hinweis

Dreifarbenlehre: Der Mensch besitzt drei Zapfentypen (**S**hort, **M**edium, **L**ong) für Blau, Grün und Rot. Ein Farbsystem benötigt daher mindestens **drei Grundtöne**.

4.2 RGB-Modell (Additiv)

Additives Farbmodell – basiert auf der Mischung von farbigem Licht.

- **Prinzip:** Farben als gewichtete Summe von Rot, Grün, Blau. Maximale Intensität aller drei $\rightarrow$ **Weiß**.
- **Darstellung:** Einheitswürfel (Color Cube); Achsen R, G, B von 0 bis 1 (bzw. 0 bis 255).

Probleme:

- **Geräteabhängigkeit:** Farbton hängt von den physikalischen Grundfarben des jeweiligen Monitors ab.
- **Mangelnde Intuition:** Schwierig, eine Wunschfarbe durch R-G-B-Anteile zu schätzen.
- **Eingeschränkter Farbraum:** Nicht alle wahrnehmbaren Farben lassen sich durch additive Mischung dreier realer Grundfarben erzeugen.

Darstellung: Vektor (R, G, B) mit $R, G, B \in [0, 1]$ (bzw. 0–255 als 8-Bit-Wert).

Charakteristische Punkte:

- **Schwarzpunkt:** $(0, 0, 0)$ – kein Licht, kein Farbanteil.
- **Weißpunkt:** $(1, 1, 1)$ bzw. $(255, 255, 255)$ – maximale Intensität aller drei Kanäle.

- **Grau:** Hauptdiagonale des Würfels: (α, α, α) mit $\alpha \in [0, 1]$ – gleiche Intensität in allen Kanälen.

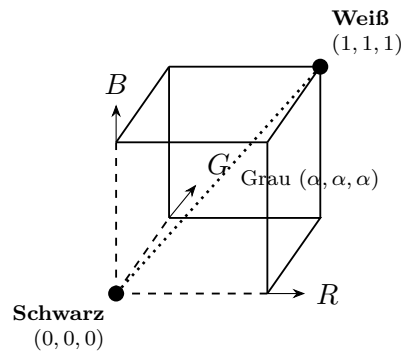


Abbildung 8: RGB-Einheitswürfel: Schwarzpunkt $(0, 0, 0)$, Weißpunkt $(1, 1, 1)$, Grau auf der Hauptdiagonale.

4.3 CIE-Modell (Geräteunabhängiges Referenzmodell)

Definiert von der *Commission Internationale de l'Éclairage* – löst das Problem der Geräteabhängigkeit durch **theoretische** Komponenten.

Komponenten X, Y, Z (statt R, G, B):

$$X = k \int P(\lambda) \bar{x}(\lambda) d\lambda \quad Y = k \int P(\lambda) \bar{y}(\lambda) d\lambda \quad Z = k \int P(\lambda) \bar{z}(\lambda) d\lambda$$

- **Y-Komponente** entspricht exakt der **Helligkeitsempfindlichkeit** des menschlichen Auges.
- **Normalisierung** (Chromatizitätskoordinaten):

$$x = \frac{X}{X + Y + Z} \quad y = \frac{Y}{X + Y + Z} \quad z = \frac{Z}{X + Y + Z}$$

Da $x + y + z = 1$, reichen **zwei Werte** (x und y) zur Beschreibung des Farbtons.

- **CIE-Farbtondiagramm:** Typische „Schuhsohlen-Form“. Der Rand = reine Spektralfarben. Der Gamut eines Monitors wird als Dreieck eingezeichnet.
- **sRGB:** Industriestandard (HP & Microsoft) auf CIE-Basis mit **D65-Weißpunkt** als Referenz.

Messinstrumente:

- **Colorimeter:** Misst X, Y, Z .
- **Spektroradiometer:** Misst X, Y, Z + das gesamte Spektrum.
- **Fotometer:** Misst ausschließlich die Helligkeit (Y).

Darstellung: 3D-Vektor (X, Y, Z) ; nach Normalisierung zweidimensionale Chromatizitätskoordinaten (x, y) plus Helligkeit Y (xyY-Modell).

Charakteristische Punkte:

- **Schwarzpunkt:** $(X, Y, Z) = (0, 0, 0)$ – kein Licht; im Chromatizitätsdiagramm undefiniert ($Y = 0$).
- **Weißpunkt (D65):** $(x, y) = (0,3127, 0,3290)$, $Y = 1,0$ – Norm-Weißpunkt nach sRGB-Standard.
- **Grau:** Gleiche Chromatizität wie der Weißpunkt ((x, y) konstant), nur Y variiert zwischen 0 und 1 – Grau liegt auf der **Neutralachse** im xy -Diagramm.

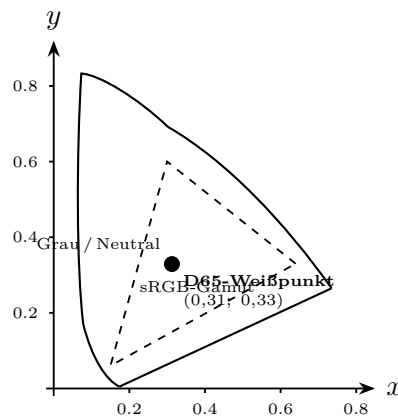


Abbildung 9: CIE-Chromatizitätsdiagramm (schematisch): Spektrallokus, D65-Weißpunkt und sRGB-Gamut-Dreieck. Schwarz ($Y = 0$) liegt außerhalb der xy -Ebene.

4.4 HSV- und HLS-Modelle (Nutzerorientiert)

Intuitiver für die Farbauswahl als RGB.

HSV-Parameter:

- **H (Hue / Farbton):** Grundfarbton als Winkel 0–360°.
- **S (Saturation / Sättigung):** Weißanteil; Reinheit der Farbe (0–1).
- **V (Value / Hellwert):** Schwarzanteil; Helligkeit (0–1).

HLS: Ähnlich aufgebaut, mit **Lightness** anstelle von Value.

Hinweis

Die mathematische Umrechnung zwischen HLS/HSV und RGB ist **nicht** prüfungsrelevant.

Darstellung: Vektor (H, S, V) mit $H \in [0^\circ, 360^\circ]$, $S \in [0, 1]$, $V \in [0, 1]$.

Charakteristische Punkte:

- **Schwarzpunkt:** $V = 0$ (Kegelspitze) – kein Licht, H und S irrelevant.
- **Weißpunkt:** $S = 0$, $V = 1$ (Mittelpunkt des Deckels) – kein Farbanteil, maximale Helligkeit.
- **Grau:** $S = 0$, $V \in (0, 1)$ – Mittelachse des Kegels; H irrelevant.

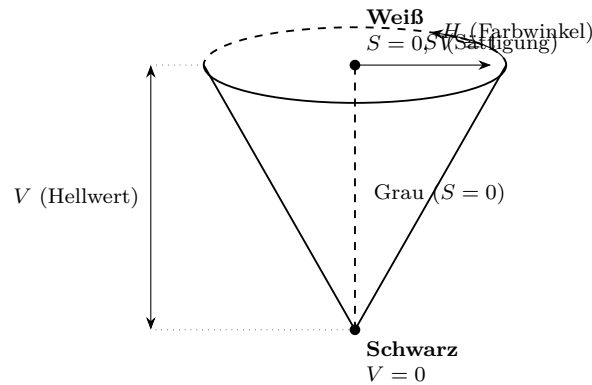


Abbildung 10: HSV-Kegel: H = Farbwinkel (0° – 360°), S = Sättigung (0 – 1), V = Hellwert (0 – 1). Schwarz an der Spitze, Weiß im Zentrum des Deckels, Grau auf der Mittelachse.

HLS-Parameter:

- **H (Hue / Farbton):** Grundfarbton als Winkel 0° – 360° .
- **L (Lightness / Helligkeit):** Helligkeit von Schwarz (0) über gesättigte Farben ($0,5$) bis Weiß (1).
- **S (Saturation / Sättigung):** Reinheit der Farbe (0 – 1).

Darstellung: Vektor (H, L, S) mit $H \in [0^\circ, 360^\circ]$, $L \in [0, 1]$, $S \in [0, 1]$.

Charakteristische Punkte:

- **Schwarzpunkt:** $L = 0$ (untere Spitze) – H und S irrelevant.
- **Weißpunkt:** $L = 1$ (obere Spitze) – H und S irrelevant.
- **Grau:** $S = 0$, $L \in (0, 1)$ – Mittelachse des Doppelkegels.
- **Maximale Sättigung:** $L = 0,5$ (Äquator des Doppelkegels).

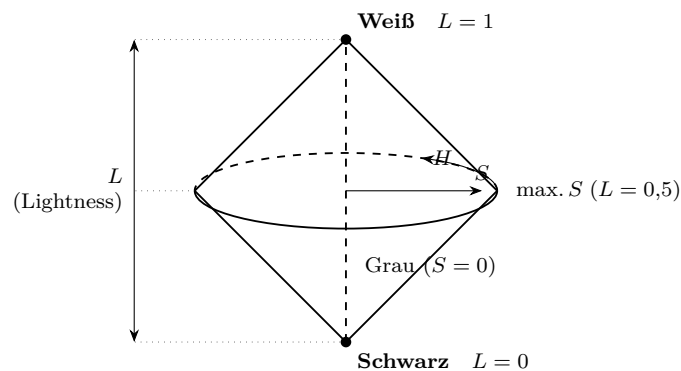


Abbildung 11: HLS-Doppelkegel: Schwarz ($L = 0$) und Weiß ($L = 1$) an den Spitzen, maximale Sättigung am Äquator ($L = 0,5$), Grau auf der Mittelachse ($S = 0$).

4.5 CMYK-Modell (Subtraktiv)

Subtraktives Farbmodell – für reflektierende Medien (Drucker, Papier).

- **Prinzip:** Pigmente absorbieren bestimmte Wellenlängen. Alle Farben zusammen → theoretisch **Schwarz**.
- **Warum K (Black) als 4. Komponente?**
 1. Reines Schwarz aus C+M+Y kaum erreichbar (meist dunkles Braun).
 2. Hoher Farbauftrag dreier Schichten würde das Papier zu feucht machen.

Darstellung: 4-Komponenten-Vektor (C, M, Y, K) mit $C, M, Y, K \in [0, 1]$ (bzw. 0%–100%).

Charakteristische Punkte:

- **Weißpunkt:** $(0, 0, 0, 0)$ – kein Farbauftrag, Papierfarbe.
- **Schwarzpunkt:** $(0, 0, 0, 1)$ mit $K = 1$ – reines Schwarz durch Schwarztinte; theoretisch auch $(1, 1, 1, 0)$, aber in der Praxis nur dunkles Braun.
- **Grau:** $C = M = Y = 0, K \in (0, 1)$ – reine Schwarztinte abgestuft (sauberstes Grau in der Praxis).

5 Kapitel C.4: Diskretisierung

Diskretisierung (= Abtastung = Digitalisierung = Digitierung = Sampling) bezeichnet die Umwandlung eines kontinuierlichen Signals in ein **digitales (diskretes) Signal**.

Beispiel: Temperatur ist kontinuierlich (ändert sich jede Millisekunde). Diskret: wir messen nur jede Stunde und runden auf ganze Zahlen.

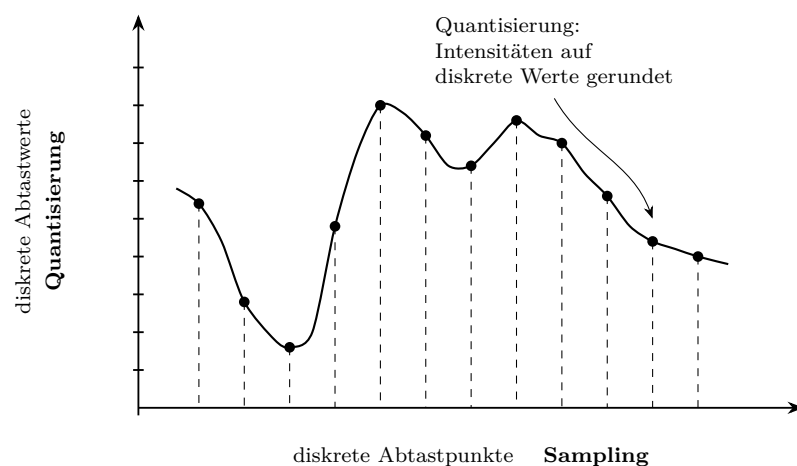


Abbildung 12: Sampling (gleichmäßige Abtastpunkte, x -Achse) und Quantisierung (Rundung der Intensität auf diskrete Stufen, y -Achse).

Zwei Hauptschritte:

1. **Abtastung (Sampling):** Diskretisierung des **Ortes** – wir wählen, wo wir Werte nehmen (z.B. jede Stunde). Das Ergebnis: eine Folge von Werten, die nur an bestimmten Punkten definiert sind.
2. **Quantisierung:** Diskretisierung der **Intensität** – wir runden, welchen Wert wir speichern ($16,5\check{\text{r}} \rightarrow 17\check{\text{r}}$). Dadurch geht Information verloren: **Quantisierungsrauschen**.

5.1 Mathematische Grundlagen: Komplexe Zahlen

Komplexe Zahlen werden nur benötigt, um die Fouriertransformation zu beschreiben.

- **Kartesische Form:** $z = a + bi$, mit Realteil a und Imaginärteil b ; es gilt $i^2 = -1$.
- **Polarform:** $z = |z| \cdot e^{i\phi} = |z|(\cos \phi + i \sin \phi)$, wobei $|z|$ die **Amplitude** (Betrag) und ϕ die **Phase** (Winkel) ist.

5.2 Fouriertransformation (FT)

Kernidee: FT ist eine mathematische Operation, die ein Signal in seine **Frequenzkomponenten zerlegt**. Jedes Signal (Ton, Bild, beliebiges Signal) kann als Summe von Sinuswellen dargestellt werden mit unterschiedlichen:

1. **Frequenzen** (u) – wie schnell schwingt die Welle,
2. **Amplituden** ($|F(u)|$) – wie stark schwingt sie,
3. **Phasen** (ϕ) – wo beginnt sie.

Anschauliches Beispiel: Ein musikalischer Akkord ist die Summe einzelner Noten: $C + E + G = C\text{-Dur-Akkord}$. Die FT macht dasselbe für jedes beliebige Signal.

5.2.1 Eindimensionale FT

Für eine integrierbare Funktion $f(x)$, $f : \mathbb{R} \rightarrow \mathbb{R}$:

$$F(u) := \int_{-\infty}^{\infty} f(x) \exp[-iux] \, dx, \quad u \in \mathbb{R}, \quad i := \sqrt{-1}.$$

Was bedeutet die Formel intuitiv? Für jede Frequenz u prüfen wir, wie gut das Signal $f(x)$ mit einer Sinuswelle der Frequenz u „übereinstimmt“. Das Ergebnis $F(u)$:

- $|F(u)|$ **groß** $\Rightarrow$ Frequenz u ist **stark vorhanden** im Signal.
- $|F(u)|$ **klein** $\Rightarrow$ Frequenz u ist **kaum vorhanden** im Signal.

Bedeutung der Variablen:

- $f(x)$ – ursprüngliches Signal (Ortsbereich); $x =$ Position (Bild) oder Zeit (Audio).
- $F(u)$ – Ergebnis der FT (Frequenzbereich): zeigt, wie viel von jeder Frequenz u im Signal enthalten ist.

- $u \in \mathbb{R}$ – Frequenz (kann $+$, $-$, 0 sein).
- $\exp[-iux] = e^{-iux}$ – komplexe Exponentialfunktion; Kombination aus Sinus und Kosinus nach der **Euler-Formel**:

$$\exp[iy] = \cos y + i \sin y.$$

$F(u)$ ist **komplexwertig** und hat zwei Teile:

- **Betrag (Amplitude):** $|F(u)| = \sqrt{\operatorname{Re}^2(F(u)) + \operatorname{Im}^2(F(u))}$
– wie stark ist die Frequenz u im Signal vorhanden?
- **Phase $\phi(u)$** – an welcher Stelle (Zeitpunkt) beginnt die Sinuswelle der Frequenz u ?

Polarform:

$$F(u) = |F(u)| \cdot \exp[i\phi(u)] = |F(u)|(\cos \phi + i \sin \phi).$$

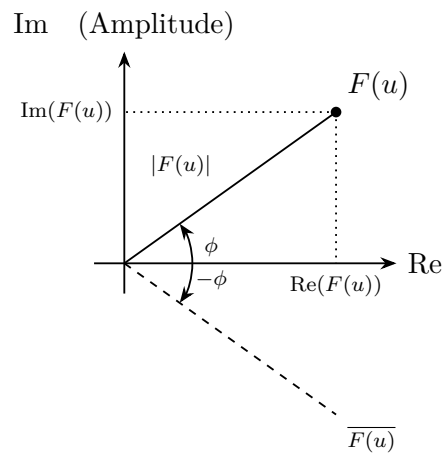


Abbildung 13: Komplexe Ebene: $F(u)$ als Punkt mit Betrag $|F(u)|$ (Amplitude) und Phase ϕ (Phasenverschiebung). Polarform: $F(u) = |F(u)| \cdot e^{i\phi(u)}$.

Hinweis

Hermiteische Symmetrie (reellwertige Signale) Für ein reellwertiges Signal $f(x, y)$ gilt:

$$F(u, v) = \overline{F(-u, -v)}$$

d. h. konjugiert-symmetrisches Spektrum. Daraus folgt:

- $|F(u, v)| = |F(-u, -v)|$ – das Amplitudenspektrum ist punktsymmetrisch.
- Die Quadranten III und IV enthalten nur die konjugierten Punkte der Quadranten I und II.
- Es gibt daher höchstens $N/2$ unabhängige Frequenzen.
- Die FFT reellwertiger Signale muss daher nur die Hälfte des Spektrums berechnen.

Vorwärts- und Rücktransformation:

- **Vorwärts (FT):** $F(u) := \int_{-\infty}^{\infty} f(x) \exp[-iux] dx$ Signal $f(x) \rightarrow$ Frequenzen $F(u)$
- **Rückwärts (iFT):** $f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} F(u) \exp[+iux] du$ Frequenzen $F(u) \rightarrow$ Signal $f(x)$

$$f(x) \xrightarrow{\text{FT}} F(u) \xrightarrow{\text{iFT}} f(x).$$

Rücktransformation – Frequenz im Signal:

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{\infty} |F(u)| \cos\left(\underbrace{u}_{\text{Frequenz}} x + \underbrace{\phi(u)}_{\text{Phase}}\right) du.$$

Wir rekonstruieren $f(x)$, indem wir alle Sinuswellen aufaddieren mit: (1) Frequenz u , (2) Amplitude $|F(u)|$, (3) Phase $\phi(u)$.

Je dichter die Stützstellen $F(u)$ (höhere Auflösung), desto besser wird $f(x)$ approximiert.

5.2.2 Rechtecksignal als Beispiel

Ein Rechteckimpuls der Breite X und Höhe A im Ortsbereich besitzt im Frequenzbereich ein si-förmiges Spektrum mit Hauptmaximum AX bei $u = 0$ und Nullstellen bei $u = \pm 1/X, \pm 2/X, \dots$

$$\text{si}(\xi) := \frac{\sin \xi}{\xi}.$$

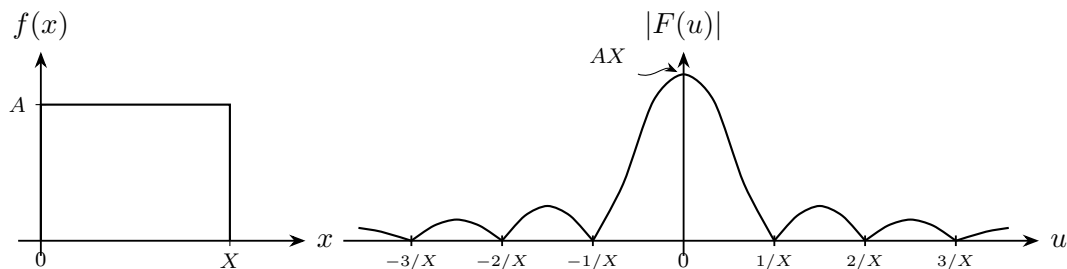


Abbildung 14: Rechtecksignal $f(x)$ (Höhe A , Breite X) und sein Betragsspektrum $|F(u)|$: Hauptmaximum AX bei $u = 0$, Nullstellen bei $u = \pm n/X$.

Interpretation: FT als Korrelation

Die FT misst für jede Frequenz u , wie gut die Schwingung e^{-iux} mit dem Signal $f(x)$ korreliert – hoher Betrag $|F(u)|$ bedeutet gute, niedriger schlechte Korrelation.

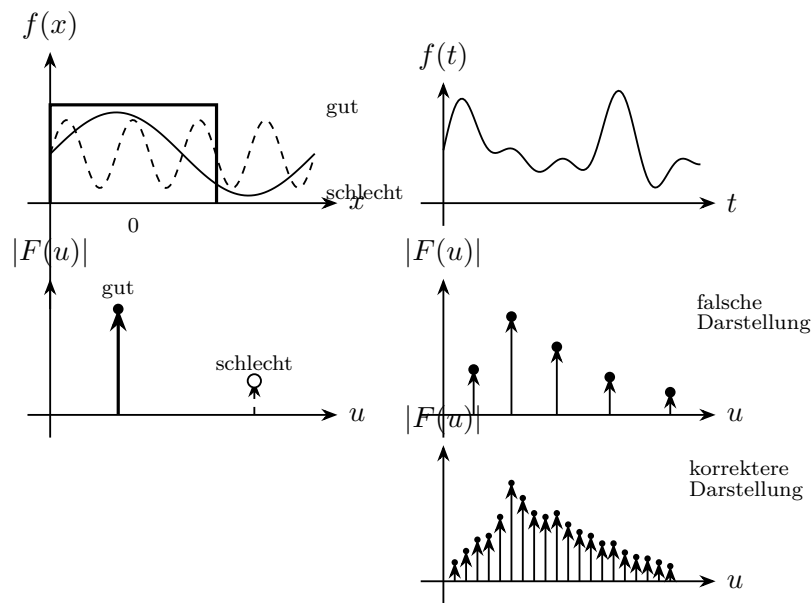


Abbildung 15: Links: Gut korrelierende (durchgezogen) und schlecht korrelierende (gestrichelt) Schwingung mit dem Rechtecksignal, und die entsprechenden $|F(u)|$ -Werte. Rechts: Falsche Darstellung (wenige Linien) vs. korrektere Darstellung (viele dichte Linien) eines kontinuierlichen Spektrums.

5.2.3 Zweidimensionale FT (für Bilder)

$$F(u, v) := \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) \exp[-i(ux + vy)] dx dy, \quad u, v \in \mathbb{R}.$$

Bezeichnungen:

- $f(x)$ und $F(u)$ bzw. $f(x, y)$ und $F(u, v)$ heißen **Fouriertransformationspaar**, Schreibweise:

$$f(x) \circ \text{---} \bullet F(u) \quad f(x, y) \circ = \bullet F(u, v).$$

- x, y heißen **Ortskoordinaten** (räumliche Position).
- $f(x, y)$ heißt **Ortsfunktion** (das Bild im Ortsbereich).
- u, v heißen **Ortsfrequenzkoordinaten**.
- $F(u, v)$ heißt **Ortsfrequenzspektrum**.
- xy -Ebene: **Ortsbereich** | uv -Ebene: **Ortsfrequenzbereich**.

Eigenschaften von $F(u, v)$:

- $F(u, v)$ ist üblicherweise **komplexwertig**:

$$F(u, v) = \operatorname{Re} F(u, v) + i \cdot \operatorname{Im} F(u, v) = |F(u, v)| \exp[i\phi(u, v)].$$

- $|F(u, v)| = \sqrt{\operatorname{Re} F(u, v)^2 + \operatorname{Im} F(u, v)^2}$ heißt **Amplitudenspektrum** ($|F(u)|$ – wie stark ist Frequenz (u, v) im Bild vertreten).

- $\phi(u, v) = \arctan\left(\frac{\operatorname{Im} F(u, v)}{\operatorname{Re} F(u, v)}\right)$ heißt **Phasenspektrum** (ϕ – wo beginnt die Welle der Frequenz (u, v)).
- $|F(u, v)|^2 = F(u, v) \cdot \overline{F(u, v)}$ heißt **Leistungsspektrum** von $f(x, y)$ ($\overline{} :=$ konjugiert komplex).

Frequenz (u, v) bei Bildern: Eine Sinus-/Cosinuswelle, bei der helle Grautöne große Werte und dunkle Grautöne niedrige Werte visualisieren.

Diskrete 2D-FT (für ein $M \times N$ -Bild):

$$f(m, n) = \frac{1}{MN} \sum_{k=0}^{M-1} \sum_{l=0}^{N-1} |F(k, l)| \cos\left(2\pi\left(\frac{mk}{M} + \frac{nl}{N}\right) + \phi(k, l)\right).$$

5.2.4 Klausurrelevante Eigenschaften der FT

- **Vertauschungssatz:** Wenn $f(x, y) \circ = \bullet F(u, v)$, dann gilt:

$$F(x, y) \circ = \bullet 4\pi^2 f(-u, -v).$$

d. h. Orts- und Frequenzbereich sind vertauschbar (bis auf Skalierung und Vorzeichenumkehr).

- **Separierbarkeit:** Die 2D-FT kann durch zwei aufeinanderfolgende 1D-FTs berechnet werden – erst zeilenweise, dann spaltenweise (oder umgekehrt):

$$f(x, y) \xrightarrow{x} F^x(u, y) \xrightarrow{y} F(u, v)$$

$$f(x, y) \xrightarrow{y} F^y(x, v) \xrightarrow{x} F(u, v)$$

(F^x = Zwischenergebnis nach FT in x -Richtung.)

- **Faltungssatz:** Eine Faltung (Filterung) im Ortsbereich entspricht einer **Multiplikation** im Frequenzbereich:

$$f(x, y) * h(x, y) \circ = \bullet F(u, v) \cdot H(u, v).$$

$\Rightarrow$ Rechenaufwand erheblich geringer.

- **FFT (Fast Fourier Transform):** Divide-&-Conquer-Algorithmus zur effizienten Berechnung der diskreten Fouriertransformation. Sie nutzt die Symmetrien der DFT aus, um den Rechenaufwand von $O(N^2)$ auf $O(N \log N)$ zu reduzieren, indem das Signal rekursiv in zwei Hälften geteilt und die Zwischenergebnisse wiederverwendet werden.

5.3 Dirac-Funktion und Dirac-Feld

Definition

Die **Dirac-Funktion** (Dirac-Distribution) ist der Grenzwert zunehmend schmaler Rechteckimpulse gleicher Fläche:

$$\delta(x, y) := \lim_{a \rightarrow 0} \frac{1}{4a^2} \operatorname{rect}_{a,a}(x, y).$$

Anschaulich: Ein Rechteckimpuls der Breite $2a$ und Höhe $\frac{1}{4a^2}$ hat immer die Fläche 1. Lässt man $a \rightarrow 0$, wird der Impuls unendlich schmal und unendlich hoch – die Fläche bleibt dabei stets 1. Das Ergebnis ist ein „unendlich dünner Stich“ an der Stelle $(0, 0)$.

Eigenschaften:

- **Normierung** – die Fläche unter δ ist immer 1:

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \delta(x, y) \, dx \, dy = 1.$$

- **Sieb-Eigenschaft** – δ liest einen einzigen Funktionswert aus:

$$\int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) \delta(x - a, y - b) \, dx \, dy = f(a, b).$$

Warum? $\delta(x - a, y - b)$ ist überall 0, außer genau bei (a, b) – das Integral „siebt“ deshalb nur den Wert $f(a, b)$ heraus.

- **Verschiebungseigenschaft** – Faltung mit δ verschiebt die Funktion:

$$f(x, y) * \delta(x - a, y - b) = f(x - a, y - b).$$

Warum? δ ist das neutrale Element der Faltung. Genau wie Multiplikation mit 1 die Zahl unverändert lässt, lässt Faltung mit δ die Form unverändert – nur verschoben.

Interpretation: Modell eines **idealen Punktsensors** – Grenzfall eines rechteckigen Sensors, dessen Fläche gegen 0 schrumpft, während die Empfindlichkeit proportional wächst (Gesamtempfindlichkeit bleibt konstant). Analogie in der Physik: Punktmasse als Grenzfall eines ausgedehnten Körpers.

Definition

Das **Dirac-Feld** (Dirac-Kamm, Shah-Funktion) ist ein regelmäßiges 2D-Gitter aus Dirac-Impulsen mit den Abständen Δx und Δy :

$$s(x, y) := \sum_{m=-\infty}^{\infty} \sum_{n=-\infty}^{\infty} \delta(x - m\Delta x, y - n\Delta y), \quad m, n \in \mathbb{Z}.$$

Anschaulich: Das Dirac-Feld ist das mathematische Modell eines **Pixelrasters**: In jedem Rasterpunkt sitzt ein δ -Impuls – genau wie jeder Pixel einer Kamera nur an einem einzigen Ort misst.

Fouriertransformierte des Dirac-Feldes:

Die FT des Dirac-Feldes ist wiederum ein Dirac-Feld – diesmal im Frequenzbereich:

$$S(u, v) = \Delta u \Delta v \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} \delta(u - k\Delta u, v - l\Delta v), \quad \Delta u := \frac{2\pi}{\Delta x}, \quad \Delta v := \frac{2\pi}{\Delta y}.$$

Hinweis

Dualität: Orts- und Frequenzgitter sind umgekehrt proportional. Je **feiner** das Ortsgitter (kleines Δx), desto **gröber** das Frequenzgitter (großes $\Delta u = 2\pi/\Delta x$) – und umgekehrt. Dichtes Abtastnetz $\Rightarrow$ weit auseinanderliegende Spektralkopien $\Rightarrow$ keine Überlappung $\Rightarrow$ kein Aliasing.

Modell der diskreten Abtastung:

Um ein Signal $f(x, y)$ an den Gitterpunkten abzutasten, multipliziert man es mit dem Dirac-Feld:

$$f_{\Delta}(x, y) = f(x, y) \cdot s(x, y) = \sum_{m,n} f(m\Delta x, n\Delta y) \delta(x - m\Delta x, y - n\Delta y).$$

Warum funktioniert das? Dank der Sieb-Eigenschaft „liest“ jeder δ -Impuls genau den Wert $f(m\Delta x, n\Delta y)$ aus. Zwischen den Gitterpunkten ist das Ergebnis null – das Signal ist diskretisiert.

Im Frequenzbereich liefert der Faltungssatz ($\cdot$ im Ort $\Leftrightarrow **$ in der Frequenz):

$$F_{\Delta}(u, v) = \frac{1}{4\pi^2} F(u, v) ** S(u, v) = \frac{\Delta u \Delta v}{4\pi^2} \sum_{k,l} F(u - k\Delta u, v - l\Delta v).$$

Das Spektrum $F_{\Delta}(u, v)$ besteht aus **unendlich vielen periodischen Kopien** von $F(u, v)$, die im Abstand Δu bzw. Δv aneinandergereiht sind. Überlappen sich diese Kopien, entsteht **Aliasing** – der Grund, warum das Abtasttheorem existiert.

5.4 Abtasttheorem (Whittaker-Shannon)**Definition**

Das ursprüngliche Signal ist aus dem Abtastsignal **rekonstruierbar**, wenn die Abtastung mit einer **mehr als doppelt so hohen Frequenz** wie der höchsten im Signal vorkommenden Frequenz erfolgt:

$$b_x := \frac{2\pi}{\Delta x} > 2b_u, \quad b_y := \frac{2\pi}{\Delta y} > 2b_v.$$

Was ist b_u ? – b_u ist der **Radius** im Ortsfrequenzbereich, d. h. die höchste Frequenz, die der Sensor (Chip) überhaupt erfassen kann. Ein Chip hat eine bestimmte physische Ausdehnung; diese bestimmt, welche maximale Frequenz er darstellen kann. b_u ist also keine Eigenschaft des Signals, sondern des **Systems** (Sensor).

Zusammenhang zwischen Abtastabstand und Abtastfrequenz:

Abtastabstand Δx und Abtastfrequenz b_x sind **umgekehrt proportional**:

$$b_x = \frac{2\pi}{\Delta x}$$

Je **kleiner** der Abtastabstand (Messpunkte liegen eng beieinander), desto **größer** die Abtastfrequenz – und umgekehrt. *Anschaulich:* Wer jeden Millimeter misst, misst häufiger als wer nur jeden Zentimeter misst.

Warum Δu und Δx antiproportional sind:

$$\Delta u = \frac{2\pi}{\Delta x}$$

Je feiner das Ortsgitter (kleines Δx), desto weiter auseinander liegen die Spektralkopien (großes Δu) – und umgekehrt. Abtastung im Ortsbereich entspricht dem Abstand Δx ; im Ortsfrequenzbereich dem Abstand Δu .

Anschaulich – wann ist das Theorem erfüllt?

Das Theorem ist genau dann erfüllt, wenn die **Spektralkopien (Hütchen) sich nicht überlappen**. Jede Kopie hat die Ausdehnung b_u nach links und rechts; der Abstand zwischen den Mittelpunkten ist Δu . Damit keine Überlappung entsteht, muss $\Delta u > 2b_u$ gelten.

Chip besteht aus einem Rect-Signal: Jeder Sensor-Pixel hat eine rechteckige Ausdehnung. Die FT eines Rechtecks ist eine si-Funktion – daher hat jeder Chip im Frequenzbereich ein si-förmiges Spektrum mit einer endlichen Bandbreite b_u .

Äquivalente Bedingung für die Abtastintervalle:

$$\Delta x \leq \frac{\pi}{b_u}, \quad \Delta y \leq \frac{\pi}{b_v}.$$

(Hälfte der kleinsten Periode der höchsten vorkommenden Frequenz.)

Rekonstruktion – Idee:

Rekonstruktion bedeutet: wir wenden einen **Rechtecktieffpassfilter** H_{TP} auf das Spektrum $F_\Delta(u, v)$ im Ortsfrequenzbereich an. Dieser Filter schneidet genau die zentrale Kopie (den „primären Anteil“) heraus und verwirft alle anderen:

$$F(u, v) = F_\Delta(u, v) \cdot H_{TP}(u, v) \xrightarrow{\text{iFT}} f(x, y) = f_\Delta(x, y) ** h_{TP}(x, y).$$

Damit sehen wir garantiert nur die Anteile unterhalb der Grenzfrequenz – **Garantie der Überlappungsfreiheit**.

Warum funktioniert das in der Praxis nicht perfekt? Ein idealer Rechtecktieffpassfilter entspricht im Ortsbereich einer unendlich ausgedehnten si-Funktion. Reale Filter sind immer begrenzt, daher ist die Rekonstruktion nur näherungsweise möglich.

Die explizite Rekonstruktionsformel:

$$f(x, y) = \frac{\Delta x \Delta y \xi \eta}{\pi^2} \sum_{m, n} f(m\Delta x, n\Delta y) \text{si}(\xi(x - m\Delta x)) \text{si}(\eta(y - n\Delta y)),$$

wobei ξ, η die Ausdehnung des Rechteck-Tiefpassfilters bestimmen.

Aliasing bei Verletzung des Theorems:

- **Moiré-Muster:** Falsche Streifenmuster auf fein strukturierten Flächen – entstehen durch Überlappung der Spektralkopien.
- **Lösung:** Vor der Abtastung einen Tiefpassfilter anwenden, der hohe Frequenzen abschneidet $\Rightarrow$ Bild wird leicht unscharf, aber Aliasing verschwindet.

Tiefpass- vs. Hochpassfilter:

- **Tiefpassfilter:** Lässt tiefe Frequenzen (langsame Änderungen, grobe Strukturen) durch, löscht hohe Frequenzen (feine Details, Kanten) $\Rightarrow$ Bild wird **unscharf**.
- **Hochpassfilter:** Lässt hohe Frequenzen durch, löscht tiefe Frequenzen $\Rightarrow$ Bild wird **schärfer / kontrastreicher** (Kanten werden betont).

Hinweis

Bemerkung: Hinreichende, nicht notwendige Bedingung Das Whittaker-Shannon-Abtasttheorem liefert nur eine **hinreichende** Bedingung für Rekonstruierbarkeit, keine notwendige. Signale mit bestimmten Spektraleigenschaften (z. B. Bandpasssignale) können mitunter auch bei gröberer Abtastung rekonstruiert werden.

5.5 Quantisierung

Quantisierung = Rundung der kontinuierlichen Intensitätswerte auf eine **endliche Menge diskreter Stufen**.

Mach-Band-Effekt

Bei zu wenigen Graustufen werden Helligkeitsverläufe als Treppenstufen dargestellt. An jeder Stufenkante nimmt das Auge **illusorische helle und dunkle Streifen** wahr, die im Original nicht vorhanden sind.

Warum? Das visuelle System verstärkt Kontraste an Kanten – ein Mechanismus, der in der Natur nützlich ist (schärfere Wahrnehmung von Objekträndern), bei grober Quantisierung aber als Artefakt sichtbar wird. Der Effekt ist rein neuronal: das Auge „erfindet“ Streifen dort, wo nur eine abrupte Helligkeitsstufe ist.

Lineare vs. Logarithmische Quantisierung

Lineare Quantisierung: Die N Stufen sind gleichmäßig über den gesamten Helligkeitsbereich verteilt – gleicher Abstand zwischen je zwei benachbarten Werten.

Problem: Das menschliche Auge ist **nicht linear empfindlich**. Im dunklen Bereich nimmt es sehr feine Unterschiede wahr; im hellen Bereich sind dieselben absoluten Unterschiede kaum sichtbar. Bei linearer Quantisierung:

- Im **dunklen Bereich:** zu wenige Stufen $\Rightarrow$ sichtbare Treppenstufen, Mach-Bänder.
- Im **hellen Bereich:** zu viele Stufen $\Rightarrow$ Präzision, die das Auge ohnehin nicht unterscheiden kann – verschwendete Bits.

Logarithmische Quantisierung: Die Stufen werden **nicht gleichmäßig**, sondern entsprechend der menschlichen Wahrnehmung verteilt – im dunklen Bereich eng beieinander, im hellen Bereich weiter auseinander.

Ergebnis: Bei gleicher Anzahl von Bits wirkt das Bild subjektiv deutlich besser – weniger Banding, keine Mach-Bänder. Dieses Prinzip steckt z. B. in der Gamma-Korrektur von Bildschirmen und in Videocodecs (JPEG, H.264).

Dithering: Pixel verschiedener Farben werden so verteilt, dass das Auge aus der Distanz einen neuen Farbton wahrnimmt (siehe Abschnitt 5.6).

5.6 Dithering

Dithering ist eine Methode zur verbesserten Bilddarstellung auf Ausgabegeräten mit **beschränkter Farbpalette**: Durch geeignete räumliche Verteilung der verfügbaren

Farben in Punktmustern wird dem menschlichen Auge ein größerer Farbreichtum suggeriert.

Zwei Teilaufgaben:

- **Farbquantisierung (A):** Berechnung einer optimalen kleinen Farbtabelle.
- **Farbersetzung (B):** Berechnung des Ergebnisbildes mithilfe dieser Tabelle.

A – Farbquantisierung: Medianschnitt-Algorithmus

Idee: Die im Bild vorkommenden Farben liegen als Punkte im Farbraum. Durch iteratives Halbieren des Hüllquaders beim Median der längsten Achse entsteht ein binärer Baum; jedes Blatt liefert genau eine Repräsentationsfarbe (gewichtetes Mittel der darin enthaltenen Farben).

Algorithmus (Medianschnitt auf k Farben):

1. Initialisiere eine Liste mit dem **Hüllquader** aller Bildfarben im Farbraum.
2. Wiederhole, bis die Liste k Quader enthält:
 - a. Entferne den Quader mit den **meisten Farben** aus der Liste.
 - b. Unterteile ihn beim **Median seiner längsten Achse** in zwei Teilmengen.
 - c. Füge die **Hüllquader** der beiden Hälften in die Liste ein.
3. Füge das **gewichtete Mittel** der Farben jedes Quaders in die Farbtabelle ein.

B – Farbersetzung

Aufgabe: Ersetze jeden Pixel des Originalbildes durch einen der verfügbaren Farben aus der Farbtabelle. Ziel: das Ergebnis soll dem Original visuell möglichst ähnlich sein.

Methode 1 – Nächste-Nachbar-Farbersetzung:

Wähle für jedes Pixel die **nächstliegende Farbe** in der Farbtabelle. Einfach und schnell, erzeugt jedoch sichtbares *Banding* – an glatten Farbverläufen entstehen harte Stufen, weil der Fehler einfach verworfen wird.

Methode 2 – Floyd-Steinberg-Fehlerübertrag:

Idee: Wenn ein Pixel auf den nächsten Tabellenwert gerundet wird, entsteht ein **Quantisierungsfehler** $e = x - k$. Statt ihn zu verwerfen, wird er auf noch nicht verarbeitete Nachbarpixel **vorwärts verteilt**. Diese werden dadurch korrigiert – zu hell gerundete Pixel ziehen die Nachbarn dunkler, zu dunkel gerundete heller. Im Durchschnitt einer kleinen Region stimmt die Farbe.

Analogie: Du musst 10,70 € zahlen, hast aber nur Münzen. Du gibst 11 € – Fehler +0,30 €. Diesen Betrag ziehst du von der nächsten Zahlung ab. Über mehrere Transaktionen gleicht sich alles aus.

Definition

Algorithmus (Floyd-Steinberg): Gegeben ein $m \times n$ -Farbbild f und eine Farbtabelle F :

1. Für jedes Pixel (i, j) (zeilenweise, links nach rechts):
 - a. $x \leftarrow f[i][j]$ (Original-Farbe lesen)
 - b. $k \leftarrow \operatorname{argmin}_{c \in F} \|x - c\|$ (nächsten Repräsentanten wählen)
 - c. $f'[i][j] \leftarrow k$ (quantisiertes Pixel ausgeben)
 - d. $e \leftarrow x - k$ (Fehler berechnen)
 - e. Fehler e auf Nachbarn verteilen:

aktuell	$+\frac{3}{8}e$
$+\frac{3}{8}e$	$+\frac{1}{4}e$

Rechts: $+\frac{3}{8}e$ Unten: $+\frac{3}{8}e$ Diagonal unten-rechts: $+\frac{1}{4}e$

Hinweis

Warum funktioniert das? Das Auge mittelt Farben benachbarter Pixel. Indem der Fehler auf die Nachbarschaft verteilt wird, stimmt der **lokale Farbdurchschnitt** mit dem Original überein – auch wenn kein einzelner Pixel exakt die richtige Farbe hat. Das Ergebnis wirkt visuell deutlich weicher als reine Nächste-Nachbar-Ersetzung. Am Bildrand muss der Algorithmus angepasst werden (fehlende Nachbarpixel).

5.7 Signalverarbeitungspipeline

Der Weg eines physikalischen Reizes vom Sensor bis zur Interpretation – 7 Schritte in 3 Phasen:

Phase 1: Signalgewinnung

- **Signalaufnahme:** Sensoren (Kamera, Mikrofon) erfassen physikalische Reize. Problem: Informationsverlust, Rauschen, Messfehler.
- **Signaldiskretisierung:** Analoges Signal $\rightarrow$ digitale Werte (Abtastung + Quantisierung). **Abtasttheorem** beachten, um Alias-Effekte zu vermeiden.

Phase 2: Signalbearbeitung

- **Signalrestauration:** Modellbasierte Korrektur von Aufnahme Fehlern (z. B. Kalibrierung).
- **Signalverbesserung:** Anwendungsbezogene Aufbereitung – Rauschreduktion, Kontrasterhöhung.
- **Signalsegmentierung:** Signal in bedeutungsvolle Teilsignale zerlegen (Vorder-/Hintergrund, Wortgrenzen).

Phase 3: Signalerkennung

- **Merkmalsextraktion:** Teilsignalen spezifische Merkmale zuordnen (Kanten, Formen, Spektrum).
- **Klassifikation/Synthese:** Signal einer Klasse zuordnen (Objekterkennung) oder neues Signal erzeugen.

Ergebnis: Einzelwerte oder Wertfolgen – Zeichen, Positionen, Töne, Bilder. Wichtigste Gütemerkmale: **Latenz** und **Abweichung** (Soll vs. Ist).

5.8 Lernkonzepte

Der Unterschied liegt darin, *welche Information* beim Training verfügbar ist:

- **Überwachtes Lernen (Supervised):** Lernen aus Daten-Label-Paaren – das Modell weiß, was richtig ist. Beispiele: Klassifikation, Regression, strukturierte Vorhersage.
- **Unüberwachtes Lernen (Unsupervised):** Keine Labels – das Modell findet selbst Struktur in den Daten. Beispiele: Clustering, Dimensionsreduktion, generative Modelle.
- **Selbst-überwachtes Lernen (Self-supervised):** Labels werden automatisch aus den Daten selbst erzeugt (Daten-Daten-Paare) – kein manuelles Annotieren nötig. Beispiele: Self-supervised Stereo/Flow, Kontrastives Lernen.
- **Verstärkungslernen (Reinforcement):** Agent lernt durch aktive Exploration und erhält spärliche Belohnungssignale. Beispiele: Deep Q-Learning, Policy Gradient, Actor-Critic.

6 D – Bilderzeugung

6.1 Daten-/Prozessmodell der Bilderzeugung

Das Modell beschreibt, wie aus einer abstrakten Szenenbeschreibung eine menschliche Wahrnehmung entsteht – den vollständigen Daten- und Prozessfluss von Input bis Auge.

1. Szenenbeschreibung (Input): Alle statischen Informationen über die virtuelle Welt: Geometrie der Objekte, Beleuchtungsmodell (z. B. Phong), Materialeigenschaften (Farbe, Reflexionsgrad, Transparenz) sowie die Kamera (Position, Blickrichtung, Projektion/FOV).

2. CPU – Szenenmanagement: Die CPU verwaltet und bereitet die Szene vor: Laden und Generieren von Modellen/Texturen, Organisation der Geometrie in Datenstrukturen (z. B. BVH für schnelle Schnitttests), Verwaltung des Szenengraphen (Transformationshierarchie, Abhängigkeiten).

3. GPU – Rendering: Die GPU übernimmt den rechenintensiven Teil. Ausgabe ist zunächst die **Radianz der Pixel** – physikalisch berechnete Lichtmengen mit hohem Dynamikumfang (HDR), der weit über die Darstellungsfähigkeit von Monitoren hinausgeht.

4. Post-Processing & Ausgabe:

- **Tone Mapping:** Wandelt HDR-Radianzwerte in darstellbare RGB-Werte um, da Monitore nur begrenzte Helligkeitsstufen abbilden können.
- **Wahrnehmung:** Das Endergebnis ist der Farbreiz auf den Netzhautzellen des Betrachters – im Idealfall entsprechend der realen Weltwahrnehmung.

5. Interaktiver Kreislauf: Benutzerinteraktion (Bewegung, Eingabe) löst eine Neuberechnung aus – entweder ab der Szenenbeschreibung (Szenenänderung) oder ab der CPU-Verarbeitung. Dies macht den Prozess zu einem **dynamischen Regelkreis** statt einer einmaligen Berechnung.

Hinweis

Kernidee Szenenbeschreibung → CPU (Szenenmanagement) → GPU (Rendering) → Tone Mapping → menschliche Wahrnehmung $\circlearrowright$ Interaktion.

6.2 Bilderzeugungs-Pipeline

Die Bilderzeugungs-Pipeline beschreibt, wie aus einer abstrakten 3D-Szenenbeschreibung ein fertiges 2D-Rasterbild auf dem Monitor entsteht.

Eingabe in die Pipeline – Grafikelemente:

- **Polygonbasierte Darstellung:** Objekte werden durch Punktmengen und verbindende Geraden (Polygone, meist Dreiecke) beschrieben. Einfache Form, keine optimale Form.
 - *Nachteil 1:* Zu viele Daten werden benötigt.
 - *Nachteil 2:* Man kann die Auflösung nicht nachträglich ändern.
- **Nicht-polygonbasierte Darstellung (implizit):** Man definiert eine Funktion $f(x)$, die das Objekt beschreibt, z. B. eine Kugel durch $x^2 + y^2 + z^2 = r^2$. Man nutzt in der Regel eine Kombination aus beiden Arten.

2D- vs. 3D-Pipeline:

- **2D-Pipeline:** Transformation → Clipping → Verrasterung.
- **3D-Pipeline:** Ergänzt um **Beleuchtung, Projektion** und **Sichtbarkeitsberechnung**.

Die 6 Schritte der Pipeline

1. Transformation

Affine Abbildungen (Verschieben, Drehen, Skalieren) werden auf die Geometrie angewendet. Objekte wandern durch drei Koordinatensysteme:

- **Lokales KS** (Objektraum) → **Weltkoordinatensystem** (Platzierung in der Szene) → **Kamerakoordinatensystem** (Sicht der Kamera).

Wie? Matrizen werden auf die Eckpunkte (Vertices) der Dreiecke angewendet.

Definition

Eine **affine Abbildung** (auch **rigide Transformation**, da Winkel erhalten bleiben) im 3D-Raum besteht aus einer linearen Abbildung (Matrix **A**) und einer Translation (Vektor **t**):

$$\bar{\mathbf{p}} = \mathbf{A} \cdot \mathbf{p} + \mathbf{t}, \quad \mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix}, \quad \mathbf{t} = \begin{pmatrix} t_1 \\ t_2 \\ t_3 \end{pmatrix}.$$

Eigenschaften affiner Abbildungen:

- **Parallelentreue:** Parallele Geraden bleiben parallel.
- **Teilverhältnis:** Verhältnisse von Streckenabschnitten auf einer Geraden bleiben erhalten.

Homogene Darstellung

Problem: In $\bar{\mathbf{p}} = \mathbf{A} \cdot \mathbf{p} + \mathbf{t}$ ist die Translation eine Addition – alle anderen Transformationen sind Multiplikationen. Das macht Hintereinanderausführung umständlich.

Lösung: **Homogene Koordinaten** – der Punkt (x, y, z) wird um eine vierte Komponente $w = 1$ erweitert: $\mathbf{p}^* = (x, y, z, 1)^T$. Jetzt lässt sich **jede** affine Abbildung als eine einzige Matrizenmultiplikation schreiben:

$$\bar{\mathbf{p}}^* = \mathbf{A}^* \cdot \mathbf{p}^*.$$

Struktur der 4×4 -Transformationsmatrix:

$$\mathbf{A}^* = \begin{pmatrix} a & b & c & t_x \\ e & f & g & t_y \\ i & j & k & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Obere linke 3×3 : **Rotation, Skalierung, Scherung**
- Rechte Spalte (t_x, t_y, t_z) : **Translation**
- Untere Zeile $(0, 0, 0, 1)$: **Homogene Erweiterung** – Einsen auf der Diagonale bedeuten: keine Skalierung, keine Rotation; der Punkt wird nur verschoben.

Elementare Transformationsmatrizen (homogen):

Translation um (t_x, t_y, t_z) :

$$T(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Skalierung mit Faktoren (s_x, s_y, s_z) :

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation um die z -Achse:

$$R_z(\phi) = \begin{pmatrix} \cos \phi & -\sin \phi & 0 & 0 \\ \sin \phi & \cos \phi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation um die x -Achse:

$$R_x(\phi) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi & 0 \\ 0 & \sin \phi & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation um die y -Achse:

$$R_y(\phi) = \begin{pmatrix} \cos \phi & 0 & \sin \phi & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \phi & 0 & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Hintereinanderausführung: Mehrere Transformationen werden durch Matrizenmultiplikation kombiniert. Die erste Transformation steht rechts (direkt neben $\mathbf{p}^*$):

$$\mathbf{A}_{\text{ges}}^* = \mathbf{A}_n^* \cdots \mathbf{A}_2^* \cdot \mathbf{A}_1^*.$$

Vorteil: Alle Matrizen werden einmal vorausberechnet – für jeden Punkt des Modells dann nur noch eine einzige 4×4 -Multiplikation.

Rotation um eine beliebige Achse (5 Schritte):

1. T : Drehachse in den Ursprung verschieben.
2. R_z : Achse in die xz -Ebene drehen.
3. R_y : Achse mit der z -Achse zur Deckung bringen.
4. $R_z(\phi)$: Eigentliche Rotation um ϕ .
5. Rücktransformation: Schritte 1–3 invers und in umgekehrter Reihenfolge.

Beispiel

Klausurbeispiel: $R1 \rightarrow R2$ Gegeben: R1 mit Zentrum (1,5; 2), Breite 1, Höhe 2 (Hochformat).

Gesucht: R2 mit Zentrum (4,5; 4,5), Breite 3, Höhe 1 (Querformat).

R1 ist Hochformat, R2 Querformat $\Rightarrow$ Rotation um 90° nötig.

Schritt 1 – T_1 : Zentrum von R1 in den Ursprung verschieben

$$T_1 = \begin{pmatrix} 1 & 0 & -1,5 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{pmatrix}$$

Schritt 2 – R_{90° : Rotation um 90° gegen den Uhrzeigersinn ($\cos 90^\circ = 0$, $\sin 90^\circ = 1$)

$$R_{90^\circ} = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Schritt 3 – S : Skalierung ($s_x = 1,5$ damit Breite $2 \rightarrow 3$, $s_y = 1$)

$$S = \begin{pmatrix} 1,5 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Schritt 4 – T_2 : Ins Zentrum von R2 verschieben

$$T_2 = \begin{pmatrix} 1 & 0 & 4,5 \\ 0 & 1 & 4,5 \\ 0 & 0 & 1 \end{pmatrix}$$

Gesamtmatrix $A^* = T_2 \cdot S \cdot R_{90^\circ} \cdot T_1$:

$$A^* = \begin{pmatrix} 0 & -1,5 & 7,5 \\ 1 & 0 & 3 \\ 0 & 0 & 1 \end{pmatrix}$$

Probe mit Ecke (1,1): $0 \cdot 1 + (-1,5) \cdot 1 + 7,5 = 6$, $1 \cdot 1 + 0 \cdot 1 + 3 = 4 \Rightarrow (6,4) \checkmark$

Beispiel

Klausurbeispiel: Translation in homogenen Koordinaten Gesucht: homogene Matrix, die $(1, 2, 3)^T$ auf $(8, 6, 4)^T$ abbildet (nur Verschiebung).

Translationsvektor: $\mathbf{t} = (7, 4, 1)^T$.

$$T = \begin{pmatrix} 1 & 0 & 0 & 7 \\ 0 & 1 & 0 & 4 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Probe: $T \cdot (1, 2, 3, 1)^T = (8, 6, 4, 1)^T \checkmark$

2. Beleuchtung (Lighting)

Berechnung der Lichtintensität an den Eckpunkten anhand von Materialeigenschaften, Oberflächennormalen und Lichtquellen.

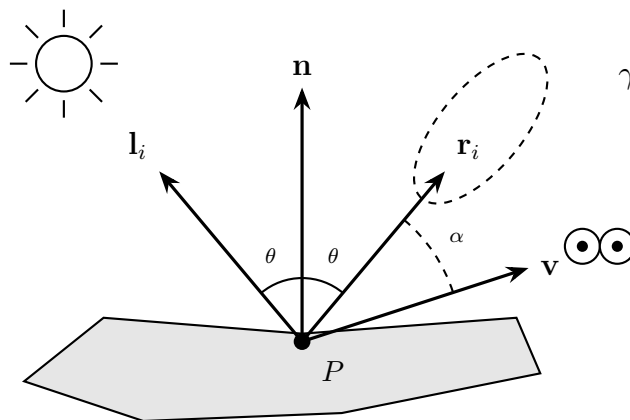
Das Phong-Modell ist ein **lokales Beleuchtungsmodell**: Es betrachtet nur die Interaktion zwischen einer Lichtquelle und einem einzelnen Oberflächenpunkt – keine Reflexionen zwischen Objekten werden berücksichtigt.

Definition

Phong-Formel (zusammengefasst)

$$I(\mathbf{v}) = \underbrace{k_a \cdot I_{amb}}_{I_a} + \sum_{i=1}^l \frac{I_i}{f(d_i)} \cdot \left[\underbrace{k_d (\mathbf{n} * \mathbf{l}_i)}_{I_d} + \underbrace{k_g (\mathbf{v} * \mathbf{r}_i)^\gamma}_{I_g} \right]$$

Bemerkung: $I(\lambda, v) = \sum_l I_l(\lambda, v)$ wäre physikalisch korrekt (wellenlängenabhängig); λ wird zur Vereinfachung weggelassen.



Geometrische Parameter

Ein Oberflächenpunkt P besteht aus: **Koordinaten**, **Farbwert**, **Normalvektor** (wird aus den umliegenden Punkten berechnet).

Eine Lichtquelle hat: **Einfalls- und Ausfallswinkel**, **Reflexionsvektor $\mathbf{r}$** ; und besteht aus **Farbe** und **Intensität/Stärke**.

- $\mathbf{n}$ – auf Länge 1 normierte Oberflächennormale
- $\mathbf{l}_i$ – normierter Richtungsvektor zur i -ten Lichtquelle
- $\mathbf{v}$ – normierte Blickrichtung (vom Punkt zum Auge des Betrachters)
- $\mathbf{r}_i$ – normierter Reflexionsvektor zu $\mathbf{l}_i$ nach dem Reflexionsgesetz (Einfallswinkel = Ausfallswinkel)
- $\gamma > 0$ – Glanzexponent (Ausdehnung des Glanzlichtes / Shininess)

Hinweis

Notation: Was bedeutet “*” in den Formeln? “*” steht für die Transponierung des Vektors. Die Ausdrücke $\mathbf{n} * \mathbf{l}_i$ und $\mathbf{v} * \mathbf{r}_i$ sind damit **Skalarprodukte** – sie ergeben den **Kosinus des Winkels** zwischen den beiden normierten Vektoren.

Die drei Anteile

- **Ambienter Anteil** $I_a = k_a \cdot I_{amb}$
Grundhelligkeit der Szene, simuliert indirekt gestreutes Licht. **Unabhängig** von Betrachterstandpunkt und Lichtquellenposition.
- **Diffuser Anteil** $I_d = k_d \sum_{i=1}^l (\mathbf{n} * \mathbf{l}_i) \cdot I_i / f(d_i)$
Reflexion an *matten* Oberflächen (Lambert’sche Objekte). Licht wird gleichmäßig in alle Richtungen reflektiert – Helligkeit **unabhängig vom Betrachterstandpunkt**. Beispiele: Papier, Tafel, unbearbeitetes Holz.
- **Glänzender (spekularer) Anteil** $I_g = k_g \sum_{i=1}^l (\mathbf{v} * \mathbf{r}_i)^\gamma \cdot I_i / f(d_i)$
Gerichtete Reflexion an spiegelnden Oberflächen. **Abhängig vom Betrachterstandpunkt** $\mathbf{v}$. Erzeugt Glanzpunkte. Beispiele: Metall, glänzender Kunststoff, Glas.

Lambert’sches Kosinus-Gesetz (Grundlage des diffusen Anteils)**Hinweis**

Lambert’sches Gesetz Die Lichtstärke I_0 eines flächenhaften Strahls variiert mit dem Kosinus des Winkels θ zur Normalen:

$$I = I_0 \cdot (\mathbf{n} * \mathbf{l}) = I_0 \cdot \cos \theta$$

Da der Mensch nur die **Leuchtdichte** L wahrnehmen kann, erscheint ein bestrahlter Körper unabhängig vom Betrachtungswinkel gleich hell: $I = L \cdot A \cdot \cos \theta$.

Anschaulich: Fällt Licht schräg auf eine Fläche, verteilt sich dieselbe Lichtmenge auf eine *größere* Fläche $\Rightarrow$ jeder einzelne Punkt bekommt weniger Licht $\Rightarrow$ die Fläche erscheint dunkler.

Einflussfaktoren

- k_a, k_d, k_g – Materialeigenschaften (Reflexionskoeffizienten) mit Farbkomponenten R, G, B
- $f(d_i)$ – Abstandsdämpfung: Licht wird mit zunehmender Entfernung d_i zur Quelle schwächer
- γ – Glanzexponent: kleines γ (z. B. 2) = breiter, verschwommener Glanzfleck; großes γ (z. B. 125) = kleiner, scharfer Glanzpunkt (metallisch wirkend)

Schattenberechnung

Erweiterung der Phong-Formel durch Schattenterm S_i :

$$I(\mathbf{v}) = k_a \cdot I_{amb} + \sum_{i=1}^l S_i \cdot \frac{I_i}{f(d_i)} \cdot \left[k_d(\mathbf{n} * \mathbf{l}_i) + k_g(\mathbf{v} * \mathbf{r}_i)^\gamma \right]$$

$$S_i = \begin{cases} 0 & \text{wenn Licht } i \text{ an diesem Punkt blockiert (Schatten)} \\ 1 & \text{sonst} \end{cases}$$

Realisierungsmöglichkeiten:

- **Analytischer Sichtbarkeitsalgorithmus**
- **Pixelbasiert (Shadow-Map):** Berechnet z-Buffer vom Betrachter- und vom Lichtquellen-Standpunkt aus.

Algorithmus: Für alle Pixel des Bildes:

1. Bestimme Koordinaten des Pixels aus Sicht der Lichtquelle (x, y, z)
2. Wenn Tiefe > Wert im z-Buffer der Lichtquelle $\Rightarrow$ Punkt liegt im Schatten ($S_i = 0$)

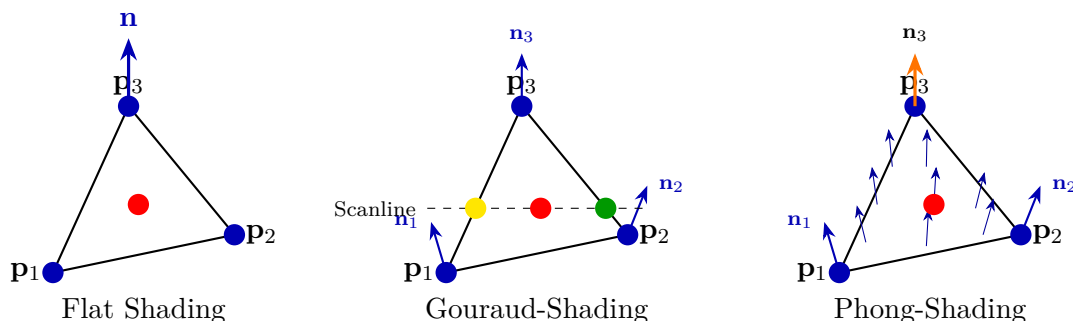
Shading-Modelle (D.3.3)

Beleuchtungsmodell = Lichtquellenmodell + Reflexions- und Transmissionsmodell (Material) + Lichttransportmodell

Ziel: Zuordnung von Farben zu den Pixeln polygonaler 3D-Modelle. Die Verfahren unterscheiden sich darin, *an welchen Stellen* das Beleuchtungsmodell ausgewertet wird.

Einfache Techniken (interpolative Shading-Verfahren): Über das Beleuchtungsmodell werden Farbwerte an den Eckpunkten berechnet; Farbwerte aller inneren Punkte ergeben sich durch **lineare Interpolation**.

Jeder Vertex p_j enthält: Koordinaten (x, y, z) , Normalenvektor (n_x, n_y, n_z) , Farbwert (r, g, b) .



Flat-Shading berechnet *einen einzigen* Farbwert für das gesamte Polygon – entweder als Mittelwert der Vertex-Farben oder für einen repräsentativen Punkt. Jeder Pixel des Polygons bekommt diese eine Farbe. Das Ergebnis: Man sieht jedes Dreieck als klare, gleichmäßig gefärbte Fläche – wie ein Papier-Polygon. Schnell, aber unrealistisch.

Gouraud-Shading wertet das Beleuchtungsmodell nur an den *Vertices* aus und interpoliert die Farben linear ins Innere (entlang der Scanlines). Dadurch verschwinden die harten Kanten zwischen Polygonen – die Oberfläche sieht glatt aus. Das Problem: Wenn ein Glanzpunkt physikalisch in der Mitte eines Polygons liegen müsste, liegt er auf keinem Vertex. Da nur zwischen den (dunklen) Eckfarben interpoliert wird, verschwindet der Glanzpunkt einfach.

Phong-Shading löst genau dieses Problem. Statt Farben zu interpolieren, werden die *Normalenvektoren* der Vertices über die Fläche interpoliert. An *jedem Pixel* wird dann das volle Beleuchtungsmodell mit dem lokalen interpolierten Normalenvektor ausgewertet. Dadurch entstehen korrekte Glanzpunkte auch im Inneren von Polygonen – zum Preis von deutlich mehr Berechnungen.

Beispiel

Klausurbeispiel: Rechenaufwand Dreieck mit **3 Eckpunkten** und **28 inneren Pixeln** (31 Punkte gesamt):

- **Gouraud:** 3 Beleuchtungsberechnungen (nur Vertices) + günstige Interpolation für 28 Pixel
- **Phong:** 31 Beleuchtungsberechnungen (jeder der 31 Punkte einzeln)

3. Clipping

Alles außerhalb des View Frustums abschneiden, damit nur sichtbare Geometrie weiterberechnet wird.

Clipping entscheidet für jedes Objekt: ist es **komplett sichtbar**, **komplett unsichtbar** oder **halb-halb**? Komplett unsichtbare Objekte werden sofort verworfen – das spart den teuersten Teil der Pipeline. Nur teilweise sichtbare Objekte müssen tatsächlich geschnitten werden.

Im 3D-Fall definiert das **View Frustum** (Sichtpyramide) den sichtbaren Bereich durch sechs Ebenen (links, rechts, oben, unten, nah, fern). Alles außerhalb fliegt raus.

Arten des Clippings:

- **Punkt-Clipping** – trivialer Vergleich mit den Grenzen des Bereichs
- **Strecken-Clipping** – z. B. Cohen-Sutherland: Endpunkte werden mit Bit-Codes klassifiziert, um Linien schnell zu akzeptieren oder zu verwerfen (*nicht prüfungsrelevant*)
- **Polygon-Clipping** – z. B. Sutherland-Hodgman (*nicht prüfungsrelevant*)
- **Kreis-Clipping** – erst das achsenparallele Hüllquadrat (Bounding Box) testen; nur bei Überschneidung den Kreis in Quadranten zerlegen und genauer untersuchen

Picking (D.4.3)

Picking ist das Rückproblem: Klick auf dem 2D-Bildschirm → welches 3D-Objekt ist gemeint? Man muss am Ende herausfinden, wo der Punkt im Szenenraum liegt.

Problem 1 – Dünne Objekte (Punkte, Linien): Ein einzelner Pixel lässt sich kaum exakt treffen. Lösung: Ein Objekt gilt als getroffen, wenn der Klick *hinreichend nah* dran liegt. Dafür braucht man ein Abstandsmaß.

Abstandsmaße zwischen Klick q und Objekt p :

- **Euklidischer Abstand:** $d = \sqrt{(q_x - p_x)^2 + (q_y - p_y)^2}$ – exakt, aber teuer (Wurzel)
- **Manhattan-Abstand:** $d = |q_x - p_x| + |q_y - p_y|$ – schnell, geeignete Näherung
- **Maximum-Metrik:** $d = \max\{|q_x - p_x|, |q_y - p_y|\}$ – noch einfacher

Hinweis

Tipp: Wurzel sparen Für den reinen Vergleich zweier Abstände muss die Wurzel *nicht* berechnet werden: $d_1 < d_2 \iff d_1^2 < d_2^2$. Man vergleicht einfach die Quadrate.

Problem 2 – Punkt im Polygon (Ray-Casting): Um zu testen, ob der Klick im Inneren eines Polygons liegt, schickt man einen Strahl vom Klickpunkt in eine beliebige Richtung und zählt die Kantenschnitte.

Definition

Point-in-Polygon-Test (Strahlschnitt-Regel) Ungerade Anzahl an Kantenschnitten $\Rightarrow$ Punkt liegt **im Polygon**.
Gerade Anzahl $\Rightarrow$ Punkt liegt **außerhalb**.

Effizienz-Vortest: Bei komplexen Objekten zuerst die **Bounding Box** (kleinstes achsenparalleles Rechteck) testen. Liegt der Punkt schon außerhalb der Box, entfällt der teure Genauigkeitstest.

4. Projektion

3D-Szene $\rightarrow$ 2D-Bildebene. Kernfrage: was ist sichtbar und wie groß erscheint es?

Lochkamera (das Standardmodell in der CG): kein Objektiv, nur ein Punkt (Projektionszentrum) und eine Bildebene. Kein Unschärfe-Effekt, da alles durch denselben Punkt läuft. In einer echten Lochkamera steht das Bild auf dem Kopf – in der CG schiebt man die Bildebene einfach *vor* das Projektionszentrum, dann ist es wieder aufrecht.

Brennweite f = Abstand zwischen Projektionszentrum und Bildebene. Sie bestimmt Zoom und Sichtfeld (Field of View):

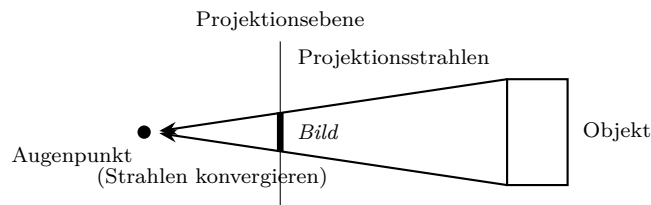
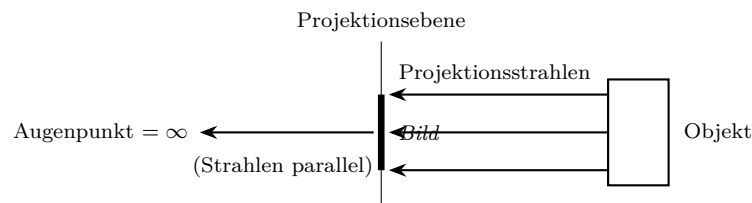
- Große $f \rightarrow$ Bildebene weit vom Projektionszentrum $\rightarrow$ kleiner Öffnungswinkel $\rightarrow$ **Teleobjektiv** (starker Zoom)
- Kleine $f \rightarrow$ Bildebene nah am Projektionszentrum $\rightarrow$ großer Öffnungswinkel $\rightarrow$ **Weitwinkelobjektiv**

Parallelprojektion: Das Projektionszentrum liegt im Unendlichen – die Kamera ist angenommen unendlich weit weg, daher fallen alle Projektionsstrahlen exakt parallel

auf die Bildebene. Die z-Koordinate wird einfach auf 0 gesetzt:

$$\tilde{\mathbf{P}} = (p_x, p_y, 0)^\top \quad \Leftrightarrow \quad \underline{\tilde{\mathbf{P}}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \underline{\mathbf{P}}$$

Objekte erscheinen gleich groß unabhängig vom Abstand. Geeignet für technische Zeichnungen und CAD.



Perspektivische Projektion: Das Projektionszentrum liegt im Endlichen – alle Strahlen laufen in einem Punkt zusammen. Entferntere Objekte erscheinen kleiner, weil sich dieselbe Größe auf einen kleineren Winkel verteilt. Entspricht dem menschlichen Sehen.

Die Formel ergibt sich direkt aus dem **Strahlensatz**:

$$\frac{\tilde{p}_x}{f} = \frac{p_x}{p_z} \quad \Rightarrow \quad \tilde{p}_x = f \cdot \frac{p_x}{p_z} \quad (\text{analog für } \tilde{p}_y)$$

Je größer p_z (weiter weg), desto kleiner $\tilde{p}_x$ – das ist die Tiefenkürzung. In homogenen Koordinaten:

$$\underline{\tilde{\mathbf{P}}} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & f & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \underline{\mathbf{P}}$$

f steht auf der Diagonale als Skalierungsfaktor; die letzte Zeile $(0, 0, 1, 0)$ übernimmt p_z als Nenner (homogene Division). In OpenGL zeigt die Kamera in die negative z -Richtung $\rightarrow$ letzte Zeile wird zu $(0, 0, -1, 0)$.

Hinweis

Parallel vs. Perspektivisch	Parallelprojektion		Perspektivisch
	Projektionszentrum	im Unendlichen	im Endlichen
	Strahlen	parallel	konvergieren im Projektionszentrum
	Größe mit Abstand	gleich	kleiner (Tiefenkürzung)
	Anwendung	CAD, techn. Zeichnung	Spiele, Film, VR

Kamerakalibrierung (D.4.5)

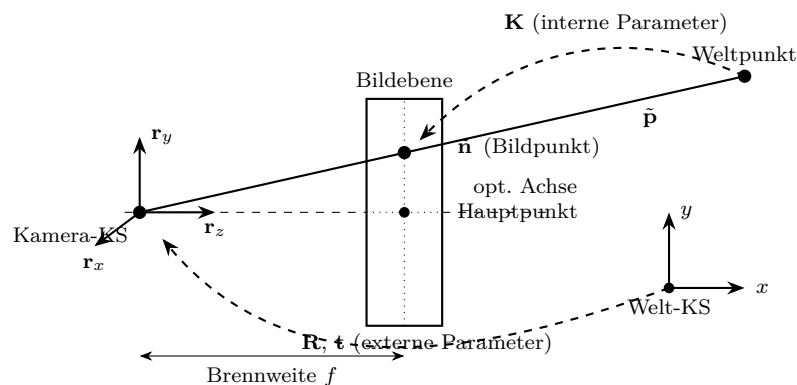
Das Lochkamera-Modell beschreibt die *ideale* Abbildung. Reale Kameras weichen davon ab – durch Linsenverzerrung, Sensorversatz, Pixelgröße. Kamerakalibrierung ermittelt die konkreten Parameter, um diese Abweichungen zu korrigieren.

Abbildungsfunktion:

$$\tilde{\mathbf{n}} = \mathbf{K} \circ \mathbf{L} \tilde{\mathbf{p}}$$

$\tilde{\mathbf{p}}$: 3D-Weltpunkt (homogene Koordinaten, 4D) $\tilde{\mathbf{n}}$: Ergebnis in homogenen Pixelkoordinaten (3D).

Tatsächliche Pixelposition: $n_x = \tilde{n}_x / \tilde{n}_z$, $n_y = \tilde{n}_y / \tilde{n}_z$.



Der **Projektionsstrahl** verläuft von der Kamera durch $\tilde{\mathbf{n}}$ auf der Bildebene weiter zum Weltpunkt $\tilde{\mathbf{p}}$. Die **externen Parameter** ($\mathbf{R}, \mathbf{t}$) beschreiben die Transformation vom Weltkoordinatensystem ins Kamerakoordinatensystem – also wo die Kamera sitzt und wohin sie zeigt. Die **internen Parameter** ($\mathbf{K}$) bilden den Weltpunkt auf den Bildpunkt auf der Bildebene ab – also wie Optik und Sensor zusammenarbeiten. Die **Brennweite** f ist der Abstand vom Projektionszentrum (Kamera-KS Ursprung) zur Bildebene; der **Hauptpunkt** ist der Schnittpunkt der optischen Achse mit der Bildebene.

Externe Kameraparameter $\mathbf{L}$ – Wo ist die Kamera und wohin schaut sie?

$$\mathbf{L} = \mathbf{R}^* (\mathbf{I}_3 \mid -\mathbf{t}) \quad (3 \times 4 \text{ Matrix})$$

wobei $\mathbf{I}_3$ die 3×3 Einheitsmatrix ist. Die Notation $(\mathbf{I}_3 \mid -\mathbf{t})$ bezeichnet die um $-\mathbf{t}$ erweiterte Matrix:

$$(\mathbf{I}_3 \mid -\mathbf{t}) = \begin{pmatrix} 1 & 0 & 0 & -t_x \\ 0 & 1 & 0 & -t_y \\ 0 & 0 & 1 & -t_z \end{pmatrix}$$

Sie verschiebt den homogenen Weltpunkt um $-\mathbf{t}$ (zieht die Kameraposition ab); anschließend dreht $\mathbf{R}$ das Ergebnis in die Kameraausrichtung.

- $\mathbf{t}$: Position des Projektionszentrums in Weltkoordinaten.
- $\mathbf{R} = (\mathbf{r}_x, \mathbf{r}_y, \mathbf{r}_z)$: Rotationsmatrix aus den normierten Kameraachsen; $\mathbf{r}_z$ zeigt entlang der optischen Achse.

L bringt jeden Weltpunkt ins Kamerakoordinatensystem.

Interne Kameraparameter K – *Wie bildet der Sensor einen Punkt auf Pixel ab?*

$$\mathbf{K} = \mathbf{D} \circ \mathbf{P} \quad (3 \times 3 \text{ Matrix})$$

Projektionsmatrix P – ideale perspektivische Projektion mit Brennweite f :

$$\mathbf{P} = \begin{pmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Digitalisierungsmatrix D – rechnet kontinuierliche Bildkoordinaten in diskrete Pixel um:

$$\mathbf{D} = \begin{pmatrix} 1/d_x & 0 & c_x \\ 0 & 1/d_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

- d_x, d_y : physische Größe eines Sensorelements auf dem Chip. Je größer ein Sensorelement, desto weniger Pixel passen drauf $\rightarrow$ größere d = gröberes Bild.
- c_x, c_y : **Hauptpunkt** (Bildmittelpunkt) – wo die optische Achse senkrecht auf den Sensor trifft. Liegt meist nahe der Bildmitte, aber nicht exakt (Fertigungstoleranz).

Linsenverzerrung: Reale Linsen erzeugen geometrische Verzerrungen, die durch Kalibrierung korrigiert werden:

- **Kissenförmig:** Kanten wölben sich nach innen – typisch bei Teleobjektiven.
- **Tonnenförmig:** Kanten wölben sich nach außen – typisch bei Weitwinkel.

View Frustum & NDC:

FOV (Field of View) ist der Öffnungswinkel der Kamera – wie viel von der Szene sie erfasst. Kleiner FOV = schmaler Blick, Objekte erscheinen groß (Teleobjektiv). Großer FOV = breiter Blick, mehr Szene sichtbar (Weitwinkel). Typisch: 60–90° in Spielen.

Das **View Frustum** (Sichtpyramide) ist der Bereich, den die Kamera tatsächlich sieht: ein Pyramidenstumpf, begrenzt durch die *near plane* (zu Nahes abschneiden), die *far plane* (zu Fernes abschneiden) und den FOV-Winkel. Alles außerhalb wird geclippt.

NDC (Normalized Device Coordinates): Das Problem ist, dass jede Kamera andere Parameter hat – anderen FOV, andere Auflösung, andere near/far-Abstände. Die GPU kann nicht für jede Kamera individuell rechnen. Die Lösung: die Projektionsmatrix quetscht das View Frustum in einen einheitlichen $2 \times 2 \times 1$ -Quader mit $x, y \in [-1, 1]$ und $z \in [-1, 1]$. Das sind die NDC. Alle sichtbaren Punkte landen garantiert in diesem Würfel – egal welche Kamera. Danach genügt eine einfache Multiplikation mit der Auflösung, um Pixelkoordinaten zu erhalten. Der Faktor $\frac{z_f + z_n}{z_n - z_f}$ in der Projektionsmatrix normiert genau die z -Koordinate in diesen Bereich.

5. Sichtbarkeitsberechnung

Welche Polygone sind vom Augenpunkt aus sichtbar, welche werden verdeckt? Ziel: nur das vorderste Objekt pro Pixel zeichnen.

Back-face Culling (Vorverarbeitungsschritt): Polygone, deren Normalenvektor vom Betrachter *wegzeigt*, sind Rückseiten – sie können niemals sichtbar sein und werden sofort verworfen. Das halbiert im Schnitt die Anzahl zu verarbeitender Polygone.

Kriterium: **Vorzeichen des Skalarprodukts** zwischen Blickrichtungsvektor $\mathbf{v}$ und Normalenvektor $\mathbf{n}$ des Polygons:

$$\mathbf{v} \cdot \mathbf{n} > 0 \Rightarrow \text{Rückseite (verwerfen)} \quad \mathbf{v} \cdot \mathbf{n} \leq 0 \Rightarrow \text{Vorderseite (behalten)}$$

Die verbleibenden Verfahren teilen sich in zwei Klassen – mit einem wichtigen architektonischen Unterschied:

- **Szenenbasiert:** erst Sichtbarkeit berechnen (Polygone sortieren), dann verastern. Reihenfolge: Sichtbarkeit $\rightarrow$ Verrasterung.
- **Rasterbildbasiert:** erst verastern (alle Polygone auf die Bildebene projizieren), dann pixelweise Sichtbarkeit entscheiden. Reihenfolge: Verrasterung $\rightarrow$ Sichtbarkeit.

Szenenbasierte Verfahren (D.5.2)

- **Painter's Algorithm:** Polygone werden nach fallender Entfernung sortiert und von hinten nach vorne gezeichnet – nähere übermalen fernere. Einfach, aber falsch bei **zyklischen Überlagerungen** (A verdeckt B, B verdeckt C, C verdeckt A) oder sich durchdringenden Polygonen, weil keine eindeutige Sortierung existiert.
- **BSP-Baum (Binary Space Partition):**

Aufbau (einmalig): Wähle ein Polygon P – seine Ebene teilt die Szene in zwei Hälften. Polygone, die diese Ebene schneiden, werden in Teilpolygone zerlegt. P wird Wurzel des Baums; die beiden Teilszenen werden rekursiv als linker und rechter Teilbaum aufgebaut.

Traversal (pro Frame): Durchlaufe zuerst den Teilbaum, dessen Halbraum den Augpunkt *nicht* enthält (= weiter entfernte Seite). Füge dann das Polygon des aktuellen Knotens vorne an die Prioritätsliste. Durchlaufe dann den anderen Teilbaum. Das Ergebnis ist eine korrekte Back-to-Front-Reihenfolge in $O(n)$.

Vorteil: Baum nur einmal bauen, beliebig oft für verschiedene Augenpunkte nutzen – ideal für **statische Szenen**.

Rasterbildbasierte Verfahren (D.5.3)

- **Z-Buffer (Tiefenpuffer):** Der PC verwaltet zwei Arrays der Größe $n \times m$ (Bildauflösung):
 - **Bildbuffer (Color-Buffer):** speichert die Farbe jedes Pixels – das, was am Ende auf dem Monitor zu sehen ist.
 - **Tiefenpuffer (Z-Buffer):** speichert für jeden Pixel die Tiefe z (Distanz zur Kamera) des Objekts, das diesen Pixel gerade „besitzt“.

Algorithmus: (1) Initialisierung: Bildbuffer mit Hintergrundfarbe füllen, Tiefenpuffer überall mit ∞ . (2) Verrasterung: alle Polygone nacheinander auf die Bildebene projizieren; für jeden Bildpunkt $[i, j]$ die exakte Tiefe z berechnen. (3) Tiefentest: ist $z < \text{Tiefenpuffer}[i, j]$? Ja $\rightarrow$ Farbe und Tiefenwert überschreiben. Nein $\rightarrow$ ignorieren. Einfach, schnell, in Hardware implementiert – **Standardverfahren** in moderner CG. Visualisierung: dunkel = nah, hell = weit.

Z-Fighting: Störender visueller Fehler – zwei Polygone liegen in der 3D-Welt exakt an derselben Stelle oder extrem nah beieinander. Farben und Texturen flackern ineinander, weil der Z-Buffer nicht entscheiden kann, welches Polygon vorne liegt. Ursache: begrenzter Speicher + Rundungsfehler bei der Tiefenberechnung $\rightarrow$ beide Tiefen sind gleich, keine deterministische Lösung. *Vorbeugen:* Geometrie physisch trennen; Near- und Far-Plane enger setzen (verbessert Z-Buffer-Auflösung).

- **Scan-Line-Verfahren:** Das Bild wird zeilenweise abgearbeitet. Pro Zeile: Schnittpunkte mit aktiven Polygonkanten bestimmen, nach x sortieren, dann mit dem **In/Out-Prinzip** füllen: von links nach rechts laufen – trifft man die erste Kante eines Polygons, geht man *hinein* (In); trifft man die nächste Kante desselben Polygons, geht man *hinaus* (Out). Alle Pixel im In-Intervall gehören zum Inneren und werden eingefärbt. Bei **überlagernden Polygonen:** PC hält eine Liste aktiver Polygone; an jedem Schnittpunkt wird geprüft, welches Polygon an dieser Stelle die geringste Tiefe (= am nächsten zur Kamera) hat – das gewinnt das Intervall.
- **Warnock-Algorithmus:** Die Bildebene wird rekursiv durch Vierteilung in Teilbilder zerlegt. Pro Teilbild werden Polygone in vier Kategorien eingeteilt:
 - **außerhalb:** komplett außerhalb des Teilbilds $\rightarrow$ ignorieren
 - **innerhalb:** komplett innerhalb $\rightarrow$ direkt zeichnen
 - **schneidend:** schneidet den Rand des Teilbilds
 - **umfassend:** umschließt das gesamte Teilbild

Abbruch (kein weiteres Unterteilen) wenn: alle Polygone außerhalb / nur ein Polygon vorhanden / ein umfassendes Polygon verdeckt alle anderen / Teilbild hat Pixelgröße.

Hinweis

	Verfahren	Typ	Merkmal
Verfahren im Vergleich	Back-face Culling	Vorverarbeitung	Rückseiten sofort verwerfen
	Painter's Algorithm	szenenbasiert	einfach, scheitert an Zyklen
	BSP-Baum	szenenbasiert	statische Szenen, $O(n)$ pro Frame
	Z-Buffer	bildbasiert	Hardware-Standard, pixelweise T
	Scan-Line	bildbasiert	zeilenweise Verarbeitung
	Warnock	bildbasiert	rekursive Vierteilung

6. Verrasterung (Rasterization)

Umwandlung von Dreiecken und Linien in diskrete Pixel. Welche Pixel liegen im Inneren eines Polygons?

Bresenham-Algorithmus (Strecken-Verrasterung):

Ein Bildschirm hat nur ganzzahlige Pixel. Eine diagonale Linie geht durch das Raster – pro Schritt nach rechts ($x + 1$) muss entschieden werden: bleibe ich auf der gleichen Zeile, oder gehe ich eine Zeile hoch?

Idee: Statt die ideale Linie für jeden Pixel auszurechnen (teuer), testet Bresenham den **Mittelpunkt** ($i, j + 0.5$) zwischen den zwei Kandidaten j und $j + 1$ – also genau den Punkt zwischen den zwei möglichen Pixeln:

- $F(i, j + 0.5) > 0 \rightarrow$ Mittelpunkt liegt *oberhalb* der Linie $\rightarrow j := j + 1$
- $F(i, j + 0.5) \leq 0 \rightarrow$ Mittelpunkt liegt *unterhalb* $\rightarrow j$ bleibt

Warum effizient? F muss nie neu berechnet werden – je nachdem ob j erhöht wurde oder nicht, ändert sich F nur um eine feste Konstante (Addition). Keine Multiplikation, keine Division.

Besonderheit in OpenGL: Linien werden „halboffen“ gezeichnet – der **Endpunkt wird nicht mitgezeichnet**, damit bei zusammengesetzten Linienzügen kein Punkt doppelt gerendert wird.

Polygon-Verrasterung (Scan-Line-Verfahren):

Ziel: alle Pixel im Inneren eines Polygons finden. Das Scan-Line-Verfahren arbeitet zeilenweise:

1. Alle Kanten des Polygons nach ihrem **maximalen y -Wert** sortieren.
2. Die Scan-Line läuft von oben nach unten durch jede Zeile.
3. Pro Zeile: **aktive Kanten** bestimmen – welche Kanten schneidet die aktuelle Zeile? Die Schnittpunkte werden nach x sortiert.
4. Zwischen den Schnittpunkten abwechselnd füllen/nicht füllen (**Paritätsregel**): erstes Intervall innen, zweites außen, usw.
5. Die x -Koordinaten für die nächste Zeile werden **inkrementell** aus der Kantensteigung berechnet – kein Neuberechnen nötig.

Spezialfall: Horizontale Kanten werden ignoriert. Pixel auf der linken/unteren Kante eines Polygons gehören dazu, auf der rechten/oberen nicht – so entstehen keine Lücken oder Doppelbelegungen bei benachbarten Polygonen.

Anti-Aliasing (D.6.3):

Problem (Aliasing): Verasterte Linien sehen treppenartig aus – „Zerfallen“ der Strecke in einzelne Pixelstufen. Ursache: zu grobe Abtastung (das Pixelraster ist zu grob für die ideale Linie).

Lösung: Statt einem Pixel entweder ganz zu zeichnen oder nicht, werden **Graustufen/Intensitäten** genutzt. Die Helligkeit eines Pixels ist proportional zum Anteil seiner Fläche, der von der (leicht verbreiterten) Linie überdeckt wird – Pixel direkt auf der Linie = hell, Pixel am Rand = halb-transparent. Bei Farbdarstellung: Überblendung zwischen Streckenfarbe und Hintergrundfarbe entsprechend dem Überdeckungsanteil.

Hinweis

Supersampling **Supersampling** ist eine konkrete Methode für Anti-Aliasing: Jedes Pixel wird in ein feines Untergitter unterteilt (z. B. $4 \times 4 = 16$ Subpixel), jeder Subpixel einzeln berechnet, und die Farben anschließend **gemittelt** (Downsampling). Das Ergebnis: weiche Übergänge an Kanten, weil die Subpixel unterschiedlich stark von der Geometrie überdeckt werden.

Intuition: Man rendert das Bild intern in höherer Auflösung und skaliert es dann herunter – das Auge sieht keine Stufen mehr, sondern einen glatten Übergang.

Nachteil: Rechenaufwand steigt quadratisch mit dem Upsampling-Faktor. **MSAA** (Multisample AA) ist eine GPU-optimierte Variante, die nur an Kanten abtastet.

Koordinatentransformations-Pipeline

- **Modell-Transformation** (M_{model}): platziert das Objekt aus seinem lokalen Koordinatensystem in die gemeinsame Weltszene (Translation, Rotation, Skalierung).
- **View-Transformation** (M_{view}): verschiebt die gesamte Szene so, dass die Kamera im Ursprung sitzt und entlang $-z$ blickt. Modell- und View-Matrix werden oft zur **Modelview-Matrix** zusammengefasst.
- **Projektions-Transformation** (M_{proj}): definiert das Sichtfeld (View Frustum); überführt in den Clip Space. Hier findet Clipping statt.
- **Perspektivische Division** ($\div w$): teilt alle Koordinaten durch die homogene Komponente $w \rightarrow$ erzeugt den perspektivischen Tiefeneffekt und liefert NDC mit $x, y, z \in [-1, 1]$.
- **Viewport-Transformation**: skaliert NDC auf die tatsächliche Pixelfläche des Ausgabefensters.

Gesamte Transformation im Vertex Shader:

$$v_{\text{clip}} = M_{\text{proj}} \cdot M_{\text{view}} \cdot M_{\text{model}} \cdot v$$

Durch homogene Koordinaten lassen sich alle Stufen zu einer einzigen **Model-View-Projection-Matrix** (MVP) kombinieren.

E. GPU-basierte Graphik-Pipeline

Die abstrakte 6-Schritte-Pipeline (Kapitel D) beschreibt *was* berechnet wird. Die GPU-Pipeline beschreibt *wie* es in Hardware umgesetzt wird.

Client-Server-Prinzip: OpenGL trennt CPU (Client) und GPU (Server). Die CPU verwaltet die Szene und schickt Befehle; die GPU verarbeitet alles parallel und schreibt das fertige Bild in den Framebuffer. Vorteil: OpenGL ist dadurch **netzwerktransparent** – Client und Server können auf verschiedenen Rechnern laufen.

Zustandsmaschine (State Machine): Die GPU befindet sich immer in einem definierten Zustand (aktive Textur, Shader, Transformationsmatrix usw.). Ein gesetzter Zustand bleibt erhalten, bis er explizit geändert wird – jede Zeichenoperation wird durch den aktuellen Zustand beeinflusst. Zustände können mit `glGet*` / `glIsEnabled` gelesen werden, was aber die Pipeline zum Stillstand bringen und die Performance beeinträchtigen kann.

Objektkonzept: OpenGL kapselt Ressourcen (Texturen, Puffer, Shader) in Objekten – kein OOP, sondern ein **Handle-System**: `glGen*` erzeugt ein Objekt, `glBind*` aktiviert es, `glDelete*` gibt es frei.

Feste vs. programmierbare Pipeline:

Früher war die Pipeline „fest verdrahtet“ – kein Einfluss auf Beleuchtung oder Texturierung möglich. Moderne GPUs (OpenGL 2.0+) erlauben programmierbare Stufen via **GLSL-Shader**:

- **Vertex Shader** (programmierbar): verarbeitet jeden Eckpunkt einzeln – Transformation (Model-View-Projektion) und Beleuchtung pro Vertex (Gouraud). Kann keine Vertices löschen oder neu erzeugen.
- **Rasterizer** (fest): wandelt Dreiecke in Fragmente um und interpoliert Vertex-Attribute (Farben, Normalen) über die Fläche. Nicht programmierbar.
- **Fragment Shader** (programmierbar): berechnet die endgültige Farbe pro Fragment – Texturierung, Phong-Shading pro Pixel. Kann Fragmente mit `discard` verwerfen (→ werden nicht gezeichnet).

Zentrale Begriffe:

- **Vertex:** Enthält alle Attribute eines Punktes: 3D-Koordinaten, Farbe, Normale, Texturkoordinaten.
- **Fragment:** Ein „potenzieller Pixel“ mit Farbe, Tiefe, Alpha – entsteht durch Verrasterung. Noch nicht endgültig sichtbar (kann im Z-Test scheitern).
- **Pixel:** Die tatsächlich sichtbare Information auf dem Bildschirm – das Ergebnis nach allen Tests.
- **Framebuffer:** Die digitale Leinwand (Rastergrafik), in die OpenGL die fertigen Pixel schreibt.

SIMD-Prinzip: Derselbe Shader-Code läuft gleichzeitig auf Tausende von Vertices/Fragmenten (Single Instruction, Multiple Data). Deshalb hat die GPU viele kleine parallele Kerne.

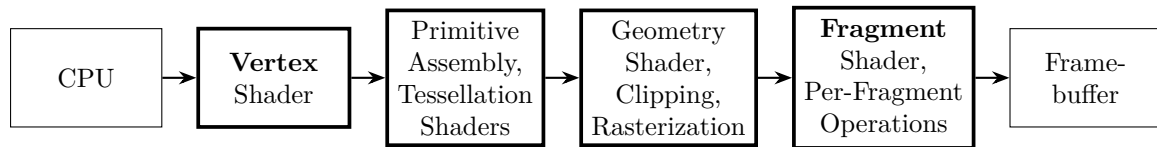
GLSL (OpenGL Shading Language): Shader werden in GLSL geschrieben – C-ähnliche Syntax, mit Vektoren (`vec2/3/4`) und Matrizen (`mat4`) als eingebaute Typen. Jeder Shader braucht zwingend eine `void main()`. Rekursion ist verboten.

Variablen-Qualifizierer steuern den Datenfluss:

- `in` / `out` – Eingabe vom vorherigen Schritt / Ausgabe an den nächsten Schritt

- **uniform** – von der CPU gesetzt, für alle Vertices/Fragmente eines Draw Calls konstant (z. B. Transformationsmatrix)
- **buffer** – globaler GPU-Speicher, les- und schreibbar

Stufen der GPU-Pipeline (in Reihenfolge):



- **Vertex Shader** (programmierbar): Transformation (MVP) und Beleuchtung pro Vertex.
- **Primitive Assembly + Tessellation**: Punkte zu Dreiecken zusammenfügen; Tessellation Shader unterteilt grobe Geometrie (Patches) in feinere Primitive – für LOD.
- **Geometry Shader + Clipping + Rasterization**: Geometry Shader kann Primitive ändern/erzeugen (optional); Clipping verwirft unsichtbare Geometrie; Rasterizer wandelt Dreiecke in Fragmente um.
- **Fragment Shader** (programmierbar): Texturierung + Phong-Beleuchtung pro Fragment. Kann Fragmente mit `discard` verwerfen.
- **Framebuffer**: Ziel aller fertigen Pixel nach Z-Test und Blending.

Compute Shader: Allgemeine GPU-Berechnungen *außerhalb* der Grafik-Pipeline (z. B. Physik, KI) – kein fester Platz in der Pipeline-Kette.

Datenübertragung CPU → GPU:

- **Immediate Mode (veraltet)**: Vertices einzeln mit `glVertex` senden – GPU wartet auf CPU, extrem ineffizient.
- **VBO (Vertex Buffer Object)**: Großer Speicherblock direkt auf der GPU – Vertex-Daten (Positionen, Farben, Normalen) werden einmal hochgeladen und dann beliebig oft genutzt.
- **Element Array Buffer (Index Buffer)**: Speichert Indizes in das VBO – ermöglicht, gemeinsame Vertices zwischen Dreiecken nur einmal zu speichern (spart Speicher).
- **VAO (Vertex Array Object)**: Speichert den Zustand der Vertex-Attribute und deren Verknüpfung mit VBOs – ein einziger Bind-Befehl aktiviert die gesamte Geometrie.
- **Texture Object**: Speichert Bilddaten auf der GPU. Zugriff aus dem Fragment Shader erfolgt über **Sampler Objects**, die das Sampling (Filterung, Wrapping) steuern.

- **FBO (Frame Buffer Object):** Die digitale Leinwand – wohin OpenGL das fertige Bild schreibt. Kann auch als Zwischenziel für Offscreen-Rendering verwendet werden (z. B. Schatten, Post-Processing).

Fensterverwaltung: Die OpenGL-API selbst hat kein Fenster- oder Eingabe-System. Hilfsbibliotheken wie **GLUT** oder **GLFW** übernehmen die Fenstererstellung und Benutzerinteraktion.

E.2. Szenengraphen

Ein Szenengraph ist eine hierarchische Datenstruktur zur Verwaltung komplexer 3D-Szenen. Grundstruktur: **gerichteter azyklischer Graph (DAG)** – keine Zyklen, Beziehungen nur in eine Richtung. In der einfachsten Form ein Baum.

Knotentypen:

- **Wurzelknoten:** Ausgangspunkt der gesamten Szene.
- **Gruppenknoten (innere Knoten):** Fassen Kindknoten zusammen. Enthalten **Transformationen** (Translation, Rotation, Skalierung als homogene Matrix), die auf den gesamten darunterliegenden Teilbaum wirken.
- **Blattknoten:** Enthalten die eigentliche **Geometrie** (Primitive: Punkte, Strecken, Polygone) oder andere Eigenschaften (Klang, Verhalten). **Geometrie steht immer in den Blättern.**

Wichtige Konzepte:

- **Attributvererbung:** Farbe, Textur, Reflexion usw. werden vom Elternknoten an Kindknoten vererbt, sofern nicht explizit überschrieben.
- **Lichtquellen und Kameras** sind ebenfalls Knoten – sie können an Geometrien geheftet werden und bewegen sich mit ihnen (z. B. Autoscheinwerfer).
- **Wiederverwendung:** Derselbe Teilbaum kann mehrfach referenziert werden – vier identische Räder = ein Knoten, viermal eingebunden.
- **Zustandspfad:** Der Pfad von Wurzel zu einem Blatt definiert den vollständigen Zustand (alle kumulierten Transformationen + Materialien) dieses Objekts.

Java 3D (Implementierungsbeispiel):

- **VirtualUniverse** → **Locale** (Referenzkoordinatensystem)
- **Inhaltsgraph (Content Branch):** Geometrie, Licht, Klang, Transformationen.
- **Ansichtgraph (View Branch):** Kameraparameter (Ort, Blickrichtung).
- **Node** = Knoten im Graphen; **NodeComponent** = Geometrie/Aussehen, wird von Knoten referenziert, ist aber selbst kein Knoten.

Hinweis

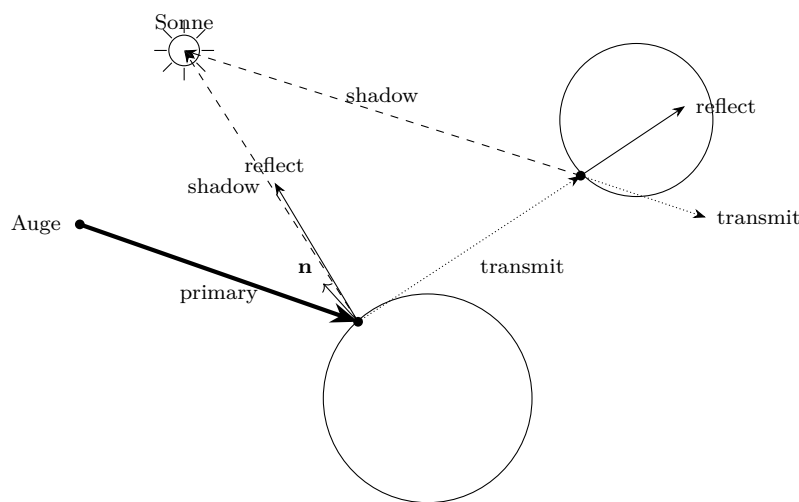
Prüfungshinweis **Geometrie steht immer in den Blättern.** Transformationen stehen in den inneren Gruppenknoten und wirken auf alle darunter liegenden Knoten.

Globale Beleuchtungsmodelle

Das Phong-Modell ist *lokal* – es betrachtet nur eine Lichtquelle und einen Objektpunkt. Globale Modelle berücksichtigen die Wechselwirkung **aller Objekte untereinander**: indirekte Beleuchtung, Schlagschatten, Spiegelungen, Brechungen.

Ray Tracing (Strahlverfolgung) – „From Eye to Light:

Statt Objekte auf den Bildschirm zu projizieren, wird der Lichtweg *rückwärts* verfolgt – vom Betrachter in die Szene.



Hauptalgorithmus: Für jedes Pixel (i, j) der Auflösung $m \times n$ wird ein **Sehstrahl** vom Augenpunkt a durch das Pixelzentrum in die Szene geschickt. Dann wird **Strahlverfolgung**(a, v) aufgerufen.

Unteralgorithmus **Strahlverfolgung**(p, v):

1. **FindeSchnitt:** nächsten Schnittpunkt q des Strahls mit einem Objekt bestimmen.
2. **Lokale Beleuchtung** (Phong) am Punkt q berechnen.
3. **Schattenstrahlen:** für jede Lichtquelle L_i einen Strahl von q nach L_i schicken – trifft er kein Objekt, trägt L_i zur Farbe bei (kein Schatten); trifft er ein Objekt, liegt q im Schatten.
4. **Rekursion:**
 - spiegelnde Oberfläche $\rightarrow$ Reflexionsrichtung r berechnen, **Strahlverfolgung**(q, r) aufrufen
 - transparente Oberfläche $\rightarrow$ Brechungsrichtung b berechnen, **Strahlverfolgung**(q, b) aufrufen

5. **Pixelfarbe** = gewichtete Summe aus lokalem Beitrag + Reflexion + Brechung.

Abbruchbedingungen: maximale Rekursionstiefe erreicht **oder** Intensität unter Schwellenwert (zu schwach um sichtbar zu sein).

Eingabegrößen: 3D-Szene (Geometrie), Lichtquellenpositionen, Oberflächenattribute (Reflexionsgrad), Augenpunkt + Projektionsebene, Bildauflösung.

Effizienz beim Ray Tracing

Das naive Testen jedes Strahls gegen jedes Dreieck ist zu langsam ($O(n)$ pro Strahl). Lösung: **Beschleunigungsstrukturen**, die den Raum oder die Geometrie hierarchisch organisieren, sodass der Strahl nur wenige relevante Objekte testen muss.

Hüllquader (Bounding Volumes): Jedes Objekt wird in eine einfache geometrische Form eingeschlossen (z. B. achsenparallele Box = AABB). Trifft der Strahl den Hüllquader *nicht*, wird das Innere komplett übersprungen. *Vorteil:* schneller Schnitttest. *Nachteil:* Hüllquader approximiert nur – kann den Strahl „treffen“, auch wenn er das eigentliche Objekt verfehlt (false positive → weiterer Test nötig).

Geschachtelte Hüllquader (Bounding Volume Hierarchy, BVH): Hüllquader werden hierarchisch zu einem Baum zusammengefasst. Man prüft von oben nach unten: trifft der Strahl keinen Elternknoten, werden alle Kinder übersprungen. Sehr populär in der Praxis (z. B. in Vulkan/DXR).

kd-Baum (k-dimensional tree): Der Raum wird rekursiv durch achsenparallele Ebenen in zwei Hälften geteilt – abwechselnd entlang x , y , z . Stell dir vor, du legst erst eine vertikale Wand in die Szene, dann eine horizontale, dann eine Tiefenwand, und so weiter. Jede entstandene Zelle enthält nur die Dreiecke, die in ihr liegen. Beim Raytracing verfolgt man den Strahl durch den Baum: liegt eine Zelle komplett neben dem Strahl, wird sie übersprungen – nur die Zellen, die der Strahl wirklich durchquert, müssen geprüft werden. *Vorteil:* funktioniert sehr gut, wenn die Geometrie ungleichmäßig verteilt ist (große leere Bereiche werden schnell übersprungen).

Octree: Rekursive Unterteilung des Raums in $2 \times 2 \times 2 = 8$ **gleich große Würfel**. Einfach zu implementieren, aber weniger adaptiv als kd-Baum.

Reguläres Gitter: Szene wird in gleichmäßige Voxel unterteilt. Jeder Voxel enthält die ihn durchschneidenden Primitive. Sehr schnelle Traversierung, aber ineffizient bei ungleichmäßig verteilter Geometrie (leere Zellen verschwenden Speicher). **Hierarchische Gitter** lösen das durch Untergitter in dichten Bereichen.

Hinweis

Engpass bei Echtzeit-Ray Tracing Moderne GPUs (NVIDIA RTX, RT-Cores) können Ray Tracing in Echtzeit berechnen – der Schlüssel ist massiver **SIMD-Parallelismus** (viele Strahlen gleichzeitig). Der eigentliche **Engpass** ist die Aktualisierung der Suchdatenstruktur (z. B. BVH) bei *bewegten Szenen*: die BVH muss jedes Frame neu aufgebaut oder refittet werden. Statische Szenen profitieren am meisten von Echtzeit-Ray Tracing.

Radiosity (Strahlungsverfahren) – „From Light to Surface:

Physikalisch basiertes Verfahren für **diffuse** Oberflächen (Lambert). Die Szene wird

in kleine Teilflächen (**Patches**) zerlegt. Licht wird von Lichtquellen ausgesendet und iterativ zwischen Patches ausgetauscht, bis ein **Energiegleichgewicht** erreicht ist. Effekte: weiches indirektes Licht, Farbbluten zwischen Oberflächen. **Vorteil:** Lösung ist blickpunktunabhängig – einmal berechnet, freie Kamerabewegung ohne Neuberechnung.

Hinweis			
Ray Tracing vs. Radiosity		Ray Tracing	Radiosity
	Prinzip	From Eye to Light	From Light to Surface
	Reflexion	spekular (glänzend)	diffus (matt)
	Haupteffekt	Spiegelung, Brechung	indirektes Licht, Farbbluten
	Aufwand	abhängig von Pixelzahl	abhängig von Geometrie (Patch)
	Blickpunkt	abhängig	unabhängig

Rendering Equation (Kajiya): Beschreibt die abgestrahlte Radianz an einem Punkt als Summe aus Eigenleuchten und dem integral über alle einfallenden Richtungen reflektierten Licht. Eine **exakte Lösung ist im Allgemeinen nicht möglich** – Ray Tracing und Radiosity sind Näherungsverfahren dafür.

7 E – Computergraphik und OpenGL

Kapitel D beschreibt, *was* bei der Bilderzeugung berechnet wird. Kapitel E beschreibt, *wo* es berechnet wird und *wie* man es beauftragt: die Graphik-Pipeline auf der GPU, ihre Programmierschnittstelle OpenGL, die Szenenverwaltung im Szenengraphen und als Gegenentwurf die Strahlverfolgung.

7.1 Von der festen zur programmierbaren Pipeline

Die monolithischen Graphik-Workstations der 80er Jahre wurden durch modulare GPUs ersetzt. Frühe GPUs implementierten eine Pipeline mit **festen Funktionen** (fixed function) in Hardware; ab den späten 90er Jahren wurden sie programmierbar, Meilenstein ist die NVIDIA GeForce 3 von 2001 mit den ersten programmierbaren Shadern. Heutige GPUs sind massiv parallele Prozessoren, und OpenGL ist eine Schnittstelle zu ihnen.

Klassische Pipeline	OpenGL fixed function	Programmierbar
Graphikelemente	Graphikelemente	Graphikelemente
Transformation	Transformation	
Beleuchtung	Beleuchtung	Vertex Shader
Projektion und Clipping	Projektion	
	Clipping	Clipping
Verrasterung	Verrasterung, Interpolation, Tiefe	Verrasterung, Interpolation, Tiefe
Textur und Beleuchtung	Textur und Beleuchtung (Phong)	Fragment Shader
Sichtbarkeitsbestimmung	Sichtbarkeitsbestimmung	Fragment Tests
Bild	Bild	Bild

Die programmierbare Pipeline ersetzt zwei feste Blöcke durch Programme: der **Vertex Shader** übernimmt Transformation, Beleuchtung und Projektion pro Eckpunkt, der **Fragment Shader** Texturierung und Beleuchtung pro Fragment. Alles dazwischen, Clipping, Verrasterung, Tiefeninterpolation, bleibt fest verdrahtet, weil es sich nicht sinnvoll variieren lässt.

Definition

Vertex: alle Attribute zur vollständigen Spezifikation eines Eckpunktes eines Primitives: 3D-Koordinaten, Farbe, Normale, Texturkoordinaten.

Fragment: die Menge von Daten (Farbe, Koordinaten, Tiefe, Alpha), die zum Einfärben *eines* Pixels nötig sind. Ein Pixel kann von mehreren Fragmenten beansprucht werden; erst die Fragment-Tests entscheiden, welches gewinnt.

Pixel: die sichtbare Information auf dem Bildschirm, gemessen in Bytes im Framebuffer.

Framebuffer: die Leinwand, auf die OpenGL zeichnet, eine Rastergrafik.

Hinweis

Der Unterschied zwischen Fragment und Pixel ist klausurrelevant und wird häufig verwechselt: die Verrasterung erzeugt *Fragmente*, und ein Fragment wird erst durch die Fragment-Tests (Tiefentest, Alphatest, Stenciltest) zu einem beitragenden Pixelwert oder verworfen.

7.2 OpenGL als Schnittstelle

OpenGL ist ein Software-Interface für Graphik-Hardware, hardware- und herstellernabhängig, überwacht vom Architecture Review Board und weiterentwickelt von der Khronos-Gruppe. Bewusst **nicht** enthalten sind Fensterverwaltung und Benutzereingabe; dafür sind Zusatzbibliotheken zuständig.

Baustein	Aufgabe
OpenGL ES	für eingebettete und handgehaltene Geräte, reduzierter Funktionsumfang, energieeffizient; Version 3.2 beruht auf OpenGL 3.x
WebGL	JavaScript-Implementierung von OpenGL ES 2.0 im Browser, ohne Erweiterungen
GLUT/freeglut	fenstersystemunabhängige Schnittstelle zum Fenstersystem und zur Eingabe, Präfix <code>glut</code>
GLU	Hilfsfunktionen für Matrizen, Polygonzerlegung, NURBS-Flächen, Präfix <code>glu</code> ; in aktuellen Versionen nicht mehr unterstützt, ersetzt durch <code>glm</code>
GLEW	Laufzeitmechanismus zur Abfrage der auf der Zielpattform verfügbaren Extensions
Vulkan	moderne Alternative mit deutlich geringerem Treiber-Overhead; die Anwendung linkt gegen einen <i>Loader</i> statt direkt gegen den Treiber

Ein OpenGL-Programm kennt nur drei Arten von Funktionen: Zustand ändern, Daten übergeben, Geometrie darstellen. Befehle beginnen mit `gl`, Konstanten mit `GL_`.

Immediate Mode und Retained Mode

Im **Immediate Mode** werden Primitive sofort nach ihrer Festlegung an die Pipeline geschickt; die GPU kennt immer nur die Primitive, die gerade durchlaufen. Der Code ist kurz und die Verarbeitung langsam, weil jeder Aufruf nur wenige Daten überträgt und die Hardware ständig auf Nachschub wartet:

```
glBegin(GL_TRIANGLES);
    glVertex3f(-0.5f, -0.5f, 0.0f);
    glVertex3f( 0.5f, -0.5f, 0.0f);
    glVertex3f( 0.0f, 0.5f, 0.0f);
glEnd();
```

Seit OpenGL 3.1 werden `glBegin` und `glVertex` nur noch im Compatibility Profile unterstützt. Der **Retained Mode** überträgt die Vertexdaten stattdessen in großen Blöcken und hält sie in **Buffer Objects** auf der Graphikkarte.

Vertex Buffer Objects und Vertex Array Objects

Ein VBO wird in drei Schritten installiert und in drei weiteren benutzt:

1. `glGenBuffers(1, &bufID)` erzeugt einen eindeutigen Bezeichner
2. `glBindBuffer(GL_ARRAY_BUFFER, bufID)` erstellt und aktiviert den Puffer
3. `glBufferData(GL_ARRAY_BUFFER, size, inData, GL_STATIC_DRAW)` überträgt die Daten

Beim Zeichnen wird der Puffer gebunden, Stride und Startadresse werden festgelegt, und ein einziger Aufruf `glDrawArrays(GL_TRIANGLES, first, count)` rendert den gesamten Inhalt. Freigegeben wird mit `glDeleteBuffers`.

Ein **VAO** enthält demgegenüber **nur die Zustände**, nicht die Vertexdaten: welche Puffer gebunden sind, wie die Attribute im Speicher liegen. Es erlaubt, die Konfiguration mehrerer Teilszenen vorzubereiten und mit einem einzigen `glBindVertexArray` umzuschalten.

Hinweis

Die Datenübergabe ist der eigentliche Engpass, nicht die Rechenleistung. Der Übergang vom Immediate zum Retained Mode ist deshalb kein Stilwandel, sondern die Voraussetzung dafür, dass eine massiv parallele Einheit überhaupt ausgelastet werden kann.

7.3 Die Koordinatentransformations-Pipeline im Shader

Der Vertex Shader führt die Kette aus, die in Kapitel D hergeleitet wurde:

1. **Model Transform:** Anordnung der Objekte in der Welt
2. **View Transform:** Position und Orientierung der Kamera
3. **Projection Transform:** Festlegung des Kameraobjektivs, also Brennweite, Zoomfaktor und Sichtvolumen
4. **Viewport Transform:** Abbildung auf den Bildschirmbereich

Anschließend rasterisiert die feste Stufe jedes Primitiv in Fragmente und interpoliert dabei alle Vertex-Attribute. Der Fragment Shader arbeitet auf diesem Fragment-Strom; die vier Schritte der Gesamtverarbeitung heißen Vertex-Verarbeitung, Rasterisierung, Fragment-Verarbeitung und Zusammenführen der Ausgabe.

7.4 Szenengraphen

Ein Szenengraph ist die Datenstruktur, in der die CPU die Szene hält, bevor sie an die Pipeline geht.

Knotentyp	Bedeutung
Wurzel	die gesamte Welt
Gruppenknoten	enthalten eine Transformation als homogene Matrix und ein oder mehrere Kinder
Blätter	grundlegende 3D-Objekte

Neben der Geometrie enthält der Graph auch die **Erscheinungsform** (Farbe, Reflexion, Transparenz, Textur). Sie kann teilweise spezifiziert werden; nicht spezifizierte Werte werden **vererbt**, was Wiederverwendung ohne Duplikation erlaubt.

Lichtquellen und Kameras stehen ebenfalls im Graphen, weil auch sie Position und Richtung brauchen und weil sie dadurch in lokalen Koordinatensystemen von Geometriegruppen liegen können: Autoscheinwerfer bewegen sich mit dem Auto, die Kamera mit dem Fahrer, ohne dass eine einzige Transformation von Hand nachgeführt werden muss.

Java 3D formalisiert diese Struktur als Grammatik:

```

Szene   := Transformation Gruppe
Gruppe  := {Bestandteil, ..., Bestandteil}
Bestandteil := Transformation Gruppe | Graphikelement
Transformation := eine affine Transformation
Graphikelement := Strecke | Polygonzug | Markierung | ...

```

7.5 Strahlverfolgungsbasierte Bilderzeugung

Die Pipeline fragt für jedes Primitiv, welche Pixel es bedeckt. Die Strahlverfolgung fragt umgekehrt für jedes Pixel, welches Objekt dort sichtbar ist. Weil ein Strahl von jedem Punkt in jede Richtung ausgehen kann, sind Schatten, Spiegelung und Brechung dieselbe Operation, erneut angewandt.

Definition

Basisalgorithmus (Whitted):

1. schicke einen Strahl vom Betrachterstandpunkt durch jedes Pixel der Bildebene
2. bestimme den nächstgelegenen Schnittpunkt mit einem Objekt der Szene und dort den Beitrag des lokalen Beleuchtungsmodells (Phong)
3. berechne den Strahl in Reflexionsrichtung und gegebenenfalls in Brechungsrichtung
4. bestimme deren Beiträge **rekursiv** durch Verfolgen dieser Strahlen
5. die Farbe des Pixels ist die gewichtete Summe der drei Beiträge

Eingabe sind eine 3D-Szene aus Flächen, l Punktlichtquellen L_i , die Oberflächenattribute, der Augenpunkt a , die Projektionsebene und die Bildauflösung $m \times n$; Ausgabe ist ein $m \times n$ -Rasterbild.

Die Rekursion bricht ab, wenn eine der beiden Bedingungen erfüllt ist:

- die vorgegebene maximale Rekursionstiefe ist erreicht
- die Dämpfung auf dem Sehweg vom Augenpunkt zum aktuellen Punkt ist stärker als eine gegebene Schranke

Die zweite Bedingung ist die interessantere: sie bricht dort ab, wo ein weiterer Strahl das Ergebnis nachweislich kaum noch ändert, statt überall gleich tief zu rechnen.

Anti-Aliasing

Ein Strahl pro Pixel tastet die Bildfunktion an genau einem Punkt ab, und eine Kante ist eine Sprungstelle mit unbeschränktem Spektrum. Nach dem Abtasttheorem (Kapitel C.4) ist damit keine Auflösung hoch genug, weshalb pro Pixel mehrere Strahlen verschossen und gemittelt werden (Supersampling), bevorzugt mit unregelmäßiger Verteilung, damit das Restrauschen weniger auffällt als ein regelmäßiges Muster.

Strahl-Dreieck-Schnitt und Effizienz

Der Schnitt gegen ein Dreieck wird nach Möller und Trumbore direkt nach dem Strahlparameter und den beiden baryzentrischen Koordinaten aufgelöst, ohne die Ebene des Dreiecks explizit zu konstruieren. Die baryzentrischen Koordinaten werden ohnehin gebraucht, um Normalen und Texturkoordinaten zu interpolieren.

Der Aufwand ist ohne Beschleunigungsstruktur das Produkt aus Pixelzahl und Objektzahl. Deshalb wird die Szene in Hüllkörperhierarchien (BVH) oder Raumteilungsstrukturen (Octree, kd-Baum) organisiert: der Schnitt gegen einen achsenparallelen Quader ist billig und verwirft den größten Teil der Szene pro Strahl, ohne je etwas zu zeichnen.

8 F – Objektrepräsentation

8.1 Geometrie und Topologie

Objekte in der Computergrafik werden durch zwei unabhängige Informationsschichten beschrieben:

Geometrie beschreibt *was* ein Objekt ist und *wie groß* – also Form und Abmessungen im Raum: Koordinaten, Radien, Längen.

Topologie beschreibt *wie die Teile zusammenhängen* – Konnektivität und Assoziativität der Objekteinheiten: welche Linie grenzt an welche, wo beginnt ein Kreisbogen, wie sind die Elemente verbunden.

Hinweis

Geometrie $\neq$ Topologie Die beiden Eigenschaften sind unabhängig voneinander:

- **Gleiche Geometrie, unterschiedliche Topologie:** Dieselben Maße (gleiche Linienlängen, gleicher Radius), aber andere Verbindungsstruktur – z. B. der Kreisbogen schließt die Form anders ab.
- **Gleiche Topologie, unterschiedliche Geometrie:** Dieselbe Verbindungsstruktur (Kreis an drei Linien), aber die Form ist verzerrt – andere Koordinaten, anderer Radius.

8.2 Geometrische Modellierungstechniken

Es gibt vier grundlegende Ansätze, um 3D-Objekte im Computer zu repräsentieren – von einfach bis vollständig:

- **2D-Projektion (Zeichnungen):** Klassische Darstellung räumlicher Objekte in der Ebene (z. B. technische Zeichnungen). Kein echtes 3D-Modell – nur eine Ansicht.
- **Drahtgitter-Modellierung:** Nur die Kanten des Objekts werden dargestellt. Schnell und einfach, aber kein Volumen, keine Flächen – man sieht „durch“ das Objekt hindurch.
- **Surface Modeling (Flächenmodellierung):** Das Objekt wird als Hülle aus Flächen beschrieben. Enthält Geometrie (Form und Abmessungen), aber *keine*

Topologie – die Verbindungsstruktur zwischen den Flächen ist nicht gespeichert. Netzrepräsentation, kein Konstruktionsbaum. Umfasst analytische Flächen sowie Freiform- und skulpturale Oberflächen.

- **Solid Modeling (Volumenmodellierung):** Im Gegensatz zur Flächenmodellierung enthalten diese Modelle sowohl Volumen- als auch Randinformationen. Das Objekt ist vollständig und eindeutig definiert – man kann z. B. das Gewicht berechnen oder prüfen, ob ein Punkt innen liegt.

8.3 Darstellung von Kurven und Flächen

F.2.1 Parameterdarstellung einer Kurve

Die Koordinaten eines Punktes $\mathbf{p}$ auf der Kurve werden als Funktionen einer einzigen reellen Variable t (dem Parameter) beschrieben:

$$\mathbf{p} = \mathbf{k}(t), \quad t \in P \subseteq \mathbb{R}$$

$\mathbf{p}$ – der resultierende Punkt im Raum; $\mathbf{k}$ – die Abbildungsfunktion der **Kurve**; t – der Parameter. In 3D ergibt das drei Komponentenfunktionen $x(t)$, $y(t)$, $z(t)$. Der Parameterbereich P ist typischerweise ein Intervall wie $[0, 1]$.

Beispiele:

- **Kreis** in der xy -Ebene: $x = m_x + r \cos t$, $y = m_y + r \sin t$, $t \in [0, 2\pi]$
- **Schraubenlinie**: $x = \sin t$, $y = \cos t$, $z = t$

F.2.2 Explizite vs. implizite Darstellung – Beispiel Kreis

1) Explizite Darstellung – intuitiv, aber man muss mehrere Funktionen zusammenbauen:

$$y = f(x) = \sqrt{1 - x^2}, \quad y = -f(x) = -\sqrt{1 - x^2}$$

Der Kreis zerfällt in zwei Hälften, weil die explizite Form **rechtseindeutig** sein muss: für jedes x darf es nur *ein* y geben. Vorteil: man kann klare visuelle Geometrie direkt ablesen.

2) Implizite Darstellung – eine einzige Gleichung, dafür schwieriger abzuleiten:

$$F(x, y) = 0 \quad \Rightarrow \quad x^2 + y^2 - 1 = 0$$

$F = 0$ bedeutet: der Punkt liegt *genau auf dem Rand* des Objekts. Das erlaubt einen einfachen **Inzidenztest** (liegt ein Punkt auf der Kurve?) – Koordinaten einsetzen, Ergebnis gleich null? Auf der Kurve. Die implizite Form kennt keine „Rechtseindeutigkeit“ und beschreibt den ganzen Kreis auf einmal.

F.2.3 Parameterdarstellung einer Fläche

Erweiterung auf **zwei Parameter** (u, v) :

$$\mathbf{p} = \mathbf{f}(u, v), \quad (u, v) \in P \subseteq \mathbb{R}^2$$

$\mathbf{p}$ – der Punkt im Raum; $\mathbf{f}$ – die Abbildungsfunktion der **Fläche**; (u, v) – die zwei Parameter. Hält man einen Parameter fest (z. B. $u = u^*$), beschreibt $\mathbf{f}(u^*, v)$ eine

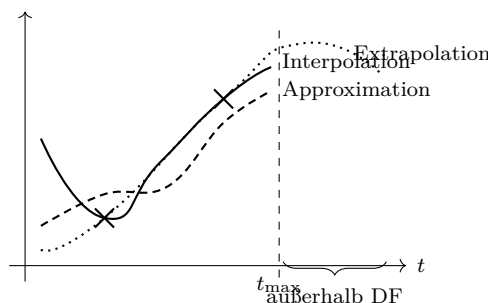
Kurve im Raum. Eine Fläche ist damit eine **Schar von Kurven** – man kann sie zeilenweise oder spaltenweise durchfahren.

Hinweis

Prüfungshinweis Für einfache Polynomkurven bestimmen die Koeffizienten die Form – aber das ist wenig intuitiv. In der Praxis und in der Klausur stehen deshalb **Bézier-Kurven und Splines** im Vordergrund.

8.4 Interpolation, Approximation, Extrapolation

- **Interpolation** – die Kurve geht *exakt durch* die gegebenen Punkte. Man verlässt den maximalen Definitionsbereich nicht.
- **Approximation** – grobe Beschreibung der Punktmenge durch *Fehlerminimierung*. Die Kurve muss nicht durch die Punkte gehen, liegt aber möglichst nah dran. Ebenfalls innerhalb des Definitionsbereichs.
- **Extrapolation** – wenn man *über die Grenzen* des Definitionsbereichs hinausgeht: Verlängerung der Kurve über die bekannten Punkte hinaus. Unsicher, da keine Datenbasis.



8.5 Bézier-Kurven

Grundidee

Statt einer Kurve durch Koeffizienten zu beschreiben, steuert man sie über **Kontrollpunkte** $b_0, b_1, \dots, b_n$ – das **Bézier-Kontrollpolygon**. Die Kurve folgt dem Polygon, berührt es aber nur an den Endpunkten. Sehr intuitiv: Kontrollpunkte verschieben $\rightarrow$ Kurve verändert sich vorhersehbar.

Bernstein-Polynome

Die mathematische Basis sind die **Bernstein-Polynome** vom Grad n :

$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad i = 0, \dots, n, \quad t \in [0, 1]$$

Sie wirken als **Gewichtsfunktionen** – jeder Kontrollpunkt b_i wird mit $B_i^n(t)$ gewichtet.

Eigenschaften:

- **Partition der Eins:** $\sum_{i=0}^n B_i^n(t) = 1$ – die Gewichte summieren sich immer zu 1.

- **Positivität:** $B_i^n(t) \geq 0$ für $t \in [0, 1]$.
- **Maximum:** $B_i^n(t)$ erreicht sein Maximum genau bei $t = i/n$ – Kontrollpunkt b_i hat dort den größten Einfluss.
- **Rekursion:** Ein Bernstein-Polynom vom Grad $n+1$ ist eine Konvexkombination zweier Bernstein-Polynome vom Grad n – Grundlage des De-Casteljau-Algorithmus.

Algorithmus von De Casteljau

Berechnung eines Kurvenpunkts $\mathbf{b}(t^*)$ durch **wiederholte lineare Interpolation:**

Man teilt jede Kante des Kontrollpolygons im Verhältnis $t^* : (1 - t^*)$ und verbindet die neuen Punkte zu einem kleineren Polygon. Das wiederholt man, bis nur noch ein Punkt übrig bleibt – dieser liegt auf der Kurve.

Eigenschaften des Algorithmus:

- **Aufwand:** $O(n^2)$ pro Kurvenpunkt.
- **Numerisch stabil:** Nur Konvexkombinationen, keine Subtraktionen großer Zahlen.
- **Kurvenunterteilung:** Die Zwischenpunkte sind die Kontrollpunkte der beiden Teilsegmente – jede Bézier-Kurve lässt sich so in zwei Bézier-Kurven gleichen Grades aufteilen.

Eigenschaften von Bézier-Kurven

- **Interpolation der Endpunkte:** Die Kurve geht *exakt* durch b_0 und b_n .
- **Tangente in den Endpunkten:** Die Kurve liegt dort tangential zum ersten bzw. letzten Segment des Kontrollpolygons.
- **Konvexe-Hülle-Eigenschaft:** Die gesamte Kurve liegt innerhalb der konvexen Hülle der Kontrollpunkte.
- **Beschränkte Schwankung:** Keine Gerade schneidet die Kurve öfter als das Kontrollpolygon – die Kurve „schwingt“ nie stärker als ihre Kontrollpunkte.

Hinweis

Prüfungshinweis Algebraische Herleitung und Graderhöhung sind **nicht prüfungsrelevant**. Wichtig: graphische Erklärung des De-Casteljau-Algorithmus + die vier Eigenschaften oben.

Grad und Kontrollpunkte – Zusammenhang

Bei einer Bézier-Kurve mit $n + 1$ Kontrollpunkten $b_0, \dots, b_n$ ist der Polynomgrad **fest** gleich n :

$$\text{Grad} = \underbrace{(\text{Anzahl Kontrollpunkte}) - 1}$$

Das bedeutet: **mehr Kontrollpunkte = höherer Grad**. Das hat direkte Konsequenzen:

- **Mehr Freiheit:** Eine Kurve vom Grad n kann $n - 1$ Wendepunkte haben – mehr Kontrollpunkte erlauben komplexere Formen.
- **Globale Wirkung:** Jeder Kontrollpunkt beeinflusst über sein Bernstein-Polynom die *gesamte* Kurve – kein Punkt ist „lokal“. Bei 10 Kontrollpunkten = Grad 9, jede Änderung wirkt überall.
- **Rechenaufwand steigt:** De Casteljau braucht $O(n^2)$ – bei hohem n teuer und numerisch instabil.

Fazit: Bézier ist ideal für einfache Formen mit wenigen Punkten. Für komplexe Modelle → B-Splines.

8.6 B-Splines

Nachteile der Bézier-Technik

- **Keine lokale Kontrolle:** Jede Änderung eines Kontrollpunkts verändert die *gesamte* Kurve – gezielte Korrekturen an einer Stelle sind nicht möglich.
- **Hoher Polynomgrad:** Mit jedem weiteren Kontrollpunkt steigt der Grad um 1 → hoher Rechenaufwand und numerische Ungenauigkeiten bei komplexen Formen.

Lösung: B-Splines

Statt einer einzigen Kurve hohen Grades setzt man die Form aus **Segmenten niedrigen Grades** zusammen. Die Übergänge sind glatt – der Nutzer sieht eine einzige Kurve.

Cox-de Boor-Rekursion – Basisfunktionen $N_{i,k}(t)$:

Basisfall ($k = 1$): Sprungfunktion –

$$N_{i,1}(t) = \begin{cases} 1 & t \in [x_i, x_{i+1}[\\ 0 & \text{sonst} \end{cases}$$

Rekursionsschritt: Höhere Ordnungen als gewichtete Summe zweier Funktionen niedrigerer Ordnung:

$$N_{i,k}(t) = \frac{t - x_i}{x_{i+1+k-1} - x_i} N_{i,k-1}(t) + \frac{x_{i+k} - t}{x_{i+k} - x_{i+1}} N_{i+1,k-1}(t)$$

Eine B-Spline-Kurve $P(t)$ wird durch **de Boor-Punkte** B_i (Kontrollpunkte), Basisfunktionen $N_{i,k}$ und einen **Knotenvektor** x_i definiert. Der Polynomgrad ist stets $k - 1$.

Eigenschaften von B-Splines

- **Lokale Kontrolle:** Ein Kontrollpunkt beeinflusst die Kurve nur in seinem **Support-Bereich** (bei Ordnung k : genau k Intervalle). Änderungen bleiben lokal.
- **Stetigkeit:** Eine B-Spline-Kurve der Ordnung k ist C^{k-2} -stetig (d. h. $(k-2)$ -mal stetig differenzierbar).
- **Konvexe Hülle:** Kurve liegt innerhalb der konvexen Hülle der Kontrollpunkte.
- **Partition der Eins:** $\sum_i N_{i,k}(t) = 1$ für alle t .

Ordnung, Grad und Kontrollpunkte – Zusammenhang

Bei B-Splines gibt es drei Parameter, die man unterscheiden muss:

Parameter	Symbol	Bedeutung
Anzahl Kontrollpunkte	n	wie viele de Boor-Punkte
Ordnung	k	Anzahl Punkte pro Segment
Grad	$k - 1$	Polynomgrad in jedem Segment

Ordnung k steuert alles außer der Anzahl der Kontrollpunkte:

- **Grad des Polynoms** = $k - 1$ (in jedem Segment). Ordnung 3 $\rightarrow$ quadratische Segmente, Ordnung 4 $\rightarrow$ kubische Segmente.
- **Stetigkeit** = C^{k-2} . Ordnung 4 $\rightarrow C^2$ -stetig (zweimal differenzierbar, glatte Kurve).
- **Support** = k Intervalle. Ein Kontrollpunkt beeinflusst genau k benachbarte Segmente – je größer k , desto weiter reicht sein Einfluss.

Mehr Kontrollpunkte n erhöhen *nicht* den Grad – sie fügen einfach mehr Segmente hinzu. Die Kurve wird länger/komplexer, bleibt aber überall Grad $k - 1$. Das ist der Kernvorteil gegenüber Bézier.

Maximalregel: Die Ordnung kann höchstens $k = n$ sein. Bei $k = n$ und offenem Knotenvektor $\rightarrow$ Bézier-Kurve.

Hinweis

Merkhilfe Bézier: Grad = Anzahl Punkte -1 (zwingend).
B-Spline: Grad = $k - 1$ (frei wählbar, unabhängig von n).

Knotenvektor

Der Knotenvektor $x_0 \leq x_1 \leq \dots$ steuert den Verlauf der Basisfunktionen. Einzige Bedingung: **Monotonie** ($x_i \leq x_{i+1}$), keine strikte Monotonie nötig – Mehrfachknoten sind erlaubt. Wird in der Cox-de Boor-Formel der Nenner null, gilt per Definition: $0/0 := 0$.

Typen:

- **Periodisch uniform:** gleichabständige Knoten über den gesamten Bereich. Die Basisfunktionen sind lediglich verschobene Kopien voneinander: $N_{i,k}(t) = N_{i-1,k}(t - 1)$. Kurve interpoliert die Endpunkte *nicht*.
- **Offen uniform:** je k Mehrfachknoten an beiden Enden, z. B. $[0, 0, 0, 1, 2, 3, 3, 3]$ für $k = 3$. Die Kurve interpoliert den ersten und letzten Kontrollpunkt (wie Bézier).
- **Nichtuniform:** ungleiche Abstände oder Mehrfachknoten im Inneren – erlaubt gezielte lokale Formgebung oder Knicke.

	Periodisch uniform	Offen uniform
Anzahl Knoten	$n + k$	$n + k$
Parameterbereich	$[k - 1, n]$	$[0, n - k + 1]$
Endpunkte interpoliert	nein	ja

Geschlossene Splines: Um eine B-Spline-Kurve zu schließen, periodischen Knotenvektor verwenden und $k - 2$ Kontrollpunkte vom Anfang am Ende wiederholen.

Hinweis

Spezialfall: B-Spline = Bézier Ist die Ordnung k gleich der Anzahl der Kontrollpunkte n und wird ein offener Knotenvektor verwendet, reduziert sich die B-Spline-Basis zur **Bernstein-Basis** – die B-Spline-Kurve wird zur Bézier-Kurve.

8.7 Datenstrukturen für Netze

Polygonale Netze speichern sowohl **Geometrie** (Koordinaten) als auch **Topologie** (Nachbarschaftsbeziehungen). Es gibt vier wesentliche Verfahren:

1. Zellinzidenzgraph

Umfassende topologische Repräsentation als Graph $I = (V, E)$. Die Knotenmenge V ist nach Dimension aufgeteilt: V_0 = Eckpunkte, V_1 = Kanten, V_2 = Flächen. Kanten im Graphen verbinden nur Knoten benachbarter Dimensionen – ein Knoten $v \in V_i$ ist mit $w \in V_{i+1}$ verbunden, wenn v eine Randzelle von w ist. Zusätzlich gibt es Hilfsknoten V_{-1} (verbunden mit allen Eckpunkten) und V_{d+1} (verbunden mit allen Flächen).

2. Winged-Edge-Datenstruktur

Drei Klassen: **Vertex**, **Face**, **Edge**. Das Herzstück ist die Kante – sie speichert:

- Start- und Endknoten ($VStart$, $VEnde$)
- linke und rechte angrenzende Fläche ($FLinks$, $FRechts$)
- jeweils Vorgänger- und Nachfolgerkante links und rechts ($PELinks$, $PERechts$, $NELinks$, $NERechts$)

Vertex und Face speichern nur einen Verweis auf *irgendeine* ihrer inzidenten Kanten. Navigation durch „Weiterhangeln“ über Nachbarkanten – linearer Zeitaufwand.

3. Halbkantendatenstruktur (Half-Edge)

Reduzierte Version der Winged-Edge. Jede Kante wird in **zwei gerichtete Halbkanten** aufgeteilt – eine pro angrenzender Fläche. Jede Halbkante speichert:

- Zielknoten
- zugehörige Fläche
- nächste Halbkante im Flächenumlauf (*next*)
- Gegenkante in Gegenrichtung (*twin/opposite*)

Vertex speichert eine ausgehende Halbkante, Face speichert eine ihrer Halbkanten. Effiziente Nachbarschaftsabfragen durch Hopsen über *next* und *twin*. Bekannte Implementierung: **OpenMesh**.

4. Facetten-Knoten-Inzidenzliste

Einfachste Struktur – zwei Listen:

- **Punktliste:** Koordinaten jedes Eckpunkts (einmal gespeichert).
- **Facettenliste:** Pro Fläche die Indizes ihrer Eckpunkte, z. B. „1 2 3“.

Keine explizite Topologie – Nachbarschaft muss bei Bedarf berechnet werden. Hauptanwendung: **Dateispeicherung**. Bekannte Formate: **STL** (3D-Druck), **Geomview**.

Hinweis			
	Struktur	Topologie	Anwendung
Vergleich	Zellinzidenzgraph	vollständig	theoretische Basis
	Winged-Edge	vollständig	Modellierung
	Half-Edge	vollständig	Modellierung (Standard)
	Facetten-Knoten-Liste	keine	Dateiaustausch

8.8 Simplex, Triangulierung und polyedrische Komplexe

Simplex

Ein **Simplex** ist der einfachste geometrische Baustein in d Dimensionen: die **konvexe Hülle** von $d + 1$ Punkten, die nicht alle in einer Hyperebene liegen.

$$\begin{aligned}
 d = 1 & \quad \text{Strecke} \\
 d = 2 & \quad \text{Dreieck} \\
 d = 3 & \quad \text{Tetraeder}
 \end{aligned}$$

Triangulierung

Für eine Punktmenge $\{p_1, \dots, p_n\} \subset \mathbb{R}^d$ ist eine **Triangulierung** eine simpliziale Zerlegung der konvexen Hülle dieser Punkte – das Ergebnis ist ein lückenloses Netz aus Simplexen, das genau die gegebenen Punkte als Eckpunkte verwendet.

Polyedrischer Komplex

Ein **d -dimensionaler polyedrischer Komplex** ist eine endliche Menge von Polyedern mit zwei Bedingungen:

1. **Durchschnittseigenschaft:** Der Schnitt zweier Polyeder ist entweder leer oder ein gemeinsames **Seitenpolyeder** beider.
2. **Vollständigkeit:** Mit jedem Polyeder gehören auch alle seine Seitenpolyeder zum Komplex.

Sind alle Elemente Simplexe, heißt der Komplex **simplizialer Komplex**. Ein typischer Fehler ist die sogenannte **T-Stelle** (hängender Knoten): ein Punkt liegt im Inneren einer Kante eines anderen Simplex – der Durchschnitt ist dann kein gemeinsames Seiten-Simplex, was die Bedingung verletzt.

Kurz gefasst:

- Ein **Simplex** ist die einfachste mögliche Figur in jeder Dimension: Strecke (1D), Dreieck (2D), Tetraeder (3D). Allgemein: $d + 1$ Punkte, die nicht alle in einer Hyperebene liegen.
- Eine **Triangulierung** füllt den Raum zwischen gegebenen Punkten lückenlos und überschneidungsfrei mit Simplexen – in der Praxis: Oberfläche als Dreiecksnetz.
- Ein **polyedrischer Komplex** ist eine Sammlung von Vielecken/-flächern mit zwei Regeln: (1) Schnittmengen zweier Elemente sind immer gemeinsame Seiten, nie zufällige Stücke; (2) alle Seiten eines Elements gehören ebenfalls zum Komplex.
- **T-Stelle:** Ein Punkt liegt mitten auf einer Kante des Nachbarn – verletzt Regel (1) und ist in 3D-Netzen verboten.

8.9 Polygontriangulierung

Zerlegung eines einfachen Polygons in Dreiecke durch nicht-schneidende Diagonalen. Notwendig, da Grafik-Algorithmen (Verrasterung, Beleuchtung) primär auf Dreiecken arbeiten.

Ohrenabschneiden (Ear Clipping)

Ohr = drei aufeinanderfolgende Eckpunkte, die ein Dreieck bilden, das vollständig im Inneren des Polygons liegt und keinen nichtkonvexen Eckpunkt enthält.

Kernaussage: Jedes einfache Polygon hat mindestens ein Ohr.

Algorithmus:

1. Alle konvexen und nichtkonvexen Eckpunkte identifizieren.
2. Für jeden Punkt prüfen: bildet er mit Vorgänger und Nachfolger ein Ohr?
3. Ohr gefunden $\rightarrow$ Diagonale einfügen, Dreieck entfernen, Konvexität der Nachbarn aktualisieren.
4. Wiederholen bis nur noch ein Dreieck übrig.

Zeitaufwand: $O(n^3)$ trivial; $O(n^2)$ mit Algorithmus von Kong; $O(n \cdot k)$ wobei k = Anzahl nichtkonvexer Ecken.

Triangulierung monotoner Polygone

Ein Polygon heißt **y-monoton**, wenn jede horizontale Linie es in höchstens zwei Punkten schneidet. Solche Polygone lassen sich durch einen **Sweep-line-Algorithmus** effizienter triangulieren: Punkte nach y sortieren, mit einem Stack abarbeiten und Diagonalen zu bereits besuchten Punkten ziehen.

8.10 Voronoi-Diagramm

Für eine Punktmenge $P = \{p_1, \dots, p_n\}$ enthält die **Voronoi-Zelle** $V(p_i)$ alle Punkte der Ebene, die näher an p_i liegen als an jedem anderen Punkt aus P .

Eigenschaften:

- Voronoi-Zellen sind immer **konvex**.
- **Empty Circle Property:** Jeder Voronoi-Knoten ist Mittelpunkt eines leeren Kreises durch genau drei Punkte aus P ; jeder Punkt auf einer Voronoi-Kante liegt auf einem leeren Kreis durch zwei Punkte.
- Komplexität: $n_v \leq 2n - 5$ Knoten, $n_e \leq 3n - 6$ Kanten.
- Eulerformel: $(n_v + 1) - n_e + n = 2$.

8.11 Delaunay-Triangulierung

Ein Simplex erfüllt die **Delaunay-Bedingung**, wenn sein Umkreis (2D) bzw. seine Umkugel (3D) keinen anderen Punkt der Menge im Inneren enthält.

Eigenschaften:

- **Dualität:** Delaunay-Graph = dualer Graph zum Voronoi-Diagramm. Zwei Punkte sind verbunden $\Leftrightarrow$ ihre Voronoi-Zellen teilen eine Kante.
- **Eindeutigkeit:** Existiert immer; eindeutig wenn keine $d + 2$ Punkte kosphärisch (auf einer gemeinsamen Kugeloberfläche).
- **Winkelmaximierung (2D):** Unter allen Triangulierungen maximiert Delaunay den kleinsten Winkel – keine unnötig spitzen Dreiecke.
- **Berechnung:** Punkte auf Paraboloid projizieren: $T(p) = (p, \sum p_i^2) \in \mathbb{R}^{d+1}$, konvexe Hülle berechnen. Die von **unten** (aus Richtung der negativen $(d+1)$ -Achse) sichtbaren Facetten ergeben bei Rückprojektion die Delaunay-Triangulierung.

Hinweis

Klausur: verletzte Delaunay-Bedingung Liegt ein Punkt im Inneren des Umkreises eines Dreiecks, ist die Delaunay-Bedingung verletzt. Heilmittel: **Kantenflip** (Edge Flip) – die gemeinsame Kante der beiden betroffenen Dreiecke durch die andere Diagonale ersetzen.

8.12 Beneath-Beyond-Algorithmus

Inkrementeller Algorithmus zur Konstruktion der **konvexen Hülle**, Grundlage für die Delaunay-Berechnung.

Vorgehen:

1. Punkte **lexikographisch sortieren**.
2. Punkte nacheinander zur bestehenden Hülle hinzufügen:
 - **Pyramidal:** Punkt liegt außerhalb der affinen Hülle $\rightarrow$ Pyramide mit bisheriger Hülle als Basis und p_i als Spitze.
 - **Nichtpyramidal:** Punkt liegt im gleichen Raum $\rightarrow$ alle von p_i sichtbaren Facetten entfernen, p_i mit den Silhouettenfacetten verbinden.

Zeitaufwand: $O(n \log n)$ (dominiert durch die Sortierung).

8.13 Netzverbesserung

Qualitätskriterien

- **Geometrische Qualität:** ausgeglichene Dreiecke (kein langes, dünnes Dreieck).
- **Kombinatorische Qualität:** möglichst regulärer Knotengrad – Zielwert bei Dreiecksnetzen: **6**.
- **Flächenqualität:** Glattheit und Ausgeglichenheit (Krümmungsfunktionale).

Lokale Operatoren

- Knotenentfernung / -einfügung
- Halbkantenkontraktion / Vertex Split
- Kantenkontraktion / Edge Split
- **Kantenflip (Edge Flip):** korrigiert Knotengrad oder stellt Delaunay-Eigenschaft wieder her.
- **Knotenkontraktion:** einzige Operation, die die **Topologie** verändern kann.

Umvernetzungsalgorithmus (Kobbelt et al.)

1. **Kurze Kanten entfernen:** Kanten kürzer als $\varepsilon_{\min}$ per Halbkantenkontraktion kollabieren; Knoten mit geringerem Grad wird in den mit höherem Grad verschoben.
2. **Lange Kanten unterteilen:** Kanten länger als $\varepsilon_{\max}$ durch neuen Mittelpunkt halbieren.
3. **Knotengrad optimieren:** Kautentausche (Edge Flips) minimieren die Summe der Abweichungen vom Zielgrad 6.
4. **Iteration** bis alle Kanten im Intervall $[\varepsilon_{\min}, \varepsilon_{\max}]$ liegen.

Glättung (Denoising)

Laplace'sche Glättung: Knoten werden in Richtung des diskreten Laplace-Operators verschoben – eliminiert Rauschen aus der Abtastung. *Nachteil:* Objekt kann schrumpfen (z. B. Torus); wird durch Dämpfungsfaktor t gemindert.

*Laplace-Operator, Glättungsformel und diskreter Informationsfluss sind **nicht prüfungsrelevant**.*

9 G – Interaktionsgestaltung

9.1 Interaktionsstile

Ein Interaktionsstil beschreibt die Art des Dialogs zwischen Mensch und System.

Kommandos (CLI): Anweisungen als Zeichenfolgen/Abkürzungen. Erfordert aktives Abrufen von Syntax aus dem Gedächtnis. *Vorteil:* effizient für Experten, Skripting möglich. *Nachteil:* hoher Lernaufwand, hohe Fehlerquote bei Gelegenheitsnutzern.

Dialoge und Formulare: Vordefinierte Schritte für zusammengesetzte Aufgaben. Widgets: Textfelder, Checkboxes, Radiobuttons (Einfachauswahl), Listboxen. **Modal** = muss abgeschlossen werden bevor weiteres möglich ist; **nicht-modal** = kann jederzeit unterbrochen werden.

Direkte Manipulation (DMI): Visuelle Objekte werden direkt mit Eingabegeräten beeinflusst. WYSIWYG, sofortiges Feedback, umkehrbare Schritte. *Nachteil:* kein Geschwindigkeitsvorteil für Profis, Icons können mehrdeutig sein.

Sprachbasierte Interaktion: Kommandosprachen (SQL, Shell) oder natürliche Sprache. Natürliche Sprache: geringer Lernaufwand, aber anfällig für Mehrdeutigkeiten.

Gestenbasierte Interaktion: Posen (statische Haltungen) und **Gesten** (Bewegungen). Stile: deiktisch (zeigen/auswählen), manipulativ (Objekte steuern), symbolisch (kodierte Zeichen).

Suche, Browsen, interaktive Visualisierungen: Exploration großer Datenmengen; z. B. Paperlens für Forschungsdaten.

9.2 Gestaltprinzipien der visuellen Wahrnehmung

Prägnanzgesetz (Gesetz der guten Gestalt): Menschen bevorzugen klare, geordnete Formen – sie werden leichter wahrgenommen, besser behalten und schneller verarbeitet.

Spezielle Gestaltgesetze:

- **Gesetz der Nähe:** Nah beieinanderliegende Elemente werden als zusammengehörig wahrgenommen.
- **Gesetz der Ähnlichkeit:** Ähnliche Elemente (Form, Farbe, Größe) werden als Gruppe interpretiert.
- **Korrespondierende Muster:** Symmetrische oder sich ergänzende Elemente (z. B. Klammern, ») werden als Einheit gesehen.

Kanizsa-Dreieck: Beispiel für **subjektive Konturen** – das Gehirn ergänzt physikalisch nicht vorhandene Linien, um eine geschlossene Form wahrzunehmen.

Anwendung im Design: Gestaltgesetze reduzieren Fehler, erhöhen Verarbeitungsgeschwindigkeit und schaffen intuitive funktionale Gruppierungen.

9.3 Sensomotorische Designanforderungen

Grundlagen sensomotorischer Handlungen

Gesamtzeit = **Reaktionszeit** + **Bewegungszeit**. Reaktionszeit: auditiv ≈ 150 ms, visuell ≈ 200 ms, Schmerz ≈ 700 ms. Bewegungszeit hängt von Entfernung und Zielgröße ab.

Fitts' Law – Positionierung

$$MT = a + b \cdot \log_2 \left(1 + \frac{D}{W} \right)$$

D = Entfernung zum Ziel, W = Zielbreite. Ziele müssen **groß** und **nah** sein für schnelle Interaktion.

Steering Law – Tunnelbewegung

$$MT = a + b \cdot \frac{D}{W}$$

Beschreibt die Zeit, sich durch einen engen Pfad zu bewegen (z. B. horizontale Bewegung durch ein Untermenü). Linearer Anstieg mit Länge/Breite-Verhältnis.

Kombination im Menü-Design: Vertikale Auswahl im Obermenü → Fitts' Law; horizontales Traversieren ins Untermenü → Steering Law.

Hick's Law – Entscheidungszeit

$$RT = a + b \cdot \log_2(n)$$

n = Anzahl gleich wahrscheinlicher Alternativen; $b \approx 0,155$ s. **Konsequenz:** Flache Menüs sind schneller als tiefe Hierarchien. Beispiel: ein Menü mit 8 Einträgen ($\log_2 8 = 3$) schlägt zwei Ebenen à 4 Einträge ($2 \cdot \log_2 4 = 4$).

9.4 WIMP-Systeme

WIMP = **W**indows, **I**cons, **M**enus, **P**ointers. Verwaltet den Grafikbildschirm und bietet geräteunabhängigen Zugriff auf Ein-/Ausgabe über eine API.

Fenster-Funktionalitäten

1. Verschiebbar und skalierbar (manuell oder per Fenstermanager: gekachelt / kaskadierend).
2. Überlappbar.
3. Ikonifizierbar (auf Icon verkleinern).
4. Vordergrund/Hintergrund-Wechsel (Fokus).
5. Scrolling bei größerem Inhalt.
6. **Datenübertragung:** Clipboard (Kopieren) oder **OLE** (Object Linking & Embedding – Änderungen am Original wirken sich auf alle Referenzen aus).
7. **Koordination:** synchronisiertes Scrollen, hierarchisches Browsen, virtuelle Desktops.

Piktogramme (Icons)

Selektierbar, aktivierbar (öffnet z. B. Fenster), per Drag-and-Drop bewegbar. Gestaltung: einheitlicher Stil, klare Funktionstrennung, hoher Kontrast, Normen beachten (DIN-ISO/IEC 11581-1).

Metaphern

Übertragung bekannter Konzepte auf das System, um das mentale Modell des Nutzers zu unterstützen:

- **Desktop:** Ordner, Papierkorb, Posteingang.
- **Buch:** Seiten, Kapitel, Inhaltsverzeichnis.
- **Atlas:** Karten, Legenden, Zoom.

9.5 Zeigegeräte

Direkt vs. indirekt:

- **Direkt:** Benutzer zeigt unmittelbar auf das Display (z. B. Touchscreen).
- **Indirekt:** Bewegung findet woanders statt und wird transformiert (z. B. Maus, Trackball, Joystick).

Absolut vs. relativ:

- **Absolut:** Position des Geräts ist fest an ein Koordinatensystem gebunden – Cursor springt immer zur korrespondierenden Stelle (z. B. Grafiktablett).
- **Relativ:** Cursor bewegt sich relativ zu seiner aktuellen Position – Abheben und Wiederaufsetzen der Maus ändert den Cursor nicht.

Control/Display-Ratio (C/D-Ratio): Verhältnis zwischen physischer Handbewegung und Cursorbewegung. Großes C/D → Cursor bewegt sich langsam → hohe Präzision. **Adaptives C/D:** bei schneller Bewegung großes Verhältnis (weite Strecken), bei langsamer Bewegung kleines Verhältnis (Feinpositionierung).

9.6 Dialoge und Formulare

Dialoge lösen zusammengesetzte Interaktionsaufgaben; **Formulare** sind Dialoge mit Fokus auf Texteingabe.

Widgets:

- Dialogkopf (Titel), Textfelder, Rollbalken, Buttons (OK / Abbrechen / Übernehmen)
- **Togglebutton:** Zustand setzen/zurücksetzen; **Radiobutton:** genau eine Option aus einer Gruppe
- **Listbox:** Listenauswahl; **Combobox:** ausklappbare Liste + Textfeld
- **Paletten:** grafische Auswahl bei sich ausschließenden Optionen
- Numerische Eingabe: Pfeiltasten-Felder, **Slider**, analoge Anzeigen
- **Tabs/Karteikarten:** logische Gruppierung zusammengehöriger Elemente

Modalität:

- **Modal:** muss abgeschlossen werden bevor weitere Aktionen möglich sind.
 - *Applikationsmodal:* blockiert nur die aktuelle Anwendung.
 - *Systemmodal:* blockiert das gesamte System (z. B. kritische Fehlermeldung).
- **Nicht-modal:** kann jederzeit unterbrochen werden.

9.7 Fat Finger Problem

Zwei Kernprobleme bei Touch-Interfaces:

- **Verdeckungsproblem:** Finger verdeckt das Zielobjekt während der Interaktion.
- **Präzisionsproblem:** Fingerabdruck ist flächig und winkelabhängig – exakter Berührungspunkt ist schwer zu definieren.

Lösungen für Verdeckung:

- Große Bedienelemente (iOS/Android: 7–9 mm, Mindestabstand 2 mm).
- **SHIFT-Technik:** Inhalt unter dem Finger wird oberhalb als Kopie eingeblendet; Bestätigung erst beim Abheben.
- **Buchstabenvergrößerung** bei virtuellen Tastaturen.
- Eingabe auf der **Geräterückseite** mit Cursor-Einblendung vorne.

Lösungen für Präzision:

- **Cursor Offset:** Cursor wird leicht oberhalb des Berührungspunkts angezeigt.
- Vergrößerte sensitive Fläche (größer als visuelle Darstellung).
- **Swype-Tastatur:** kontinuierliche Wischbewegung statt einzelner Tastenanschläge – Wörterbuch erkennt den Pfad.
- **Sliding Widgets:** kleine Elemente per Wischgeste in vorgegebener Richtung aktivieren.
- **Adaptives C/D bei Slidern:** Finger weit vom Zentrum → großes C/D → Feineinstellung.

Jede Berührung soll sofort **Feedback** auslösen (visuell, akustisch oder haptisch).

10 H – Virtuelle und Augmentierte Realität

10.1 Taxonomie: Extended Reality (XR)

Milgram's Mixed Reality Continuum: Beschreibt den kontinuierlichen Übergang von komplett real bis komplett digital:

$$\underbrace{\text{Real Reality}}_{\text{komplett real}} \longrightarrow \text{AR} \longrightarrow \text{AV} \longrightarrow \underbrace{\text{VR}}_{\text{komplett digital}}$$

- **AR (Augmented Reality):** virtuelle Inhalte in reale Umgebung eingeblendet.
- **AV (Augmented Virtuality):** reale Inhalte in virtuelle Umgebung eingebettet.
- **VR (Virtual Reality):** reale Welt vollständig durch digitale Umgebung ersetzt.

- **XR (Extended Reality):** Oberbegriff für alle Formen – VR, AR, MR. Interaktive, in Echtzeit angereicherte Wahrnehmung.

Multimediated Reality (All R): Erweiterung der XR – die umfassendste Form. Realität wird nicht nur ergänzt, sondern **aktiv modifiziert**. Merkmale:

- Multimediabasiert, multidimensional, multi-sensorisch/multi-aktuatorisch.
- Schließt allgemeine Sensoren, menschliche Aktionen und Aktuatoren ein.
- Variable Sensorik und Aktorik: visuelle, auditive, haptische Kanäle kombiniert.
- Keine klare Trennung mehr zwischen realen und virtuellen Sinneseindrücken möglich.

Anforderungen an XR-Systeme:

- **Kombination:** bewusste Vermischung von Realität und Virtualität.
- **4D-Einbettung:** virtuelle Inhalte realitätsgetreu und zeitlich konsistent eingebettet.
- **Echtzeit-Interaktion:** unmittelbare Reaktion ohne wahrnehmbare Verzögerung.

10.2 Immersion, Präsenz und Illusion

Immersion (technisch) – messbares physisches Eintauchen durch multimodale Darstellung (visuell, auditiv, haptisch): Auflösung, FoV, Latenz.

Präsenz (kognitiv) – das psychologische Gefühl, tatsächlich in der virtuellen Welt zu sein („being there“).

Die drei Illusionen

Präsenz entsteht durch das Zusammenwirken von drei Illusionen:

1. **Ortsillusion (Place Illusion):** Das Gefühl, an einem anderen Ort zu sein – erzeugt durch realistische Materialien, Lichtsimulation und blickrichtungsabhängige Darstellung.
2. **Plausibilitätsillusion:** Das Gefühl, dass die Ereignisse tatsächlich passieren – erfordert natürliche, erwartungskonforme Interaktionen.
3. **Involviertheit (Involvement):** Grad der Aufmerksamkeit und des Engagements des Benutzers.

I³-Dreieck (Virtual Reality Triangle)

Drei Kernkomponenten eines XR-Systems:

- **Immersion** – technische Darstellungsqualität durch multimodale Reize (visuell, auditiv, haptisch/kinästhetisch).
- **Interaktion** – Bewegung, Exploration, Echtzeit-Manipulation; einfache und intuitive Nutzbarkeit; **echtzeitnahe Systemreaktion** ohne spürbare Verzögerung.
- **Imagination** – psychologisches Präsenzgefühl, unterstützt durch realistische 3D-Darstellung und erwartungskonforme Eingabe.

Uncanny Valley

Akzeptanz eines Avatars steigt mit zunehmendem Realitätsgrad – bis zu einem Schwellenwert, ab dem sie **rapide abfällt** (der Avatar wirkt unheimlich). Erst bei nahezu perfekter Übereinstimmung mit einem Menschen steigt die Akzeptanz wieder an.

Bewegung verstärkt den Effekt massiv: Eine fast-menschliche Figur, die sich bewegt (z. B. Zombie, humanoider Roboter), löst deutlich stärkeres Unbehagen aus als dieselbe unbewegt. Beispiele auf der Kurve:

- *Hohe Akzeptanz:* Industrieroboter, Stofftiere, humanoide Roboter (vor der Schwelle).
- *Das Tal:* Leichen, Zombies, realistische Prothesen mit kleinen Defiziten.
- *Rückkehr:* gesunde echte Menschen.

10.3 VR-Lokomotion

Navigation = **Reisen** (motorisch, Bewegung durch den Raum) + **Wegsuche/Wayfinding** (kognitiv, Orientierung und Planung).

Typ	Bewegung	Raum	Beispiele
Physisch, bewegungsbasiert	kontinuierlich	offen	Walking-in-place, Redirected Walking
Physisch, raumskalenbasiert	kontinuierlich	begrenzt	Real Walking
Künstlich, controller-basiert	kontinuierlich	offen	Joystick, kopfgesteuert
Künstlich, teleportationsbasiert	diskontinuierlich	offen	Point & Teleport

Redirected Walking: realer Pfad wird unbemerkt manipuliert, um einen größeren virtuellen Raum mit begrenztem physischem Platz zu simulieren.

10.4 Tracking und Detektion

Physikalische Wirkprinzipien

- **Aktiv:** künstliche Signalquellen (Emitter) – mechanisch, magnetisch, Ultraschall, GPS, WiFi.
- **Passiv:** keine eigenen Emitter – Inertialsensoren (Gyrometer), Computer Vision.
- **Hybrid:** Kombination mehrerer Methoden (z. B. Bild + Inertial) für höhere Genauigkeit.

Gerätetypen

- **Mechanisch:** physische Verbindung → sehr präzise, aber begrenzter Bewegungsradius.
- **Magnetisch:** misst Lage im Magnetfeld. Nachteil: anfällig für Metallgegenstände.
- **Inertialsensoren:** Gyroskop + Beschleunigungsmesser. Geringe Latenz, klein – aber hoher **Drift** (kumulierende Abweichung).
- **Optisch:**

- *Marker-basiert*: Infrarot-Marker am Körper/Gerät – robust, einfach zu berechnen.
- *Markerlos*: natürliche Merkmale der Umgebung.
- **Distanzsensoren**: Tiefenbild (z-Bild) via Time-of-Flight, Laser oder strukturiertes Licht (z. B. Kinect).

Tracking-Ansätze (Software)

Merkmalsbasiert:

- **SLAM** (Simultaneous Localization and Mapping): gleichzeitig Umgebung kartieren und eigene Position bestimmen.
- **SfM** (Structure from Motion): 3D-Strukturen aus Bewegungssequenzen ableiten.
- **Optisches Tracking**: kantenbasiert, vorlagenbasiert, Interest-Point-basiert.

Deep Learning:

- **CNNs** (Convolutional Neural Networks): lernen hierarchisch Merkmale aus Bilddaten – Convolution → ReLU → Pooling → semantische Merkmale. End-to-End-Training statt manueller Merkmalsextraktion.
- **Skelett-basierte Aktionserkennung**: Keypoints (Gelenkpunkte) detektieren, dann **GCNs** (Graph Convolutional Networks) für spatio-temporale Graphen (Posen, Bewegungssequenzen).
- **Inverses Rendering**: aus Bildern Materialeigenschaften (Normalen, Rauheit) schätzen.

Modell-basiertes Tracking: CAD-Modell des Zielobjekts gegen Kamerabild abgleichen (Kanten oder Tiefendaten).

Detektionsaufgaben

1. **Posen- und Gestenerkennung** – Skelett-basierte Körperhaltungsidentifikation.
2. **Aktionserkennung** – Bewegungsabfolgen interpretieren (gehen, laufen, ...).
3. **Dynamische 3D-Modelle** – Echtzeit-Abbilder mitbewegter Objekte.

10.5 Adaptive Streaming für XR

Grundidee: Datendurchsatz dynamisch an verfügbare Bandbreite anpassen. Basis: **DASH** (Dynamic Adaptive Streaming over HTTP) – Inhalte liegen in mehreren Qualitätsstufen vor, Client wählt je nach Pufferfüllung und Bandbreite.

XR-spezifische Varianten:

- **Tile-basiert**: Bild in räumliche Kacheln unterteilen – nur Tiles im aktuellen Sichtfeld in hoher Qualität übertragen.
- **Viewport-adaptiv**: Kopfbewegung vorhersagen (ggf. per Deep Reinforcement Learning) → Kompressionsstufen an erwarteten Blickwinkel anpassen.

- **Foveated Rendering:** Nur der fixierte Bereich (Fovea) wird hochaufgelöst gerendert; Peripherie bleibt unscharf. Erfordert Eye-Tracking und sehr geringe Motion-to-Photon-Latenz. Umsetzung: **Variable Rate Shading (VRS)**.

Standards: MPEG-I (ISO/IEC 23090) – OMAF (360°), VVC/H.266 (4K/8K/VR), Punktwolken-Kompression.

10.6 Interaktionsaufgaben in virtuellen 3D-Umgebungen

Navigation

Traveling (motorisch: Fortbewegung) + Wayfinding (kognitiv: Orientierung, Planung).
Aufgaben: Exploration, gezielte Suche, Manövrieren.

Selektion und Manipulation

Direkte Techniken:

- **Virtuelle Hand:** getrackte Hand greift direkt (Datenhandschuh, Leap Motion).
- **Go-Go:** virtueller Arm wächst überproportional – ermöglicht weit entfernte Objekte zu greifen.
- **Selektionsstrahl/-kegel:** Strahl/Kegel vom Nutzer, Objekte werden per Rang-Metrik ausgewählt (Taschenlampen-Metapher).

Indirekte Techniken: 3D-Widgets (Griffe, Rahmen) an Objekten für Präzisionssteuerung.

Systemsteuerung

Änderung des Systemzustands: Sprachkommandos, Gesten, virtuelle Werkzeuge. **Pie-Menüs:** kreisförmige 3D-Menüs, per Handrotation oder Selektionsstrahl bedient.

2D-Interaktion in 3D-Umgebungen

- **Voll-immersiv:** physische Eingabefläche für Nutzer unsichtbar, Darstellung in VR projiziert.
- **Semi-immersiv:** transparentes Zeichenbrett mit UI-Einblendung an korrekter Stelle (z. B. System *Studierstube*).

10.7 Haptische 3D-Umgebungen

Haptische Systeme übertragen Kräfte (Force Feedback) via Aktuatoren – z. B. Simulation des Greifens. Virtuelle Hand wird durch Kontrollpunkte abstrahiert, Bewegungen folgen mechanischen Gesetzen.

Haptische Interaktionsschleife

Position des virtuellen Manipulators bestimmen → Szene aktualisieren (Kollisionsbehandlung) → Reaktionskraft ans Eingabegerät zurückgeben. Diese Schleife muss mit sehr hoher Rate (~ 1 kHz) laufen, da der Tastsinn sehr empfindlich auf Latenz reagiert.

Kollisionsdetektion

Bestimmt, ob und wie der Manipulator ein Szenenobjekt berührt. Drei Dimensionen der Fallunterscheidung:

1. Anzahl der Körper

- **Paarweise:** Wechselwirkung zwischen zwei komplexen Objekten (Manipulator $\leftrightarrow$ Szene).
- **n -Körper:** viele einfache Objekte (z. B. Partikel) – quadratische Komplexität ohne Optimierung.

2. Bewegungszustand

- **Statisch:** beide Körper in Ruhe.
- **Dynamisch:** beide Körper in Bewegung.
- **Pseudo-dynamisch:** Näherung – dynamische Bewegung als Folge sehr kleiner statischer Schritte.

3. Materialeigenschaften

- **Rigid (starr):** keine Verformung bei Kollision.
- **Deformable (weich):** Objekte ändern ihre Form – deutlich aufwändiger.

Geometrische Kernaufgaben:

- **Schnitt-Test:** Existenz einer Kollision feststellen (schneiden sich zwei Dreiecke?) oder alle schneidenden Dreieckspaare ausgeben.
- **Abstandsberechnung:** minimaler Abstand zwischen Manipulator und Szene – Grundlage für Kraftberechnung.
- **Kollisionszeitpunkt:** exakter Zeitpunkt des Erstkontakts.

Suchstrukturen (z. B. OBB-Bäume, kd-Bäume) reduzieren die Anzahl nötiger Schnitttests: Zunächst schnelle Grobprüfung auf Hüllkörper-Ebene, dann nur bei Überlappung Feinprüfung auf Dreiecksebene.

11 I – Eingabesensorik

Die Kapitel D bis F behandeln den Weg vom Modell zum Bild. Kapitel I behandelt die Gegenrichtung: vom physikalischen Signal zurück zu einer Bedeutung, die das System verarbeiten kann. Das ist der Zweig *Informationsaufnahme* und *Analyse* des Interaktionsmodells aus Kapitel B.

11.1 Sensoren und Eingabegeräte

Definition

Ein **Sensor** ist ein Gerät zur Erfassung physikalischer Signale der Umwelt. Ein **Eingabegerät** ist eine Kombination von Sensoren zur Erfassung von Information vom Anwender des Systems.

Sensortyp	Beispiele
elektrooptisch	Fotozelle, CCD-Chip
elektromechanisch	Potenziometer, Schalter, Dehnstreifen, Drehmomentsensor
elektromagnetisch	Antennen
elektroakustisch	Mikrofon

Handorientierte Eingabegeräte sind Tastatur, Maus (mechanisch oder optisch), Track-Ball, Joystick, Paddles, Space-Ball, Lichtgriffel, Datentablett, Tastbildschirm und Datenhandschuh. Drei davon zeigen exemplarisch, wie unterschiedlich dieselbe Aufgabe gelöst wird.

Optische Maus: eine Leuchtdiode beleuchtet die Tischfläche, das reflektierte Licht fällt auf einen CMOS-Sensor, der Bilder aufnimmt. Ein digitaler Signalprozessor extrahiert Muster aus diesen Bildern und stellt fest, wie weit sie sich von Bild zu Bild verschoben haben; aus der Verschiebung ergeben sich die neuen Koordinaten. Die Maus ist damit ein vollständiges Bildverarbeitungssystem im Kleinen.

Tastbildschirm: in der klassischen Bauform ein Rahmen mit Infrarot-Lichtquellen auf zwei Seiten und Fotosensoren gegenüber; eine Berührung unterbricht je einen horizontalen und einen vertikalen Strahl und wird dadurch lokalisiert. Alternativen sind Ultraschallwellen im Monitorglas und die kapazitive Technik mit einer Folie auf dem Bildschirm, deren bekannter Nachteil die Verschmutzung durch Fingerabdrücke ist.

Datenhandschuh: Biegesensoren erfassen die Handpostur. Ein Glasfaserkabel mit Leuchtdiode am einen und Fotozelle am anderen Ende ist dort angeraut, wo es für Verbiegung sensitiv sein soll; je nach Verbiegung ändert sich die durchgelassene Lichtmenge.

11.2 Die Verarbeitungskette der digitalen Bildverarbeitung

Stufe	Aufgabe
Bildaufnahme	Wandlung der Szene in ein Signal, mit allen Fehlern des Sensors
Bilddiskretisierung	Abtastung und Quantisierung (Kapitel C.4)
Bildrestauration	Rückrechnung der Sensorfehler nach quantitativen Kriterien
Bildverbesserung	Aufbereitung nach subjektiven Kriterien
Bildsegmentierung	Unterteilung in bedeutungsvolle Teilbereiche
Merkmalsextraktion	Zuordnung von Merkmalen zu den Teilbereichen
Klassifikation	anwendungsbezogene Zuordnung anhand der Merkmale

Hinweis

Der Unterschied zwischen **Restauration** und **Verbesserung** ist der Bewertungsmaßstab, nicht die Technik. Restauration arbeitet gegen ein Signalmodell und lässt sich messen; Verbesserung arbeitet gegen den Eindruck eines Betrachters und lässt sich nur beurteilen.

11.3 Bildrestauration

Definition

Bildrestauration ist die Verbesserung von Bildsignalen im Sinne quantitativ definierter Kriterien, im Gegensatz zu den subjektiven Kriterien der Bildverbesserung. Sie geht in zwei Schritten vor: erst wird ein **Signalmodell** definiert, das den Zusammenhang zwischen der zu erfassenden Szene und dem gewonnenen Bildsignal beschreibt, dann wird das Originalsignal näherungsweise zurückgerechnet.

Das Standardmodell ist linear und verschiebungsinvariant, mit additivem Rauschen:

$$g(x, y) = (h * f)(x, y) + r(x, y) \quad \Longleftrightarrow \quad G(u, v) = H(u, v) F(u, v) + R(u, v)$$

Dabei ist h bzw. H die **Modulationsübertragungsfunktion** mit Tiefpasscharakteristik, etwa für das Verhalten der Sensorelemente eines CCD-Chips, und r bzw. R eine Zufallsfunktion für ihr Rauschen.

H lässt sich durch Aufnahme einfacher Testbilder bestimmen. R dagegen lässt sich nicht bestimmen; abschätzbar ist allenfalls die mittlere spektrale Störleistung aus dem Leistungsspektrum einer schwarzen Aufnahme.

Das inverse Filter und warum es scheitert

Formal ist die Lösung eine Division:

$$\hat{F}(u, v) = \frac{G(u, v)}{H(u, v)}$$

Hinweis

Die Schwierigkeit: Nullstellen von $H(u, v)$ verhindern die exakte Restauration, selbst wenn das Rauschen wegfällt. Und dort, wo H zwar nicht null, aber klein ist, wird das Rauschen mit dem Kehrwert multipliziert und dominiert das Ergebnis vollständig.

Deshalb werden Näherungslösungen auf dem diskreten Bild bestimmt, mit der diskreten statt der kontinuierlichen Fouriertransformation. Drei Verfahren:

- **amplitudenbegrenztes inverses Filter:** das inverse Filter, dessen Amplitude durch eine Konstante begrenzt wird. Im Beispiel der Vorlesung, einem mit einem 5×5 -Spalttiefpass verunscharften und mit 8 Bit quantisierten Bild, erzeugt das reine inverse Filter hochfrequente periodische Störmuster; die Begrenzung auf den Wert 25 unterdrückt sie.
- **Nebenbedingungsfilter:** Restauration unter zusätzlichen Annahmen über das Ergebnis, etwa Glattheit oder Nichtnegativität.
- **Wiener-Filter:** minimiert den mittleren quadratischen Fehler und gewichtet dazu jede Frequenz mit dem dortigen Verhältnis von Stör- zu Signalleistung.

$$\hat{F}(u, v) = \frac{H^*(u, v)}{|H(u, v)|^2 + K} G(u, v)$$

Für $K = 0$ geht das Wiener-Filter exakt in das inverse Filter über. Der Term K ist genau die Information, die dem inversen Filter fehlt: wie viel von dem, was bei dieser Frequenz ankommt, überhaupt Signal ist.

11.4 Bildverbesserung

Punktoperatoren

Ein Punktoperator bildet jeden Intensitätswert unabhängig von seiner Umgebung ab. Grundlage ist das **Intensitätshistogramm** $h(f)$, das für jeden Grauwert die Anzahl seines Auftretens angibt.

Auto-Kontrast-Anpassung: zwei vordefinierte Quantile p_{lo} und p_{hi} werden auf die Randwerte des Wertebereichs gelegt, die Zwischenwerte linear angepasst. Robuster als das Strecken auf Minimum und Maximum, weil einzelne Ausreißer den gesamten Kontrast sonst bestimmen.

Histogrammglättung (Histogrammausgleich): die Transformation ist die **kumulative Verteilungsfunktion** des Histogramms, skaliert auf den Wertebereich. Das Ergebnis ist ein möglichst gleichverteiltes Histogramm, also ein Bild, in dem jede Intensitätsstufe etwa gleich häufig vorkommt.

Lokale Operatoren im Ortsbereich

Lokale Operatoren bilden die Intensitätswerte $f(i, j)$ innerhalb eines Fensters W_{mn} gemäß eines Filterkerns $O[\cdot]$ in das Ausgabebildelement $g(m, n)$ ab. Jeder neue Pixelwert ist eine Funktion der Quellpixel in einer Region $R_{u,v}$ des Originalbildes.

Gebräuchliche Fensterformen sind rechteckig, kreuzförmig und kreisförmig. Das Hintereinanderschalten mehrerer Operatoren vergrößert den Einflussbereich. Lokale Operatoren eignen sich gut für Parallelverarbeitung, weil kein Pixel vom Ergebnis eines anderen abhängt.

Der lineare Fall ist die **Faltung**. Typische Tiefpasskerne sind Rechteck (Spalttiefpass), Gaußglocke und Kegel; sie kompensieren breitbandige Störsignale aus Quanten-, Sensor-, Übertragungs- und Quantisierungsrauschen.

Hinweis

Tiefpassfilterung ist nie kostenlos: relevante Strukturen wie Kanten und dünne Linien werden verbreitert, und der lokale Kontrast sinkt. Rauschen und Kante liegen im selben Frequenzband, und ein lineares Filter kann sie nicht unterscheiden.

Das **Medianfilter** ist der wichtigste nichtlineare Operator: von den p Elementen des Operatorfensters wird der Median als Ausgabewert genommen. Es beseitigt Strukturen, die kleiner als das halbe Fenster sind, und lässt Kanten stehen, weil ein einzelner Ausreißer den Median nicht bewegt, einen Mittelwert dagegen sehr wohl.

Verarbeitung im Ortsfrequenzbereich

Nach dem Faltungssatz (Kapitel C.4) ist die Faltung im Ortsbereich eine Multiplikation im Ortsfrequenzbereich. Ein Filter wird dort direkt als Übertragungsfunktion entworfen: Tiefpass für Glättung, Hochpass für Kanten hervorhebung, Bandpass für periodische Störmuster. Der Umweg lohnt sich, sobald der Kern groß ist, weil die FFT mit $O(n \log n)$ statt $O(nk^2)$ arbeitet.

11.5 Segmentierung

Definition

Unter der **Segmentierung** eines diskreten Bildsignals $f(m, n)$ versteht man die Unterteilung von f , als Menge von Pixeln gesehen, in disjunkte, nichtleere Teilmengen $f_1, \dots, f_p$, so dass mit einem Einheitlichkeitskriterium E gilt:

- a) $\bigcup_i f_i = f$ und $f_i \cap f_j = \emptyset$ für $i \neq j$
- b) f_i ist zusammenhängend für alle i
- c) für alle f_i ist $E(f_i)$ erfüllt
- d) für die Vereinigung zweier benachbarter f_i, f_j ist $E(f_i \cup f_j)$ *nicht* erfüllt

Bedingung d) ist die eigentliche Definition: ohne sie wäre jede Zerlegung in einzelne Pixel eine gültige Segmentierung. Sie fordert, dass die Bereiche **maximal** sind.

Ein typisches Einheitlichkeitskriterium ist die Intensitätshomogenität: die Intensitätsdifferenz zweier beliebiger Bildpunkte innerhalb eines Bereichs darf betragsmäßig einen Wert ϵ nicht überschreiten. Das Ergebnis wird als Maske (Binärbild) oder als Konturlinie dargestellt.

Bereichsorientierte Verfahren

Der **einfache Schwellwertoperator** bildet auf zwei Werte I_1, I_2 ab, getrennt durch eine Intensitätsschranke s . Anwendbar, wenn sich helle Objekte auf dunklem Hintergrund befinden oder umgekehrt; s ist aus dem Histogramm bestimmbar, typisch als Tal zwischen zwei Maxima.

Der **mehrwertige Schwellwertoperator** verallgemeinert das auf p Klassen mit $s_0 = 0$ und $s_p = f_{max} + 1$. In der Praxis wird vorher mediangefiltert, weil das glättet, ohne die Kanten zu verschieben, an denen die Schwellwerte gerade greifen sollen.

Kantenorientierte Verfahren

Statt Bereiche zu finden, wird ihre Begrenzung gesucht. Grundlage ist der Gradient, genähert durch Differenzenoperatoren (Sobel, Prewitt, Roberts), oder die zweite Ableitung (Laplace), deren Nulldurchgänge die Kanten markieren.

Beide Wege haben denselben Nachteil: Differenzieren verstärkt hohe Frequenzen, und Rauschen ist hochfrequent. Deshalb wird vorher geglättet, und deshalb ist der Laplace-Operator ohne vorgeschaltete Gaußglättung praktisch unbrauchbar.

Hinweis

Bereichs- und kantenorientierte Verfahren sind dual, liefern aber nicht dasselbe: Schwellwerte erzeugen immer geschlossene Bereiche, aber möglicherweise falsche; Kantenoperatoren erzeugen korrekt lokalisierte, aber möglicherweise unterbrochene Konturen. Die Hysterese im Canny-Verfahren ist die Standardantwort auf die zweite Schwäche.

11.6 Mustererkennung

Auf die Segmentierung folgt die **Merkmalsextraktion**: jedem Teilbereich werden Kenngrößen zugeordnet, die ihn von anderen unterscheiden, etwa Fläche, Umfang, Kompaktheit, Momente, Farbstatistiken oder Texturmaße. Die **Klassifikation** ordnet die Merkmalsvektoren dann Klassen zu.

Klassisch geschieht das mit von Hand entworfenen Merkmalen und einem statistischen Klassifikator. **Neuronale Netze** verschieben diese Grenze: das Netz lernt die Merkmale selbst aus den Daten.

Modell	Kerngedanke
Multilayer Feedforward	Schichten von gewichteten Summen mit nichtlinearer Aktivierung; universeller Approximator, aber ohne jede Struktur für Bilder
Convolutional NN	die Gewichte sind Faltungskerne, dieselben an jeder Bildposition: Translationsinvarianz und drastisch weniger Parameter
Sprachmodelle (GPT)	dieselbe Idee auf Sequenzen, mit Aufmerksamkeit statt Faltung

Hinweis

Ein CNN ist nichts anderes als eine Kaskade lernbarer lokaler Operatoren aus Abschnitt I.2.4. Der Unterschied zum klassischen Filterentwurf ist nicht die Operation, sondern die Herkunft der Koeffizienten: entworfen gegen ein Modell oder angepasst an Daten.

12 J – UX/UI-Design und Entwicklungsprozesse

12.1 UX-Design: Grundbegriffe

- **Utility (Nutzen):** Hat das Produkt einen realen Nutzen für den Benutzer?
- **Usability (Benutzerfreundlichkeit):** Wie einfach, effektiv und zufriedenstellend ist die Nutzung?
- **Usefulness (Nützlichkeit):** Utility + Usability zusammen.
- **User Experience (UX):** „Summe der Interaktionsgefühle- alles, was der Nutzer vor, während und nach der Nutzung erlebt.“
- **Banding Experience:** „Summe der Gefühle“ gegenüber einer Marke – über einzelne Produkte hinaus.

UX ist breiter als Usability (ISO 9241-210): Usability betrifft nur die *Nutzungsphase* (Effektivität, Effizienz, Spaß), UX umfasst zusätzlich *vor* der Nutzung (Erwartungen, Erfahrungen) und *danach* (Zufriedenheit, emotionale Bindung).

UX vs. UI:

- **UX Design:** Gestaltung des gesamten Nutzungserlebnisses – Forschung, Szenarien, Informationsarchitektur, Interaktionsdesign.
- **UI Design:** Gestaltung der konkreten Benutzeroberfläche – Farben, Formen, Layouts, Typografie.

12.2 Benutzergerechte Gestaltung

Anhaltspunkte

- Einsatz mentaler Modelle und Metaphern zur Verständnisunterstützung.
- Nutzung der Terminologie der Benutzenden (nicht der Entwickler).
- Reduktion kognitiver Belastung.
- Klare Strukturierung der Benutzungsschnittstelle.
- Kombination visueller und textueller Elemente.
- Sichtbarkeit von Systemzuständen und möglichen Aktionen.
- Rückkopplung (Feedback) auf Nutzeraktionen.

Konsistenz

Konsistenz bedeutet, dass gleichartige Dinge gleichartig aussehen und sich gleich verhalten – reduziert Lernaufwand:

- **Sprachliche Konsistenz:** einheitliche Terminologie für Buttons, Menüeinträge, Dialoge.
- **Strukturelle Konsistenz:** konsistente Anordnung von Bedienelementen (Menüs, Toolbars).
- **Grafische Konsistenz:** einheitliche Farben, Schatten, Linienstärken, Icons.
- **Interaktionskonsistenz:** einheitliche Reaktionen auf Funktions- und Maustasten.

Adaptierbarkeit und Adaptivität

- **Erwartungskonformes Verhalten:** Dialog entspricht den Fähigkeiten und Erwartungen der Benutzenden.
- **Adaptierbarkeit:** Nutzende können die Oberfläche selbst anpassen (Grenzen vorgegeben).
- **Adaptivität:** System passt sich automatisch an den Nutzenden an.

Querschnittsaspekte: Bewusste UX-Gestaltung, Barrierefreiheit, Fokus auf Interface-Entwicklung, Beispiele zur Erklärung der Bedienung.

12.3 ISO 9241 – Ergonomie der Mensch-System-Interaktion

7 Grundsätze für benutzergerechte Gestaltung (Teil 110):

1. **Aufgabenangemessenheit:** Geeignete Funktionalität, keine unnötigen Interaktionen.
2. **Selbstbeschreibungsfähigkeit:** System erklärt sich selbst durch Hilfe und Rückmeldungen.
3. **Lernförderlichkeit:** Anleitung, Metaphern – Ziel: minimale Erlernzeit.
4. **Steuerbarkeit:** Nutzer steuert den Dialog, nicht umgekehrt.
5. **Erwartungskonformität:** Konsistenz, Anpassung ans Benutzermodell.
6. **Individualisierbarkeit:** Anpassbarkeit an Bedürfnisse und Kenntnisse.
7. **Fehlertoleranz:** System reagiert tolerant auf Fehler oder ermöglicht einfache Fehlerkorrektur.

12.4 User Interface Engineering (UIE)

UIE ist ein **phasenorientiertes Vorgehen** der Softwareentwicklung, bei dem in allen Phasen Gebrauchstauglichkeit und UX im Fokus stehen. Zentral: Bewertung von Prototypen durch Nutzer bzw. Experten und iterative Weiterentwicklung. Methoden sind stark skalierbar – auch bei begrenztem Aufwand nützlich.

Entwicklungsprozesse können verschiedene Typen haben:

- UX Experience Design
- Szenariobasierte Entwicklung
- Contextual Design
- Partizipative Entwicklung und Living Labs

Entwurfsprozess: Iterativer Zyklus – ausgehend von der Anforderungsanalyse entstehen, diskutieren und verfeinern sich Lösungsvarianten: *Verstehen* → *Prototyping* → *Überprüfen* → *Verbessern* (und zurück).

Evaluierung von Benutzungsschnittstellen – zwei Klassen:

- **Usability Inspections:** Experten bewerten Prototyp/System anhand von Richtlinien und Heuristiken.
- **Empirische Evaluierungen:** Testen mit echten Benutzern.

12.5 Design Thinking

Systematische, nutzerzentrierte Herangehensweise an komplexe Probleme. Im Zentrum stehen Nutzerwünsche und nutzerorientiertes Erfinden. Drei Komponenten des Innovations-Dreiecks:

- **Machbarkeit** (Technologie/Umsetzbarkeit)
- **Tragfähigkeit** (Wirtschaft/Marktfähigkeit)
- **Erwünschtheit** (Mensch/Nutzen)

Innovation entsteht im Schnittpunkt aller drei.

Double Diamond

Zwei Denkmodi wechseln sich ab:

- **Divergentes Denken:** offen, kreativ, viele Ideen generieren (Out-of-the-box).
- **Konvergentes Denken:** gezielt, logisch, beste Lösung herausarbeiten.

Zwei Phasen (je ein Diamant = erst divergieren, dann konvergieren):

1. **Problemphase:** Entdecken/Verstehen → Definieren (was ist das eigentliche Problem?).
2. **Lösungsphase:** Entwickeln → Finalisieren/Bereitstellen.

Design-Thinking-Prozess (HPI-Academy, 6 Stufen)

1. **Verstehen:** Problemraum abstecken.
2. **Beobachten:** Empathie aufbauen – Nutzende im Kontext erleben.

3. **Sichtweise definieren:** Erkenntnisse verdichten, Designherausforderung formulieren.
4. **Ideen finden:** Möglichst viele Lösungsideen generieren, dann fokussieren.
5. **Prototypen entwickeln:** Konkrete Lösungen greifbar machen (Papercraft, LEGO, Storyboard, Wireframe ...).
6. **Testen:** Prototypen mit Nutzern testen, Feedback für nächste Iteration sammeln.

Phasen & Methoden (Explore → Create → Prototype → Evaluate):

- *Explore:* Research Mindmap, Persona, Customer Journey, Stakeholder Map, ...
- *Create:* Brainstorming, Brain Writing, Kill your Idea, Matrix Scale, ...
- *Prototype:* Papercraft, LEGO, Storyboard, Video, Wireframing, ...
- *Evaluate:* Test Grid Planning, Interview, Brainstorming, „How might we...“, ...